

非线性报表模型原理

蒋步星 著

中国科学技术出版社

· 北 京 ·

图书在版编目(CIP)数据

非线性报表模型原理/蒋步星著. —北京:中国科学技术出版社,2007.5

ISBN 978-7-5046-4605-7

I. 非... II. 蒋... III. 计算机算法理论 IV. F231.5-39

中国版本图书馆CIP数据核字(2007)第000442号

自2006年4月起本社图书封面均贴有防伪标志,未贴防伪标志的为盗版图书。

中国科学技术出版社出版

北京市海淀区中关村南大街16号 邮政编码:100081

电话:010-62103210 传真:010-62183872

<http://www.kjpbooks.com.cn>

科学普及出版社发行部发行

北京长宁印刷有限公司印刷

*

开本:850毫米×1168毫米 1/32 印张:4.25 字数:120千字

2007年5月第1版 2007年5月第1次印刷

印数:1-600册 定价:18.00元

ISBN 978-7-5046-4605-7/O·121

内 容 简 介

本书全面系统地阐述了非线性报表模型的数学原理。从基本概念出发,逐步引出报表及相关概念的严格数学定义,然后提出能充分体现当前复杂报表内在规律的全新数学模型和相关的实用算法,并证明其合理性及分析运算复杂度。

非线性报表模型的提出为报表软件学术界引入一个新的研究课题,填补了报表软件业的一大空白,为报表软件技术的进一步发展研究奠定了理论基础。

本书主要面向计算机科学领域的研究人员和软件开发人员,具有很高的学术价值和社会意义。对于应用相关产品的用户也有重要的指导意义。

作者简介

蒋步星:非线性报表模型的主要设计者。1989 年获得国际数学奥林匹克竞赛(IMO)金牌;1996 年毕业于清华大学计算机系并获硕士学位。长期从事报表软件的研究和开发工作。现任北京润乾软件技术有限公司董事长。

前 言

以报表为载体进行数据统计与填写是信息系统中不可缺少的重要环节。

随着应用软件技术的不断发展,报表技术经历了从直接编程、工具与编程相结合、主要依靠报表工具三个不同的阶段。然而,目前业界流行的各类报表工具在设计报表时有着巨大的困难。究其原因,我们认为主要由于这些工具采用的数学模型过于简单,没有体现复杂报表的内在规律,难以方便地描述现实应用中的报表,致使报表的开发仍需大量编码。使用这些工具并不能起到提高效率的作用。

在经历了五年多的摸索和实践、研究分析了数以千计的报表实例之后,我们提出了拥有完全自主知识产权的全新理论体系,即非线性报表模型。

由于非线性报表模型充分体现了复杂报表的规律,因而基于该理论完成报表工具能够方便地描述绝大多数复杂报表,从而极大地改进报表的设计方式,使大量复杂的报表都不再依靠繁琐的代码实现,做到真正地零编码,大幅度提高了报表的设计效率,实现了报表技术的跨越性发展。

本书将从理论上系统地阐述非线性报表模型的数学原理。全书分为6章。第1章回顾和介绍需要用到的基本数学概念;第2章给出报表的严格数学定义和基本变换算法;第3章研究报表扩展变换的算法及相关性质;第4章介绍在扩展后报表中进行运算引用的方案;第5章讨论报表计算和填写的算法并分析其复杂度;第6章作一个总结和展望。

本书中仅进行纯粹的理论探讨,而不涉及软件的实现和应用(尽管高效地实现也是非常重要的)。这些理论可以很好地指导

报表工具的设计与实现。事实上,润乾报表正是一个非线性报表模型的实例产品。

最后,我要感谢润乾报表的用户们——是他们在各种复杂、关键信息系统中的实践,充分发挥了非线性报表模型的作用;而用户的需求又进一步发展丰富了模型的内容。从而使得这一全新的理论正迅速被业界认可并逐步成为报表工具的标准。

我还要感谢我在北京润乾软件技术有限公司的同事们——是他们精湛的程序实现能力,使非线性报表模型不仅存在于理论中,而是实实在在地变成了可实用的软件产品,以实例的形式充分证明了模型的可行性。

我还要感谢在学校给予我指导和交流的老师与同学们——与他们共渡的数学训练培养了我严格抽象的思维能力,而这对于非线性报表模型理论的提出和完善是至关重要的。



2007 年 5 月 8 日

目 录

1	准备	1
1.1	集合	1
1.1.1	集合	1
1.1.2	数组	2
1.1.3	变换及符号	3
1.2	函数	4
1.2.1	函数	4
1.2.2	表达式	6
1.2.3	属性	7
2	报表	10
2.1	表格	10
2.1.1	网格	10
2.1.2	表格	11
2.2	基本变换	16
2.2.1	行列增删	16
2.2.2	子表引入	20
2.3	报表	23
2.3.1	主格	23
2.3.2	扩展带	27
2.3.3	临时报表	32
3	扩展	39
3.1	扩展带复制	39
3.1.1	扩展带复制	39
3.1.2	单元格变迁	42

3.1.3	封闭性	47
3.2	主格扩展	52
3.2.1	主格扩展	52
3.2.2	交换性	57
3.3	报表扩展	62
3.3.1	报表扩展	62
3.3.2	源格	65
4	语法	72
4.1	层次坐标	72
4.1.1	基格	72
4.1.2	层次坐标	73
4.1.3	格集	78
4.2	数据源	80
4.2.1	数据集	80
4.2.2	缺省行集	83
4.3	附加引用	85
4.3.1	线性坐标	85
4.3.2	平面函数	89
5	计算	91
5.1	计算前提	91
5.1.1	报表计算	91
5.1.2	隐含引用	97
5.2	计算方法	102
5.2.1	反复遍历	102
5.2.2	单次遍历	105
5.2.3	源格遍历	108
5.3	报表填写	112
5.3.1	数据库	112

5.3.2	平面填写	114
5.3.3	线性填写	117
6	总结	121
	参考文献	124

1 准 备

1.1 集合

1.1.1 集合

作为基本概念,我们不对集合作严格定义,集合概念的研究远远超出了本文范围,这里仅对文中需要用到的术语和符号作一些回顾和约定。

一些确定的事物汇集成集合,构成集合的事物称为集合的元素或成员。我们在 $\{\}$ 中写上元素表示集合,如 $\{1,2,3,\dots\}$ 是所有自然数的集合。

设 x 为一事物, P 为一性质, $P(x)$ 表示 x 具有性质 P ;用 $\{x|P(x)\}$ 表示一切具有性质 P 的事物构成的集合。

x 是集合 X 的元素时称 x 属于 X ,记作 $x \in A$, x 不是集合 X 的元素时称 x 不属于 X ,记作 $x \notin X$ 。

用 $|X|$ 表示集合 X 的元素个数($|x|$ 同时仍是绝对值运算符),元素个数为0或自然数的集合称为有限集。本文中涉及的集合绝大多数是有限集。

没有元素的集合称为空集,即 $\{\}$,记作 ϕ ,显然 $|\phi|=0$ 。

我们有时会在不产生混淆时忽略仅有唯一元素 x 的集合 $X=\{x\}$ 与其元素 x 的差别,即用 X 表示 x 或用 x 表示 X ;在需要明确区分两者时,则用符号 ${}_1X_1$ 来表示 x 。

设有两个集合 A 与 B :

1. 若对任何 $a \in A$, 均有 $a \in B$, 则称 A 包含于 B 或 B 包含 A , 记作 $A \subseteq B$ 或 $B \supseteq A$, 同时称 A 是 B 的子集而 B 是 A 的超集;

2. 若 $A \subseteq B$ 且 $B \subseteq A$, 则称 A 与 B 相等或 A 等于 B (或 B 等于 A), 记作 $A=B$; 否则称 A 不等于 B , 记作 $A \neq B$;

显然当 $A=B$ 时, A 与 B 拥有完全相同的元素;

3. 若 $A \subseteq B$ 且 $A \neq B$, 则称 A 真包含于 B 或 B 真包含 A , 记作 $A \subset B$ 或 $B \supset A$, 同时称 A 是 B 的真子集;

4. 集合 $\{x | x \in A \text{ 或 } x \in B\}$ 称为 A 与 B 的并集, 记作 $A \cup B$;

5. 集合 $\{x | x \in A \text{ 且 } x \in B\}$ 称为 A 与 B 的交集, 记作 $A \cap B$;

6. 集合 $\{x | x \in A \text{ 且 } x \notin B\}$ 称为 A 与 B 的差集, 记作 $A \setminus B$;

除以上集合论中通行概念外, 我们再定义几个本文中将会用到的术语:

7. 若 $A \cap B \neq \emptyset$, 则称 A 与 B 相交, 否则称 A 与 B 相离;

8. 若 $A \subseteq B$ 或 $B \subseteq A$, 则称 A 与 B 相含;

9. 若 A 与 B 相交但不相含, 则称 A 与 B 交叉。

对任何两个实数 a, b , 集合 $\{x | a \leq x \leq b\} \cup \{x | a \geq x \geq b\}$ 称为从 a 到 b 的区间, 记作 $[a, b]$; 实数 $x \in [a, b]$ 时称 x 落在区间 $[a, b]$ 内。

自然数集用大写字母 N 表示。

1.1.2 数组

将有限集的元素按某种规则排序, 得到一个有先后次序的序列, 称为数组; 数组的成员个数称为数组的长度。

由于有序性, 对于数组可使用形如第 i 个成员的术语, i 是第 i 个成员的序号。与无序的集合不同, 我们在 $()$ 中按序写上成员来表示数组, 如 $(a_1, a_2, \dots, a_n)$ 。

数组的相等概念需要重新定义，称数组 $(a_1, a_2, \dots, a_n)$ 和 $(b_1, b_2, \dots, b_m)$ 相等当且仅当 $n=m$ 且 $a_1=b_1, a_2=b_2, \dots, a_n=b_m$ 。

长度为 n 的数组称为 n 元组。特别地，2 元组又称为序偶，其成员按次序分别称为左元和右元。

n 元组一般用于定义以其为元素的集合。称某集合由 n 元组构成，即表明作为该集合的元素的数组的长度均相同且为 n ，而仅称由数组构成时则不蕴涵其中元素的长度相同。

设有 n 个集合 $X_1, X_2, \dots, X_n$ ，称 n 元组构成的集合

$$\{(x_1, x_2, \dots, x_n) | x_1 \in X_1, x_2 \in X_2, \dots, x_n \in X_n\}$$

为集合 $X_1, X_2, \dots, X_n$ 的叉积或笛卡儿积，记作 $X_1 \times X_2 \times \dots \times X_n$ 。

由多个 n 元组构成的数组称为矩阵。矩阵本身及其成员都是数组，矩阵的长度称为行数，其成员的长度称为列数。矩阵的成员又称为行，行的序号称为行号，由矩阵各行中相同序号的成员以其所在行次序为次序构成的长度为矩阵行数的数组称为矩阵的列，其成员在行中的序号（都相同）称为列号。

1.1.3 变换及符号

作为算法过程的基础动作，我们定义一些集合及其元素的基本变换及符号：

1. $S += a$

若 $a \notin S$ ，则在 S 中添入 a ，如果 S 是数组，未明确指明和约定时缺省地认为 a 将被加到 S 最后。

2. $S -= a$

若 $a \in S$, 则从 S 中删除 a 。

3. $a \rightarrow b$

对上下文中所有集合 S (包括作为其他集合元素的集合, 以下同), 若 $a \in S$, 则将 S 中的 a 替换成 b ; 注意这个动作不是简单地顺序执行 $S -= a$ 和 $S += b$, S 可能是数组, 替换时要保持原有次序 (专门说明的除外)。

4. $a \rightarrow \times$

对上下文中所有集合 S 执行 $S -= a$, 若 S 是 n 元组, 则再递归地执行 $S \rightarrow \times$; 注意这里变化的是以 a 为元素的集合而非 a 本身。显然, 若上下文中的集合都是有限集且出现的集合数量有限, 该动作经过有限步后一定会停止。

注意不要将这里的变换符号 $\rightarrow$ 与通行的符号 $\Rightarrow$ 混淆, 后者将用于表示证明过程中的推出 (蕴涵)。

1.2 函数

1.2.1 函数

设有两个集合 X 和 Y , 如果按照某种规律 f , 对任何 $x \in X$, 均存在某个 $y \in Y$ 与之对应, 则称有一个定义在 X 上在 Y 中取值的函数或映射, 仍用 f 表示; 集合 X 称为 f 的定义域, X 中的元素称为 f 的自变量或参数 (本文更常用后者), 与 x 对应的 y 称为函数 f 在 x 上的返回值, 记作 $f(x)$; 集合 $\{f(x) | x \in X\}$ 称为 f 的值域, 记作 $f(X)$; 如果对任何 $x_1 \neq x_2$ 均有 $f(x_1) \neq f(x_2)$, 则称 f 是可逆函数。

当集合 X 与 Y 分别是某可逆函数 f 的定义域和值域时, 则称 X 与 Y 基于 f 一一对应, 不关心 f 时也仅称 X 与 Y 一一对应。显然两个一一对应的集合元素数量相同, 而且若集合 X 与 Y 一一对应, Y 与 Z 一一对应, 则 X 与 Z 也一一对应。

原则上,定义一个函数需要指出其定义域、取值范围(值域的某个超集)以及对应关系,但定义域和取值范围通常可从上下文中看出,这样我们只要指明函数中的对应关系即可。

定义域是 n 个集合叉积的某个子集的函数 f 被称为 n 元函数,这时我们习惯于用 $f(x_1, \dots, x_n)$ 代替 $f(x_1, \dots, x_n)$ 表示其返回值,并称 f 有 n 个参数 $x_1, \dots, x_n$ 。特别地,当 $n=1$ 时称 f 为单元函数。

某个参数总是集合(或数组)的函数(n 元)称为聚合函数。

设某有限集 $\{x_1, \dots, x_n\}$ 为单元聚合函数 f 的参数,在不与 n 元函数相混淆的情况下,我们有时会用 $f(x_1, \dots, x_n)$ 代替 $f(\{x_1, \dots, x_n\})$ 表示其返回值。注意,作为同一个聚合函数参数的有限集的元素个数不一定相同,这与 n 元函数是不同的。

设 X 是由实数构成的有限集,在其上定义常用的聚合函数:

1. $\text{sum}(X) := X$ 中所有数的和;
2. $\text{min}(X) := X$ 中最小的数;
3. $\text{max}(X) := X$ 中最大的数;
4. $\text{avg}(X) := X$ 中所有数的平均值。

这里符号 $:=$ 用于表示函数的定义,下文中也会采用该符号。

所有返回值都是数组的函数称为集合函数,定义两个常用的集合函数:

1. $\text{list}(v_1, v_2, \dots, v_n) := (v_1, v_2, \dots, v_n)$;

2. $\text{to}[q](n, m)$, 其中 n 和 m 是同一个数组 q 中的两个成员;返回值为 q 中 n 和 m 之间的成员依原序排列而成的数组,包括 n, m , q 在上下文中意义明确时, $[q]$ 部分可不写,如

$to(2001,2005)=(2001,2002,2003,2004,2005)$ 。

需要指出的是,聚合函数和集合函数是两个无关概念,聚合函数未必一定是集合函数,反之亦然。这两个概念对于报表模型的建立至关重要,集合函数是报表自动化的基础,而聚合函数则用于处理表中不可或缺的统计运算。

1.2.2 表达式

与集合类似,数据也是一个不给定义的基本概念,它能够通俗地解释为信息系统中处理的信息基本单元,原则上可以是任何事物。

每个数据均有数据类型,不同类型的数据在处理过程中允许进行的运算和遵循的规则不同,不同类型可以转换。常见的数据类型有:

1. 数: 即实数, 可进行各种数学运算;
 2. 字符串: 以字符为成员的数组, 有连接、比较等运算;
 3. 逻辑: 真假值, 可进行与、或、非等运算;
 4. 集合: 数据本身是一个集合, 可进行交、并等运算;
- 事实上, 由于数据的多样性, 数据类型也可能有无穷多种。

字符串是很重要且常用的数据类型, 为书写方便, 我们简单地将其成员顺序地写在一起, 即用 $a_1a_2...a_n$ 表示字符串 $(a_1,a_2,...,a_n)$, 有时为与其他类型数据区分还可能加上引号, 即写成 " $a_1a_2...a_n$ "。

设字符串 $A=a_1a_2...a_n$ 和 $B=b_1b_2...b_m$, 称字符串 $a_1...a_nb_1...b_m$ 为 A 与 B 的连接, 记作 $A+B$, 显然 $|A+B|=|A|+|B|$ 。

将代表数据、函数等基本项的符号用一些运算操作符号按一定规则连接起来即可获得一个表达式。在一定条件下（符号代表的数据都已确定），表达式可被计算得到一个数据，称为表达式的计算结果。

我们这里不对表达式的概念及书写规则作详细讨论（可计算理论中有专门论述），而是简单延用计算语言化的数学表达式。如 $3+5*a$ 、 $2*\ln(x)$ 等，与纯粹的数学表达式不同，这里乘号用*表示且不能省略，而函数参数一定要用括号括起来，运算符（如逻辑运算与、或、非等）则采用 C 语言的符号体系（&&,||,!等）。

表达式的计算结果不一定是个简单的数值，原则上可以是任何数据，如字符串、数组、图片等。计算结果是数组的表达式称为集合运算，集合函数即可看作是一种集合运算。计算结果是逻辑值的表达式称为逻辑表达式。

由于表达式书写时是一串连续的符号，因而可以用字符串的形式描述。内容为一个表达式的字符串称为计算串。

定义函数 $\text{eval}(x)$ ，参数 x 是个计算串，返回值是将 x 的内容作为表达式计算后的计算结果。作为一个数据， eval 函数的返回值仍可能是计算串（字符串），可以作为 eval 的参数再次计算。

1.2.3 属性

设 p 是两集合 X, Y 的叉积 $X \times Y$ 的子集，若 p 中没有两个元素有相同的左元，则称 p 是定义在集合 X 上的一个属性。集合 $\{x | (x, y) \in p\}$ 称为 p 的定义集， $\{y | (x, y) \in p\}$ 为 p 的取值集；设 $(x, y) \in p$ ，称 y 是 x 的 p 属性值，简称 x 的 p ，记作 $x.p$ 或 $p(x)$ ；此时也称 x 有 p 属性 或 $x.p$ 有定义；若没有 y 使得 $(x, y) \in p$ ，则称 x 无 p 属性。

或 $x.p$ 无定义。

仔细考查属性和函数的定义，可以发现两者并无本质不同。事实上，属性和函数确实是等价概念，其差别只在于应用场合不同。函数更强调参数与返回值间的某种对应关系，大多数情况下这种对应关系有规律可循（如可用表达式描述）；而属性则更多地理解为事物本身的某种性质，其对应关系通常没有规律，需要逐一设置其属性值。

与函数类似，我们在定义属性时经常并不明确指出其定义集，而仅说为某些确定的元素定义某属性，并给出这些元素的相应属性值或取值集，属性的定义集可从上下文中看出。

我们尽量采用与函数不同的写法 $x.p$ 表示属性，但有时文字中符号优先级可能混淆时，也会采用写法 $p(x)$ 以用括号明确符号的作用次序，两种写法是等价的。

我们在实际应用中经常会将属性值写成表达式的形式，称为属性表达式，其计算结果即是属性值。由于任何数据值都可理解为常数表达式，因而可认为任何属性都有表达式形式。一般情况下，我们在模型讨论中无需刻意区分属性表达式和属性值，这时两者都用符号 $x.p$ 表示；而在需要明确区分时，属性表达式更为常用，此时我们约定符号 $x.p$ 表示的是元素 x 的 p 属性表达式（可能是常数），而用 $x.p$ 表示属性值（计算结果）。

取值集为逻辑值{真,假}的属性称为逻辑属性，定义集上的所有元素的属性表达式总是常数的属性称为常数属性。

约定如下与属性相关的变换动作及符号：

1. $x.p=v$

1)若 $x.p$ 有定义则做 $p^-= (x, x.p)$;

2)做 $p^+= (x, v)$ 。

2. $a \leftarrow b$

1)对上下文中所有使 $a.p$ 有定义的属性 p 做 $p^-= (a, a.p)$;

2)对上下文中所有使 $b.p$ 有定义的属性 p 做 $p^+= (a, b.p)$ 。

2 报 表

2.1 表格

2.1.1 网格

设 e 是一个以自然数序偶集 $N \times N$ 为定义域的可逆函数, 对任何 $n, m \in N$, 集合 $T = \{e(r, c) | 1 \leq r \leq n, 1 \leq c \leq m\}$ 称为一个 n 行 m 列的网格, n 和 m 分别称为 T 的行数和列数; T 的元素称为元素格, 显然 $|T| = n * m$. 函数 e 称为 T 的格映射, 记作 $E[T]$.

在讨论多个网格时, 除非上下文中明显地解释了各网格中元素格间的相关性, 否则总是假定这些网格的格映射的值域相离。

本文中将多次采用形如 $P[T]$ 的写法 (如 $E[T]$) 用于表示与网格 T 相关的某事物 P , 当上下文中能将 T 确定时, $P[T]$ 将简单地略写成 P .

对 n 行 m 列的网格 T 及其中某元素格 E , 设 $E = E(r, c)$, 称 r 为 E 的行号, 记作 $\text{row}(E)$, c 为 E 的列号, 记作 $\text{col}(E)$;

对自然数 $i \leq n$, T 的子集

$$R[T](i) := \{E | E \in T \text{ 且 } \text{row}(E) = i\}$$

称为 T 的行, i 称为 $R(i)$ 的序号或行号, $R(i)$ 称为 T 的第 i 行;

对自然数 $j \leq m$, T 的子集

$$C[T](j) := \{E | E \in T \text{ 且 } \text{col}(E) = j\}$$

称为 T 的列, j 称为 $C(j)$ 的序号或列号, $C(j)$ 称为 T 的第 j 列;
行 $R(\text{row}(E))$ 称为 E 的行, 记作 $R[E]$; 列 $C(\text{col}(E))$ 称为 E 的

列, 记作 $C[E]$ 。

由网格 T 的某些行构成的集合称为 T 的行集, 若将行集中的行的序号从小至大排列后可得到一组连续的自然数, 则称该行集为连续行集; 对称地, 可以定义列集和连续列集。

由表格 T 的序号落在区间 $[a, b]$ 的行 (列) 构成的连续行 (列) 集记作 $R[T](a, b)$ ($C[T](a, b)$), 无特别说明时一般假定 $a \leq b$ 。

定义行、列的方位, 对自然数 i, j :

1. 当 $i < j$ 时, 称 $R(i)$ 在 $R(j)$ 上边 或 $R(j)$ 在 $R(i)$ 下边;
2. 当 $i < j$ 时, 称 $C(i)$ 在 $C(j)$ 左边 或 $C(j)$ 在 $C(i)$ 右边;
3. $R(i)$ 与 $R(i+1)$ 相邻, $C(i)$ 与 $C(i+1)$ 相邻。

定义元素格的方位, 设有元素格 A, B :

1. 当 $\text{row}(A) < \text{row}(B)$ 时, 称 A 在 B 上边 或 B 在 A 下边;
2. 当 $\text{col}(A) < \text{col}(B)$ 时, 称 A 在 B 左边 或 B 在 A 右边;
3. 当 $|\text{row}(A) - \text{row}(B)| + |\text{col}(A) - \text{col}(B)| = 1$ 时, 称 A 与 B 相邻。

2.1.2 表格

设 R 是网格 T 的非空子集, 若对任何元素格 $A, B \in R$ 和自然数 $r \in [\text{row}(A), \text{row}(B)]$ 及 $c \in [\text{col}(A), \text{col}(B)]$, 都有 $E(r, c) \in R$, 则称 R 是 T 上的矩形。显然 T 本身和 T 的行、列都是矩形。

我们为某些网格 T 设置一个唯一的由 T 上矩形构成的有限集 (可以是空集) 与之对应, 称为 T 的单元格组, 记作 $UG[T]$ 。

如果设置了单元格组的网格 T 满足: 对任何 $A, B \in UG$ 均有

$A \cap B = \emptyset$, 则称 T 是一个表格, UG 中的元素称为 T 上的单元格。显然 $|UG| \leq |T|$, 即表格上单元格的数量不超过其中元素格的数量。

单元格是我们进行报表模型讨论的主体, 在对表格进行变换时需要同时变换其单元格组以保证其满足定义规则。

如果表格 T 满足: 对任何元素格 $E \in T$ 均有某个单元格 $U \in UG$ 使得 $E \in U$, 则称 T 是满的。只有满表格 T 才可能使 $|UG| \leq |T|$ 中的等号成立。

现实中几乎所有的表格都是满的, 但我们并不假定讨论的表格都是满的。

定义与单元格相关的概念, 设 U 是表格上的单元格:

1. $|U| > 1$ 时称 U 为合并格;
2. $\text{row}(U) := \min(\{\text{row}(E) | E \in U\})$ 称为 U 的行号;
3. $\text{col}(U) := \min(\{\text{col}(E) | E \in U\})$ 称为 U 的列号;
4. $\text{rows}(U) := \max(\{\text{row}(E) | E \in U\}) - \text{row}(U) + 1$ 称为 U 的行数;
5. $\text{cols}(U) := \max(\{\text{col}(E) | E \in U\}) - \text{col}(U) + 1$ 称为 U 的列数;
6. 由与 U 相交的行构成的行集称为 U 的相交行集, 记作 $R[U]$; 由与 U 相交的列构成的列集称为 U 的相交列集, 记作 $C[U]$;
7. U 的相交行(列)集包含于某行(列)集时称 U 在该行(列)集之内; U 的相交行(列)集与某行(列)集相交时称 U 与该行(列)集相交; U 的相交行(列)集与某行(列)集交叉时称 U 与该行(列)集交叉。

定义与单元格相关的方位, 对于任何单元格 A, B :

1. 若 A 任何元素均在 B 任何元素上边, 则称 A 在 B 上边 或 B 在 A 下边;

2. 若 A 任何元素均在 B 任何元素左边, 则称 A 在 B 左边或 B 在 A 右边;

3. 若 A 中某个元素和 B 中某个元素相邻, 则称 A 与 B 相邻;

4. $R[A]$ 中的行都在行 R 的上边 (下边) 时, 称 A 在 R 上边 (下边) 或 R 在 A 下边 (上边);

5. $C[A]$ 中的列都在列 C 的左边 (右边) 时, 称 A 在 C 左边 (右边) 或 C 在 A 右边 (左边);

6. 若 $R[A]$ 与 $R[B]$ 相交 (交叉、相离) 则称 A 和 B 行相交 (行交叉、行相离); 若 $C[A]$ 和 $C[B]$ 相交 (交叉、相离) 则称 A 和 B 列相交 (列交叉、列相离)。

定理 2.1 一些简单的结论。在表格上:

1. 单元格的相交行 (列) 集是连续行 (列) 集;

2. 单元格元素个数为其行数与列数之积, 因而合并格的行数或列数总有一个大于 1;

3. 没有两个单元格的行号和列号都相同, 因而行 (列) 号相同的单元格数量不超过表格的列 (行) 数;

4. 两个行 (列) 相交的单元格必然列 (行) 相离。

引理 若单元格 U 的行号、列号、行数、列数分别为 a, b, p, q , 则有 $U = \{E(x, y) | a \leq x \leq a+p-1, b \leq y \leq b+q-1\}$

【 U 的行号为 a , 即有行号为 a 的元素格 $E_a \in U$, 同理有列号为 b 的元素格 $E_b \in U$, 而 $a \in [a, \text{row}(E_b)], b \in [b, \text{col}(E_a)]$, 根据矩形定义有 $E(a, b) \in U$ 。

令 $c = l+p-1, d = t+q-1$, 则必有行号为 c 的元素格 $E_c \in U$ 和列号为 d 的元素格 $E_d \in U$ (行数、列数的定义), 同理可知有 $E(c, d) \in U$ 。

令 $V = \{E(x, y) | a \leq x \leq c, b \leq y \leq d\}$, 根据矩形定义可知 $V \subseteq U$,

而根据行号、列号、行数、列数的定义亦可知, 对任何 $E \in U$ 必有 $a \leq \text{row}(E) \leq c$ 和 $b \leq \text{col}(E) \leq d$, 即有 $U \subseteq V$, 从而有 $U=V$ 。】

网格 T 上的矩形 $\{E(x,y) | a \leq x \leq c, b \leq y \leq d\}$ 记作 $M[T](a,b,c,d)$ 。

定理 2.1 的证明:

【据引理可设单元格 $U=M(a,b,c,d)$, 则显然 $R[U]=R(a,c)$ 是连续行集, $C[U]=C(b,d)$ 是连续列集; 并且

$$|U|=|M(a,b,c,d)|=(c-a+1)*(d-b+1)=\text{rows}(U)*\text{cols}(U)$$

同时据引理, 对单元格 U 有 $E(\text{row}(U), \text{col}(U)) \in U$, 若单元格 V 与 U 有同样的行号和列号, 则必有 $U \cap V \neq \emptyset$, 与表格定义矛盾。

据引理可知, 对单元格 U 和自然数 i, j 有如下结论:

$$R(i) \in R[U] \text{ 和 } C(j) \in C[U] \Rightarrow E(i,j) \in U$$

若有单元格 U, V 使

$$R[U] \cap R[V] \neq \emptyset \text{ 和 } C[U] \cap C[V] \neq \emptyset$$

设有

$$R(i) \in R[U] \cap R[V] \text{ 和 } C(j) \in C[U] \cap C[V]$$

则 $E(i,j) \in U$ 和 $E(i,j) \in V$, 即 $U \cap V \neq \emptyset$, 与表格定义矛盾。】

表格 T 上行号为 i 、列号为 j 的单元格记作 $U[T](i,j)$, 由定理 2.1 可知这种记法唯一。在实例讨论中为简化书写, 我们常用数字表示行号、用字母表示列号 (从 A 开始排列), 而单元格则用其列号与行号连写命名, 如 A3、B4 等。这种命名称为单元格的平面坐标。

定义集是表格 T 的单元格组的某个子集的属性称为 T 的单元格属性。一个表格可以有多但有限个单元格属性。当 T 变换时, 其上所有单元格属性也要随着 $UG[T]$ 的变换而变换。

同时是逻辑属性的单元格属性称为单元格逻辑属性。

定义表格 T 的名为值 val 的单元格属性, 定义集为 UG , 设 $U \in UG$, $U.val$ 称为 U 的格表达式, $U.val$ 称为 U 的格值。

表格上的单元格可以定义次序。对单元格 A, B , 如果有

$$row(A) < row(B) \parallel (row(A) = row(B) \ \&\& \ col(A) < col(B))$$

则称 A 在 B 前边 或 B 在 A 后边, 记作 $A < B$ 。这个规则定义的次序称为原序。

若单元格 A 使得对表格 T 上任何单元格 $B \neq A$ 都有 $A < B$, 称 A 为 T 的最前格 (其合理性将由下面的定理保证)。显然, 最前格是 T 上行号最小的单元格中列号最小的; 绝大多数情况下最前格即是 $U(1,1)$ (个别情况下 $U(1,1)$ 可能不存在)。

对单元格 A, B , 设 $A < B$, 如果表格上不存在单元格 C 使得 $A < C$ 且 $C < B$, 则称 B 是 A 的跟随格。

定理 2.2 原序具有可序性和传递性。即在同一个表格上:

1. 对任何单元格 $A \neq B$, 必有 $A < B$ 或 $B < A$;
2. 对任何单元格 A, B, C , 若 $A < B$ 且 $B < C$, 则必有 $A < C$ 。

【对表格上任何单元格 A, B , 若 $row(A) \neq row(B)$, 则据定义必有 $A < B$ 或 $B < A$; 若 $row(A) = row(B)$, 据定理 2.1 可知此时必有 $col(A) \neq col(B)$, 则仍有 $A < B$ 或 $B < A$ 。

设单元格 A, B, C 满足 $A < B$ 且 $B < C$; 据定义有 $row(A) \leq row(B)$ 且 $row(B) \leq row(C)$ 即 $row(A) \leq row(C)$ 。 $row(A) < row(C) \Rightarrow A < C$; 而 $row(A) = row(C)$ 时必有 $row(A) = row(B)$ 且 $row(B) = row(C)$, 则 $col(A) < col(B)$ 且 $col(B) < col(C)$ 即 $col(A) < col(C) \Rightarrow A < C$ 。】

可序性及传递性使我们可将表格 T 上的所有单元格按原序

排列成一个数组,称为 T 的单元格数组。显然单元格数组与单元格组的元素完全相同。因此,我们可以修改符号 $UG[T]$ 在 T 是表格时的含义为 T 的单元格数组而不会产生歧义,并约定在对 UG 添加成员时会自动根据原序调整其内元素的次序。

某些算法过程中需要按此次序遍历表格上的所有单元格,术语“按原序对表格 T 上的每个单元格 U 执行 P ”即表示先以 T 的最前格作为 U 执行 P ,再以 U 的跟随格取代 U 执行 P , ..., 重复直到 U 没有跟随格为止。

需要强调的是,仅当 T 是表格时, $UG[T]$ 才是数组,否则仍只是集合。

2.2 基本变换

2.2.1 行列增删

在描述算法前需要特别指出,在某些变换过程中会导致一些概念应当满足的条件不再成立,当变换完全完成后才能重新满足这些条件。

算法 2.1 表格行列增加算法。

在 n 行 m 列的表格 T 的第 k 行后加入 1 行 ($0 \leq k \leq n$):

1. 做 $E(i,j) \rightarrow E(i+1,j)$, 其中 $k < i \leq n, 1 \leq j \leq m$;

注意该变换不仅涉及 T , 还涉及 UG 及 T 的单元格属性:

2. $T += E(k+1,j)$, 其中 $1 \leq j \leq m$, 完成后 T 的行数增加 1;

3. 若 $0 < k < n$ 且此时存在单元格 U 与 $R(k), R(k+2)$ (原 $R(k+1)$) 都相交, 则做

$$U \rightarrow U \cup \{E(k+1,j) | E(k,j) \in U\}$$

在表格的第 k 列后增加 1 列的算法与之对称。

算法 2.2 表格行列删除算法。

在 n 行 m 列的表格 T 中删除第 k 行 ($1 \leq k \leq n$):

1. 对 T 上所有相交行集是 $\{R(k)\}$ 的单元格 U , 做 $U \rightarrow \times$;
 2. $E(k, j) \rightarrow \times$, 其中 $1 \leq j \leq m$;
 3. $E(i, j) \rightarrow E(i-1, j)$, 其中 $k < i \leq n, 1 \leq j \leq m$; 完成后 T 的行数减少 1。
- 将表格中第 k 列删除的算法与之对称。

定理 2.3 对任何表格 T 执行算法 2.1 或算法 2.2 后仍是表格。

【仍以行的情况为例, 列的情况对称。

我们将算法执行前的表格记为 T , 而执行完算法后的结果记为 $\underline{T}$ 。显然 $UG[\underline{T}]$ 的成员均从 $UG[T]$ 中单元格变换而来。

延用算法中的符号, 设 T 上单元格 U 变成 $\underline{T}$ 的子集 $\underline{U}$, 根据定理 2.1 的引理, 可设 $U = M(a, b, c, d) = \{E(x, y) | a \leq x \leq c, b \leq y \leq d\}$, 考查 U 在算法中的变化:

在添加行列算法中:

1. $k \geq c$ 时, $\underline{U} = \{E(x, y) | a \leq x \leq c, b \leq y \leq d\} = M(a, b, c, d)$;
2. $a \leq k < c$ 时, $\underline{U} = \{E(x, y) | a \leq x \leq c+1, b \leq y \leq d\} = M(a, b, c+1, d)$;
3. $k < a$ 时, $\underline{U} = \{E(x, y) | a+1 \leq x \leq c+1, b \leq y \leq d\} = M(a+1, b, c+1, d)$ 。

在删除行列算法中:

4. $k > c$ 时, $\underline{U} = \{E(x, y) | a \leq x \leq c, b \leq y \leq d\} = M(a, b, c, d)$;
 5. $a \leq r \leq c$ 时, $\underline{U} = \{E(x, y) | a \leq x \leq c-1, b \leq y \leq d\} = M(a, b, c-1, d)$,
- 特别地, 当 $a=c$ 时 (必有 $k=a$), U 在步骤 1 中被删除, $\underline{U}$ 不存在;
6. $k < a$ 时, $\underline{U} = \{E(x, y) | a-1 \leq x \leq c-1, b \leq y \leq d\} = M(a-1, b, c-1, d)$ 。

即 $\underline{U}$ 为矩形。

设 T 上另一单元格 V 将变成矩形 $\underline{V}$ 。

在添加行列算法中:

1. $x \leq k$ 时, 根据变换规则有 $E(x,y) \in \underline{U} \Rightarrow E(x,y) \in U$; 可得

$$E(x,y) \in \underline{U} \cap \underline{V} \Rightarrow E(x,y) \in U \cap V$$

2. $x > k$ 时, 若 $x = k+1$, 由于行号为 $k+1$ 的元素格只能在步骤 3 中加入 $\underline{U}$, 即有 $E(x,y) \in \underline{U} \Rightarrow E(x-1,y) \in U$; 若 $x > k+1$, 据算法规则仍有 $E(x,y) \in \underline{U} \Rightarrow E(x-1,y) \in U$; 综合而得:

$$E(x,y) \in \underline{U} \cap \underline{V} \Rightarrow E(x-1,y) \in U \cap V$$

在删除行列算法中:

3. $x < k$ 时, 根据变换规则有 $E(x,y) \in \underline{U} \Rightarrow E(x,y) \in U$; 可得

$$E(x,y) \in \underline{U} \cap \underline{V} \Rightarrow E(x,y) \in U \cap V$$

4. $x \geq k$ 时, 根据变换规则有 $E(x,y) \in \underline{U} \Rightarrow E(x+1,y) \in U$; 可得

$$E(x,y) \in \underline{U} \cap \underline{V} \Rightarrow E(x+1,y) \in U \cap V$$

这几种情况均表明, 命题 $\underline{U} \cap \underline{V} \neq \emptyset \Rightarrow U \cap V \neq \emptyset$ 成立, 则 $U \cap V = \emptyset \Rightarrow \underline{U} \cap \underline{V} = \emptyset$ 也成立, 从而证明 $UG[T]$ 中成员两两相离, 即 T 是表格。】

重复执行算法 2.1 和算法 2.2 即可在表格中加入和删除多个行(列)。

对于充满的表格执行算法 2.1 和算法 2.2 并不能保证结果仍是充满的。事实上, 很容易将一个表格变成充满的, 只要将不属于任何单元格的元素格各自构成单元格加入到 UG 中即可。

设 $M = M(a,b,c,d)$ 是网格 T 上的矩形, 我们将矩形 $M' = M(a+p,b+q,c+q,d+q)$ 记作 $M+(p,q)$, 称 $M \rightarrow M'$ 的变换(包括由元素格变换引发的)为平移 M , M' 是 M 平移 p 行 q 列的结果, p 或 q 可能小于 0, $q=0$ 时 $M+(p,q)$ 可简写为 $M+p$ 。

设 R 是网格 T 上的行集, 行集 $R' = \{R(i+k) | R(i) \in R\}$ 记作 $R+k$, 称 $R \rightarrow R'$ 的变换为平移 R , R' 是 R 平移 k 行的结果, k 可能小于 0。对称地, 可定义列集的平移。

行列增加和行列删除算法中可能产生单元格、行集、列集的平移变换。

如同算法 2.1、2.2, 我们经常会对表格执行某种算法使其变换成一个新表格, 有时我们会将变换前和变换后的表格看成是两张不同但有关的表格。严格地讲, 变换过程可以理解成先为变换前表格 T 复制出一个复本 $\underline{T}$ (复制表格很简单, 只要设计一个值域相离的格映射, 然后替换元素格即可, 显然 T 与 $\underline{T}$ 上的单元格可建立一一对应), 然后在 $\underline{T}$ 上执行变换算法以得到结果。这时称 $\underline{T}$ 是 T 的导出表, T 为 $\underline{T}$ 的原表。谈论导出表时一定意味着存在某个算法变换过程。

设 $\underline{T}$ 是表格 T 的某个导出表, 对 T 上单元格 U 和 $\underline{T}$ 上单元格 $\underline{U}$, 若存在 $n \geq 0$, 并记 $U_0 = U, U_n = \underline{U}$, 在变换过程中依次执行过如下操作:

$$U_0 \rightarrow U_1, U_1 \rightarrow U_2, \dots, U_{n-1} \rightarrow U_n$$

(包括由于元素格变换引发的操作)

则称 $\underline{U}$ 是 U 的继承格, U 称为 $\underline{U}$ 的原格。特别注意这里 $n \geq 0$, 当 $n=0$ 时意味着未进行任何形如 $U_i \rightarrow U_{i+1}$ 的操作, $\underline{U}$ 即是 T 上的 U 。

一般情况下, 由于继承格将完全取代原格 (包括其一切属性), 因此我们在叙述时常常不区分这两者 (如表格 $\underline{T}$ 上的单元格 U 即指在 U 在 $\underline{T}$ 上的继承格 $\underline{U}$)。

在需要明确区分原格与继承格时, 我们约定延用上面的符号

体系, 原表 T 的导出表记作 $\underline{T}$, 原格 U 的继承格记作 $\underline{U}$ 。

显然, 如果对表格连续进行多次变换后, 继承格的继承格仍是继承格。在下面的讨论中我们会经常自然地应用这一结论。

2.2.2 子表引入

我们允许表格 T 上单元格 X 的格值是另一个表格 S , 此时称 S 为 T 的子表, T 为 S 的主表, X 称为子表格。主表可能有多个子表, 子表也可以再有子表。

算法 2.3 子表引入算法。

设 T 是有子表的表格:

1. 对 T 上所有子表格 X 及其对应子表 $S (X.val)$, 设 S 的行数和列数分别为 n 和 m , 并记 $row(X)=a$, $col(X)=b$, $rows(X)=p$, $cols(X)=q$:

a) 若 $p < n$, 在 T 的第 $a+p-1$ 行后加入 $n-p$ 行 (算法 2.1);

b) 对任何 T 上与 $R(a+p-1)$ 相交但不与 $R(a+p)$ 相交的单元格 U , 令

$$U' = \{E(i, j) | a+p \leq i < a+n \text{ 且 } E(a+p-1, j) \in U\}$$

做 $U \rightarrow U \cup U'$;

c) 若 $q < m$, 在 T 的第 $b+q-1$ 列后加入 $m-q$ 列 (算法 2.1),

d) 对任何 T 上与 $C(b+q-1)$ 相交但不与 $C(b+q)$ 相交的单元格 U , 令

$$U' = \{E(i, j) | b+q \leq j < b+m \text{ 且 } E(i, b+q-1) \in U\}$$

做 $U \rightarrow U \cup U'$;

2. 对此时 T 上所有子表格 X 及其对应子表 S , 设 S 的行数和列数分别为 n 和 m , 记此时 $row(X)=a$, $col(X)=b$, $rows(X)=p$,

$\text{cols}(X)=q$;

a) 令 $A=\phi$, $B=\phi$;

b) 若 $p>n$, 令 $A=M[T](a+n,b,a+p-1,b+m-1)$ 并做 $A\leftarrow X$;

c) 若 $q>m$, 令 $B=M[T](a,b+m,a+n-1,b+q-1)$ 并做 $B\leftarrow X$;

d) 做 $X\rightarrow\phi$; 若 $A\neq\phi$ 做 $UG[T]+=A$; 若 $B\neq\phi$ 做 $UG[T]+=B$;

e) 对任何 $U\in UG[S]$, 令

$$U'=\{E[T](a+i-1,b+j-1)|E[S](i,j)\in U\}$$

做 $UG[T]+=U'$ 和 $U'.val=U.val$ 。

定理 2.4 对表格执行子表引入算法后的结果仍是表格。

【延用算法中的符号, 考查算法步骤。

据定理 2.3, 步骤 1-a 后 T 仍是表格。

在步骤 1-b 中, $p\geq n$ 时有 $U'=\phi$, 则没有实质的变换动作, T 仍是表格。

$p<n$ 时, 可设 $U=M(x_1,y_1,a+p-1,y_2)$, 则 $U\cup U'=M(x_1,y_1,a+n-1,y_2)$ 是矩形。

若此时有 $C\in UG[T]$, 使有 $E(i,j)\in U'\cap C$, 则 $a+p\leq i\leq a+n$, 即 C 与 $R(a+p,a+n)$ 相交。考查行列增加算法可知, 仅当步骤 1-a 前 C 与 $R(a+p-1)$ 和 $R(a+p)$ 均相交才可能有这种结果, 且完成步骤 1-a 后仍有 $R(a+p-1)\in R[C]$, 则 $E(a+p-1,j)\in C$ 。这与 $E(i,j)\in U'$ 从而推出 $E(a+p-1,j)\in U$ 产生矛盾, 即 U' 不可能与任何 $UG[T]$ 中单元格相交。

而 $E(i,j)\in U'\Rightarrow E(a+p-1,j)\in U$ 还将保证对另一满足步骤 1-b 中条件的单元格 V , $U\cap V=\phi\Rightarrow U'\cap V=\phi$, 即不会有两个不同的 U' 相交。

综合可知在完成步骤 1-b 后, T 仍是表格。并且 X 作为满足

步骤 1-b 中的条件单元格, 其行数与 S 的行数相同。

步骤 1-c, 1-d 与 1-a, 1-b 对称, 因此同理可知完成这些步骤后 T 仍是表格。并且这些步骤中不会使单元格行数变小, 则在完成步骤 1 后所有子表格 X 的行数和列数均不小于其对应子表 S 的行数和列数。

在步骤 2 的前 4 步中, 分析 A, B 的构成, 显然有 $A \cap B = \emptyset$ 且 $A \subseteq X$ 和 $B \subseteq X$, 则 A, B 不与 $UG[T]$ 中除 X 外的原有单元格相交, 则 $X \rightarrow \times$ 和 $UG[T] += A$ 和 $UG[T] += B$ 后 T 必然仍是表格。

在步骤 2-e 中, 不妨设 $U = M[S](x_1, y_1, x_2, y_2)$, 则显然有 $U' = M[T](x_1, y_1, x_2, y_2) + (a, b)$ 是矩形, 且这些 U' 互不相交; 同时 $R[U'] \subseteq R(a, a+n-1)$, A 不空时 $R[A] = R(a+n, a+p-1)$, 故 $A \cap U' = \emptyset$; 同理 $B \cap U' = \emptyset$; 且由于步骤 1 的动作, 此时必有 $p \geq n$ 和 $q \geq m$, 从而 $U' \subseteq X$, 则 U' 不与 $UG[T]$ 中除 X 外的原有单元格相交, 故 $UG[T] += U'$ 后 T 是表格。】

子表引用算法的过程可以被通俗地理解为先将子表格撑大后再将子表中单元格填入。需要指出的是, 必须先将所有子表格都撑大后再来填入子表单元格, 而不能撑大一个填入一个, 否则将可能把填入的子表单元格也撑大, 导致导出表的单元格大小不正常。

递归地执行子表引入算法可将子表的子表也引入主表。

子表引入算法主要用于处理某些仅采用下文中的扩展算法仍不能完成的复杂表格, 如有限扩展和交替扩展 (其含义将在最后的总结中说明)。

2.3 报表

2.3.1 主格

我们为表格 T 引入一个抽象概念, 称为 T 的根格, 记作 $@$ 。

设 U 是表格 T 上某单元格, 定义 T 上重要的单元格属性:

●扩展方向

取值集为{纵,横}。使该属性有定义的单元格称为扩展格, 否则称为不扩展格; 扩展格根据其属性值分别称为纵主格和横主格, 分别用 $US[T]$ 、 $US_X[T]$ 、 $US_Y[T]$ 表示由 T 上扩展格、纵主格、横主格构成的集合。显然有

$$US = US_X \cup US_Y \text{ 和 } US_X \cap US_Y = \emptyset$$

●左主格 x_p (上主格 y_p)

定义集为 $UG \setminus US_Y$ ($UG \setminus US_X$), 取值集为 $\{@\} \cup US_X$ ($\{@\} \cup US_Y$); 若 $U.x_p$ ($U.y_p$) 有定义, 设取值为 P , 称 P 为 U 的左主格 (上主格), 若 $U \in US_X$ (US_Y) 则称 U 是 P 的子格, 否则称 U 是 P 的属格。显然, 纵(横)主格只有左(上)主格而没有上(左)主格, 而不扩展格即有左主格也有上主格。

●主格 pu

定义集为 US ; $U \in US_X$ (US_Y) 则 $U.pu = U.x_p$ ($U.y_p$); $U.pu$ 称为 U 的主格。

扩展方向、左主格、上主格、主格等属性通称为扩展属性。

实际情况中, 表格上的扩展格一般总是某个其他单元格的左主格或上主格, 我们将 $@$ 和扩展格通称为主格, 而不扩展格又被称为非主格 (事实上这个称呼更加常用)。注意 $@$ 是(左、上)主格但不是单元格、扩展格、纵(横)主格。

采用左(上)主格的说法是由于, 纵(横)主格一般都会在

其属格的左（上）边，但并非强制规则，有时左（上）主格也可能在其属格的右（下）边。

设 U 是定义了扩展属性的表格 T 上的扩展格，若由主格构成的数组 $(P_0, \dots, P_n)$ 如果满足如下条件：

1. $P_n = U$;
2. $P_{i-1} = P_i.pu$ ，其中 $1 \leq i \leq n$;
3. P_0 没有主格。

称该数组为 U 的主格线。

如果定义了扩展属性的表格 T 上所有扩展格均存在主格线，则称 T 为报表。

报表中非主格 U 的左主格线（上主格线）定义为满足如下条件的主格数组 $(P_0, P_1, \dots, P_n)$ ：

1. $P_n = U.xp$ （ $U.yu$ ）
2. $n=0$ 且 P_n 没有主格，或者 P_n 的主格线是 $(P_0, \dots, P_{n-1})$ 。

我们有时也称纵（横）主格的主格线为其左（上）主格线，纵（横）主格没有上（左）主格线，左主格线和上主格线通称为主格线。

定理 2.5 对于任何报表，有

1. 扩展格的主格线唯一，非主格的左（上）主格线存在且唯一；
2. 主格线内没有两个相同的成员，特别地，单元格的主格不可能是其本身；
3. 主格线的第 1 个成员是 @；
4. 任何主格线内的扩展格的扩展方向均相同，故而同一非主

格的械主格线和上主格线的交集为{@}。

【设 P 是报表 T 上的扩展格, $(P_0, \dots, P_n)$ 是 P 的一个主格线, 根据主格线的定义有 $P_{i-1} = P_i \text{ pu}$, 即 P_{i-1} 能被 P_i 唯一确定, 而 $P_n = P$ 也唯一确定, 故扩展格的主格线唯一确定; 非主格的左(上)主格存在且唯一, 其左(上)主格的主格线也存在且唯一, 故其左(上)主格线存在且唯一。

若某主格线 $(P_0, \dots, P_n)$ 中有 $P_i = P_j$ ($0 \leq i < j \leq n$), 则有

$$P_{i-1} = P_i \text{ pu} = P_j \text{ pu} = P_{j-1} \Rightarrow P_{i-2} = P_{j-2} \Rightarrow \dots \Rightarrow P_0 = P_{j-i}$$

这与 P_0 没有主格矛盾。

由于对任何扩展格 P , $P \text{ pu}$ 均有定义, 因此 P 不可能是任何主格线的第 1 个成员, 那么只有 @ 可以成为主格线的第 1 个成员。

纵(横)主格的主格除可能是 @ 外仍都是纵(横)主格, 纵主格不可能同时是横主格, 因而主格线内的扩展格的扩展方向均相同。左(上)主格线上除 @ 外全由纵(横)主格构成, 因此左主格线与上主格线的交集只有{@}。】

事实上, 如果延用图论的术语, 对任何报表, 以{主格}为节点、{(主格, 子格)}为边将构成一个以 @ 为根节点的树(这是 @ 称为根格的原因), 称为主格树。该树上由 @ 和所有纵(横)主格及相关边将构成一个子树, 而(左、上)主格线实际上是该树(子树)上从根到某节点的路径。

没有子格的主格(上述树中的叶子)称为叶子主格。

扩展格的主格线的长度减 1 称为其深度, 并规定 @ 的深度为 0。报表上深度最大的主格的深度称为报表的深度。

若主格 P 属于扩展格 U 的主格线且 $P \neq U$, 称 U 是 P 的后主

格或 P 是 U 的先主格。主格线的唯一性将保证先（后）主格没有二义性。定理 2.5 表明， $@$ 是任何扩展格的先主格。

单元格的左（上）主格线的最末一个成员称为其左导格（上导格），左导格和上导格通称为导格。显然，扩展格的导格是其本身，而非主格的左（上）导格是其左（上）主格。

以扩展格 P 为导格的单元格构成的集合称为 P 的辖区（注意子格不属于辖区），记作 $\text{DOM}[P]$ ；以扩展格 P 及其先主格为导格的单元格构成的集合称为 P 的先辖区，记作 $\text{DOM}_A[P]$ ；以扩展格 P 及其后主格为导格的单元格构成的集合称为 P 的后辖区，记作 $\text{DOM}_B[P]$ 。扩展格 P 的先辖区和后辖区的并集称为 P 的全辖区，记作 $\text{DOM}_C[P]$ 。

显然，对任何扩展格 P 有 $\text{DOM}_A[P] \cap \text{DOM}_B[P] = \text{DOM}[P]$ 。

作为扩展格的先主格， $@$ 的属格可能属于扩展格的先辖区，但 $@$ 不是扩展格而没有上述意义的（先、后）辖区。我们规定 $@$ 有两个辖区 $\text{DOM}[@_x]$ 和 $\text{DOM}[@_y]$ ，分别是由以 $@$ 为左导格和上导格的单元格构成的集合（显然其中只有非主格）。

定义纵主格 P 和横主格 Q 的交辖区为 $\text{DOM}[P] \cap \text{DOM}[Q]$ ，记作 $\text{DOM}[P, Q]$ ；纵主格 P 和 $@$ 的交辖区为 $\text{DOM}[P] \cap \text{DOM}[@_y]$ ，记作 $\text{DOM}[P, @]$ ；横主格 Q 和 $@$ 交辖区为 $\text{DOM}[@_x] \cap \text{DOM}[Q]$ ，记作 $\text{DOM}[@, Q]$ 。

根据报表中主格的性态可对报表进行形式上的简单分类：

1. 没有扩展格的报表称为静态报表（深度为 0）；
2. 所有主格的子格个数不超过 1 的报表称为单片报表，反之，存在某主格有 2 个（或以上）子格的报表称为分片报表，存

在某扩展格有 2 个（或以上）子格的报表称为真分片报表；

3. 没有横主格的报表称为单向报表，即有纵主格也有横主格的报表称为交叉报表。

以上定语可结合使用，如单片单向报表指该报表即是单向报表又是单片报表。

2.3.2 扩展带

如果报表 T 上的纵（横）主格 P 和连续行（列）集 B 满足条件：对任何单元格 $U \in \text{DOM}_B[P]$ 均有 U 在 B 之内，则称 B 是 P 的包含带，显然包含带非空。

扩展格 P 的所有包含带的交集称为 P 的扩展带，记作 $B[P]$ ，纵主格的扩展带称为纵扩展带，横主格的扩展带称为横扩展带。

注意，@不是扩展格而没有扩展带。

定理 2.6 对于任何报表：

1. 扩展格的扩展带是其包含带，因而纵（横）扩展带是非空连续行（列）集；

2. 扩展格的子格的扩展带（简为子扩展带）是其本身扩展带的子集，继而，扩展格的后主格的扩展带是其扩展带的子集。

3. 不在任何纵（横）扩展带之内的单元格的左（上）主格为@。

【以纵主格为例证明，横主格的情况对称。

设 P 是报表 T 上的纵主格。根据定义可知， P 的任何包含带均包含 $\text{DOM}_B[P]$ 中单元格的相交行集，作为这些扩展带的交集， $B[P]$ 也将包含这些行集，即 $\text{DOM}_B[P]$ 中任何单元格也在 $B[P]$ 之内。而 P 的包含带个数有限（ T 的行数有限），有限个自然数连续子集的交集仍是连续子集，故 $B[P]$ 是连续行集。综上可知，

$B[P]$ 也是包含带,即存在包含带(扩展带)是其他包含带的子集。

设纵主格 Q 是纵主格 P 的子格。由于 $DOM_B[Q] \subseteq DOM_B[P]$ 及 $B[P]$ 是 P 的包含带,可知 $B[P]$ 也是 Q 的包含带,则 $B[Q] \subseteq B[P]$ 。

设单元格 U 的左主格为 P , 若 $P \neq @$, 而 $U \in DOM_B[P]$, 则 U 一定在 $B[P]$ (纵扩展带) 之内。因此, 若 U 不在任何纵扩展带内则必有 $U.xp=@$ 。】

需要注意的是, 扩展带不一定真包含子扩展带。

有了扩展带概念, 我们可以对报表作较深刻的分类。如果报表 T 上任何扩展格 P 及与 $B[P]$ 相交的单元格 U 均满足:

1. $U \in DOM_B[P]$ 或 $R[U] \supseteq B[P]$, 称 T 为正报表;
2. $U \in DOM_C[P]$, 称 T 为实报表;
3. $U \in DOM_B[P]$, 称 T 为整报表。

不是实报表的报表称为非实报表, 即是实报表又是正报表的报表称为正实报表。显然, 整报表是正实报表。

现实中有业务含义的报表几乎全是实报表, 且绝大多数是正实报表, 这将是我们的重点关注的领域; 整报表由于比较简单, 可采用更经济高效的手段实现相关算法, 对实际应用有较大意义。

报表上两个纵(横)主格的行(列)相交称为同向相交; 两个纵(横)主格的行(列)交叉称为同向交叉; 纵主格与横主格的行相交或列相交称为异向相交。

若纵(横)主格 P 的扩展带 B 内存在行(列) L 使得没有单元格 $U \in DOM_B[P]$ 与 L 相交, 则称 B 是空心的, 简称空心带。 L 称为 B 的空心行(空心列)。

空心带定义中的 L 一定会在 B 的中部(B 中存在比 L 序号更

小和更大的元素), 否则 BVL 将是连续行(列)集从而也是 P 的包含带。

显然, 整报表上没有空心带。

定理 2.7 对于实报表, 有

1. 若有扩展格与另一扩展格的扩展带相交, 则它们必然在同一个主格线内;
2. 不存在异向相交;
3. 同向相交的两个扩展格必然在同一个主格线内;
4. 扩展格不可能同时在纵扩展带和横扩展带之内。

【若实报表 T 上扩展格 P 与扩展格 Q 的扩展带相交, 根据定义可知 $P \in \text{DOM}_A[Q]$ 或 $P \in \text{DOM}_B[Q]$, $\text{DOM}_A[Q]$ 中与 Q 不同的扩展格都是 Q 的先主格, 此时 P, Q 同在 Q 的主格线上; $\text{DOM}_B[Q]$ 中与 Q 不同的扩展格都是 Q 的后主格, 此时 P, Q 同在 P 的主格线上。

与纵(横)主格行(列)相交的扩展格必然与其扩展带相交, 因而两者必在同一个主格线内, 而纵主格与横主格不可能在同一个主格线内, 因此在实报表上不可能存在异向相交。

同向相交的两个扩展格必然互相与对方的扩展带相交, 故在同一个主格线内。

在纵主格的扩展带(纵扩展带)内的扩展格与该纵主格在同一主格线内, 因而也是纵主格, 从而不可能再在某横主格的扩展带(横扩展带)之内。】

定理 2.8 报表是实报表的充要条件是其满足:

1. 若有扩展格在另一扩展格的扩展带之内, 则它们必然在同

一主格线内;

2. 任何两个扩展带不交叉。

【先证明必要性。

对于实报表, 1 是由定理 2.7 中 1 的特例, 故成立。

设实报表 T 上有两扩展格 P, Q 的扩展带交叉, 由于纵扩展带与横扩展带交集必为空 (行集与列集交集为空), 故 P, Q 的扩展方向相同, 不失一般性, 假定它们都是纵主格 (横主格的情况对称)。

令 $B=B[P] \cap B[Q]$, 由 $B[P]$ 与 $B[Q]$ 都是连续行集且交叉可知, $B[P] \setminus B$ 也是连续行集, 则必存在单元格 $U \in \text{DOM}_B[P]$ 与 B 相交 (否则 $B[P] \setminus B$ 也是 P 的包含带但它不包含 $B[P]$)。设 U 的左导格为 V , 则 $P=V$ 或 P 是 V 的先主格; 同时, U 与 B 相交意味着 U 与 $B[Q]$ 相交, 而 T 是实报表, 则有 $U \in \text{DOM}_c[Q]$, 即 Q 是 V 的先主格或后主格, 无论哪种情况都有 P, V, Q 在同一主格线内 (V 的或 Q 的), 从而有 $B[P]$ 与 $B[Q]$ 相含, 矛盾。

再证明充分性。

设有报表 T 满足这两个条件, P 为 T 上某扩展格, U 是与 $B[P]$ 相交的单元格。设 V 是 U 的左导格。

若 $V \neq @$, 显然 U 在 $B[V]$ 之内, 即 $B[P]$ 与 $B[V]$ 相交, 由于两个扩展带不交叉, 则必有 $B[P] \supseteq B[V]$ 或 $B[P] \subseteq B[V]$, 即 V 在 $B[P]$ 之内或 P 在 $B[V]$ 之内, 无论哪种情况均可推出 V, P 在同一主格线内, 从而有 $U \in \text{DOM}[V] \subseteq \text{DOM}_c[P]$ 。

若 $V=@$, 显然有 $U \in \text{DOM}_A[P]$ 。综合可知, T 是实报表。】

有时我们也用定理 2.8 的两个条件作为判定实报表的依据。

定理 2.9 其他类型报表的一些结论:

1. 正报表上没有单元格与扩展带交叉, 也没有同向交叉;
2. 满的正报表上没有空心带;
3. 整报表上没有同向相交, 特别地, 没有两个扩展带重合;
4. 整报表上与纵(横)主格辖区中某单元格行(列)相交的单元格必属于该辖区。

【以纵主格为例证明, 横主格的情况对称。

设 P 是正报表上的纵主格, U 是与 $B[P]$ 相交的单元格, 若 $U \in \text{DOM}_B[P]$, 则必有 $R[U] \subseteq B[P]$; 若 $U \notin \text{DOM}_B[P]$, 根据正报表定义有 $R[U] \supseteq B[P]$; 两种情况下 U 均不与 $B[P]$ 交叉。

设 P, Q 是正报表上两个同向相交的纵主格。若 $P \notin \text{DOM}_B[Q]$, 显然 P 与 $B[Q]$ 相交, 则 $R[P] \supseteq B[Q] \supseteq R[Q]$; 若 $P \in \text{DOM}_B[Q]$ 则必然 $Q \notin \text{DOM}_B[P]$, 同理可知有 $R[Q] \supseteq R[P]$, 即没有同向交叉。

设 P 是满的正报表上的纵主格, 记 $j = \text{col}(P)$, 对 $B[P]$ 内的任何行 $R = R(i)$ 。由于报表是满的, 则必有单元格 U 使 $E(i, j) \in U$, 显然, U 与 R 相交。若 $U = P \in \text{DOM}_B[P]$, 此时 R 不是空心行; 若 $U \neq P$, 则有 $R(\text{row}(P)) \notin R[U]$ (定理 2.1), 故 $R[U]$ 不是 $B[P]$ 的超集, 由正报表定义可知, 必有 $U \in \text{DOM}_B[P]$, 可知此时 R 也不是空心行; 综合可知, 报表上没有空心带。

若整报表上扩展格 P, Q 同向相交, 则必有 P 与 $B[Q]$ 相交, 则 $P \in \text{DOM}_B[Q]$, 即 P 是 Q 的后主格, 同理有 Q 是 P 的后主格, 矛盾。整报表上两扩展带重合会引发类似矛盾。

设 P 是整报表上的纵主格, 单元格 $U \in \text{DOM}[P]$, 单元格 V 与 $R[U]$ 相交。由于 $R[U] \subseteq B[P]$, 从而 V 与 $B[P]$ 相交, 根据整报表定义可知 $V \in \text{DOM}_B[P]$ 。设扩展格 Q 使 $V \in \text{DOM}[Q]$, 显然也有

$Q \in \text{DOM}_B[P]$; 而由 U 与 V 的对称性可知, 同样有 $P \in \text{DOM}_B[Q]$ 。则必有 $P=Q$, 故 $V \in \text{DOM}[P]$ 。】

现实中绝大多数报表都是正实报表且是满的, 由定理 2.9 可知, 这些报表上都没有空心带, 即空心带在实际情况中很罕见。

2.3.3 临时报表

为表格上的单元格设置好扩展方向和左主格、上主格等属性后即可能获得报表。一般情况下, 报表中的扩展格不多, 逐一设置扩展方向不算复杂; 但设置主格却会涉及到表格上所有单元格, 整个过程较繁琐。为简化该工作, 我们将约定一个自动主格设置方案。

如果表格 T 上已完成所有扩展方向的设置和一部分主格的设置, 我们将未设置(左、上)主格的单元格视为没有该属性, 若这种情况下 T 上任何扩展格均存在主格线, 则称 T 为临时报表。

我们可用前述方法证明临时报表上扩展格的主格线唯一, 因此仍可合理地定义先主格和后主格。但与报表不同, 对于临时报表, 无论@是否在扩展格的主格线内, 均强制规定@是其先主格。

算法 2.4 临时报表的自动主格设置算法。

设有临时报表 T :

1. 按原序对 T 上每个单元格 U 执行: 若 U 是纵主格且 $U.xp$ 无定义, 调用 4 设置其左主格;
2. 按原序对 T 上每个单元格 U 执行: 若 U 是非主格且 $U.xp$ 无定义, 调用 4 设置其左主格;
3. 算法结束;
4. 令 $X=US_X$, $Y=\{V|V \text{ 是非主格且 } V.xp \text{ 有定义}\}$;

5. 若 U 是扩展格 (纵主格), 则再令

$$X = X \setminus \{ \forall U \text{ 是 } V \text{ 的先主格} \}$$

$$Y = Y \setminus \{ \forall U \text{ 是 } V.xp \text{ 的先主格} \}$$
 6. 定义 $Z = \{ \forall V \text{ 在 } U \text{ 左边且 } R[V] \supseteq R[U] \} \cap (X \cup Y)$;
 7. 若 $Z = \emptyset$, 则令 $U.xp = @$, 完成返回; 否则:
 8. 设 W 是 Z 中列号最大的单元格 (据定理 2.1 可知其存在即唯一), 若 $W \in X$, 则令 $U.xp = W$, 完成返回; 否则:
 9. 此时一定有 $W \in Y$, 令 $U.xp = W.xp$; 完成返回。
- 自动设置临时报表上单元格的上主格的方法对称。

注意算法过程的次序, 要先将扩展格的主格设置完毕后再进行非主格的主格设置, 次序不同时的设置结果也可能不同。

算法过程看起来较复杂, 但实际可通俗地解释为在单元格左 (上) 边找到第一个更大 (不小) 的单元格设为其左 (上) 主格, 理解和实现都不困难。

定理 2.10 对临时报表执行自动主格设置算法后将成为报表。

【考查算法内容, 其过程是按次序遍历表格上每个单元格并为之设置主格 (如果需要)。显然, 由于单元格数量有限, 算法必会在有限步内停止。并且, 算法结束时所有单元格都将有符合相应属性规则的主格, 而此时的临时报表显然一定是报表。这样只要证明每次设置单元格的主格后该表格仍然是临时报表即可。

考虑到对称性, 我们仅分析左主格的情况。由于对非主格设置主格不影响扩展格的主格线, 因此只需分析针对扩展格的主格设置。

延用算法中的符号, 设 U 是纵主格, 如果在设置 $U.xp$ 前表格上所有扩展格都存在主格线, 考查算法中的三种情况:

1. 步骤 7, 令 $U.xp=@$ 。

显然设置后 U 有主格线。

2. 步骤 8, 令 $U.xp=W$ 。

根据算法规则可知, U 不是 W 的先主格, 即 U 不在 W 的主格线内, 因而设置 $U.xp$ 不改变 W 的主格线内成员的主格, 故 W 的主格线不变化。执行 $U.xp=W$ 后, U 的主格线将变成在 W 的主格线后加入 U 形成的数组, 即仍存在。

3. 步骤 9, 令 $U.xp=W.xp$ 。

若 $W.xp=@$ 则情况与 1 相同; $W.xp \neq @$ 时情况与 2 类似。两种情况下均可知设置 $U.xp$ 后 U 的主格线仍存在;

除 U 本身外, 设置 $U.xp$ 还可能影响 U 的后主格的主格线, 而其他扩展格的主格线则显然不会变化。设置前 U 的后主格的主格线均开始于 U , 设置后其主格线将变成在其原主格线前依次插入 U 的主格线中不是 U 的成员, 故也有主格线。

综上所述并由归纳法可知, 每次执行完主格设置后该表格仍是临时报表。】

在证明用于保证自动主格设置算法合理的定理 2.10 时并未将算法中条件用足 (如定义 Z 时有关相交行集的条件), 这些条件将用于保证算法更深刻的合理性。

仔细考查扩展带及其依赖概念可知, 在临时报表上仍可用同样方法为扩展格定义扩展带, 并可类似地对临时报表作分类。如果临时报表 T 上任何纵 (横) 扩展格 P 及与 $B[P]$ 相交的单元格 U 均满足:

1. $U.xp(U.yp)$ 无定义或 $U \in \text{DOM}_C[P]$, 则称 T 为临时实报表;
2. $U.xp(U.yp)$ 无定义或 $U \in \text{DOM}_B[P]$, 则称 T 为临时整报表。

定理 2.11 如果临时报表 T 上所有扩展格都设置了主格, 对 T 执行自动主格设置算法后的导出表为 $\underline{T}$, 有:

1. 若 T 是临时实报表则导出表 $\underline{T}$ 是实报表;
2. 若 T 是临时整报表且在算法过程中没有与纵(横)扩展带相交的单元格的左(上)主格被设置成 @, 则导出表 $\underline{T}$ 是整报表。

【显然, 如果 $\underline{T}$ 是临时实(整)报表, 则一定也是实(整)报表。我们采用类似定理 2.10 证明的归纳法, 即证明在每次主格设置后 T 仍是临时实(整)报表。

由于对称性, 我们仅分析设置左主格的情况。延用算法中的符号, 设 T 是临时实报表, 考查算法步骤 7-9, 记设置后的 $U.xp$ 为 P , 由于 T 上扩展格的主格均已设置, 因此可设 U 是非主格。

如果 $P=@$, 显然设置 $U.xp$ 前后不会改变任何扩展带。

如果 $P \neq @$, 则步骤 8 或 9 中的 W 存在, 无论哪种情况, 均有 $R[U] \subseteq R[W] \subseteq B[P]$, 因而 $B[P]$ 在设置前后不会变化, 从而 P 的先主格的扩展带也不变化, 而其他扩展带则更不会变化。

设扩展格 Q 使 $B[Q]$ 与 U 相交(上述讨论可知该相交性与 $U.xp$ 的设置无关)。

如果 $P=@$, 则显然有 $U \in \text{DOM}_A[Q] \subseteq \text{DOM}_C[Q]$ 。

如果 $P \neq @$, 此时 W 存在, 由 W 在 U 左边且 $R[U] \subseteq R[W]$ 可知 $W < U$ (此结论对设置上主格时成立的原因不对称, W 在 U 上边即有 $W < U$), 根据算法过程可知 $W.xp$ 有定义 ($W.xp$ 本来有定

义或会先于 U 被设置); 而 $R[U] \subseteq R[W] \Rightarrow B[Q]$ 与 W 相交, T 是临时实报表, 则有 $W \in \text{DOM}_c[Q] \Rightarrow P \in \text{DOM}_c[Q]$, 而设置后 U 成为 P 的属格显然也有 $U \in \text{DOM}_c[Q]$ 。

T 上其他扩展带与其他相交单元格的性态没有随 $U.xp$ 的设置而变化。综合可知, 设置 $U.xp$ 后 T 仍是临时实报表。

如果 T 是临时整报表, 考查证明内容, 如果 $P=@$ 的情况仅发生在没有扩展格 Q 使 $B[Q]$ 与 U 相交的情况, 将其他情况下涉及的 $\text{DOM}_c[Q]$ 都换成 $\text{DOM}_B[Q]$, 证明仍然成立。即 T 是整报表。】

定理 2.12 如果所有单元格均未设置主格的临时报表 T 满足:

1. 不存在异向相交;
2. 同向相交的纵(横)主格 P 在 Q 左(上)边时有 $R[P] \supseteq R[Q]$ ($C[P] \supseteq C[Q]$)。

那么对 T 执行自动主格设置算法后将得到实报表。

【显然, 对 T 执行算法前, T 上任何扩展格的下辖区内只有其本身, 因而其扩展带与其相交行(列)集等同。

延用算法中的符号, 设 U 是纵主格, 如果设置 $U.xp$ 前 T 上所有扩展格的扩展带与其相交行集等同, 记设置后的 $U.xp$ 为 P 。

若 $P=@$, 显然设置 $U.xp$ 不会改变 T 上任何扩展带。

若 $P \neq @$, 由 $B[P] \supseteq R[W] \supseteq R[U] = B[U]$ 可知 $B[P]$ 不改变, 从而 T 上所有扩展带均不改变。

综合可知, 设置 $U.xp$ 后, T 上所有扩展格的扩展带仍与其相交行集等同。

将 T 上所有扩展格的主格设置后得到的临时报表(算法步骤 2 前) 记为 T' , 由归纳法与对称性可知, T' 上扩展格的扩展带与

其相交行（列）集等同。

设 P, Q 是 T' 上的同向相交纵主格，不失一般性可设 P 在 Q 左边，由条件可知 $R[P] \supseteq R[Q]$ ，从而有 $P < Q$ ，则 $P.xp$ 将先于 $Q.xp$ 设置，这样在设置 $Q.xp$ 执行算法步骤 6 中，至少有 $P \in Z$ ，则步骤 8 中的 W 存在（非主格还未设置主格）。

若 $W \neq P$ ，由 W 的定义可知， $R[W]$ 和 $R[P]$ 都包含 $R[Q]$ ，显然 W 与 P 同向相交；而由 $\text{col}(W) > \text{col}(P)$ 可知 P 在 W 左边。此时用 P, W 替代 P, Q 即可重复上述过程，该过程显然不可能无限重复下去，迭代多次后必会出现 $W = P$ 的现象，而此时的 W 必然是 Q 的先主格，即 P 是 Q 的先主格（同向相交的纵主格中在左边的是另一个的先主格）。

对 T' 上任何纵主格 P 及与 $B[P] = R[P]$ 相交的单元格 U 。 U 不可能是横主格，否则异向相交； U 是非主格则 $U.xp$ 尚未设置，即无定义； U 是纵主格则由上述结论又有 P 是 U 的先主格或 U 是 P 的先主格，即有 $U \in \text{DOM}_c[P]$ ，结合对称性可知 T' 是临时实报表，再由定理 2.11 可知最终结果将是实报表。】

定理 2.11 和定理 2.12 给出了确保自动主格设置算法得到实（整）报表的判定方法。但遗憾的是，对一般的临时实（整）报表执行算法却不一定都会得到实（整）报表，我们目前尚未找到一个能使算法结果是实（整）报表的描述简单、适用范围广泛的充分条件。

不过，好在大多临时报表上扩展格的数量并不多，而且对于常规的临时报表（如左主格总在左边等）而言，算法的实际动作并不复杂。这样使考查扩展格在算法作用下的中间结果（步骤 2 前的临时报表）并不很困难，从而可应用定理 2.11；另外，还有

相当多的临时报表能满足定理 2.12 的条件。事实上，现实应用中的大多数报表都可以也确是应用自动主格设置算法完成的。

3 扩展

3.1 扩展带复制

3.1.1 扩展带复制

算法 3.1 扩展带复制算法。

对于报表 T 上的纵主格 P , 设 $B=B[P]=R(p,q)$ ($p \leq q$), 令 $k=|R(p,q)|=q-p+1$ 。

1. 对任何与 B 交叉且与 $R(q)$ 相交的单元格 U , 令

$$V=\{E|E \in U \text{ 且 } p \leq \text{row}(E) \leq q\}, \text{ 做 } U \rightarrow V$$

2. 在 T 中第 q 行后加入 k 行 (执行 k 次算法 2.1), T 将增加 k 行;

3. 对任何此时与 B 相交的单元格 U , 记

$$U'=\{E(\text{row}(E)+k, \text{col}(E)) | p \leq \text{row}(E) \leq q \text{ 且 } E \in U\}$$

a) $U \notin \text{DOM}_B[P]$ 时, 若 $U \cup U' \neq U$ 且是矩形, 则做 $U \rightarrow U \cup U'$;

b) $U \in \text{DOM}_B[P]$ 时, 做 $U \leftarrow U \cup U'$ 和 $U' \leftarrow U$; 称 U' 是 U 的复制格;

4. 以 P 为 X 调用 5, 算法结束。

5. 对扩展格 $X \in \text{DOM}_B[P]$, 记 X' 为其复制格, 执行:

a) 对 X 的所有属格或子格 C 的复制格 C' , 做 $C'.xp=X'$;

b) 对 X 的所有子格 C , 以 C 为 X 递归调用 5; 完成后返回。

复制报表中横扩展带的方法对称。

定理 3.1 对报表执行扩展带复制算法后的结果仍是报表。

【考虑到对称性, 我们仅分析复制纵扩展带的情况, 并沿用

算法中的符号。

逐步考查算法的步骤。

在步骤 1 中, 显然有 $V \subset U$, 则 V 不可能与 UG 中除 U 外的任何单元格相交; 据定理 2.1 引理可设 $U=M(a,b,c,d)$, $a \leq c, b \leq d$ 。由于 U 与 B 交叉且与 $R(q)$ 相交, 则必然有 $p < a \leq q < b$, 从而可知 $V=M(a,b,q,d)$ 是矩形。因此执行 $U \rightarrow V$ 后, UG 仍由互不相交的矩形构成, 即此时的 T 是表格。

根据定理 2.3, 执行步骤 2 后 T 仍是表格。

记 $B'=R(p+k,q+k)$ 。

考查步骤 3 中的 U 和 U' , 设 $U=M(a,b,c,d)$, $a \leq c, b \leq d$, 则有

$$U'=\{E(x+k,y)|E(x,y) \in U \text{ 且 } p \leq x \leq q\}$$

即

$$U'=M(\max(p,a),b,\min(q,c),d)$$

同时, U 与 B 相交, 则存在自然数 x 使 $p \leq x \leq q$ 且 $a \leq x \leq c$, 即 $\max(p,a) \leq x \leq \min(q,c)$, 从而有 U' 是非空矩形, 并且显然与 U' 相交的行都属于 B' 。

记 V' 为另一个与 B 相交的单元格 V 在步骤 3 中的对应矩形, 则有

$$U' \cap V' \subseteq \{E(\text{row}(E)+k, \text{col}(E)) | p \leq \text{row}(E) \leq q \text{ 且 } E \in U \cap V\} = \emptyset$$

即步骤 3 中任何两个不同的 U' 不相交。

若此时有单元格 C 与 U' 相交, 则 C 必与 B' 中某行相交。考查行列增加算法可知, 仅当执行增加行 (步骤 2) 之前, C (原格) 与当时的 $R(q)$ 和 $R(q+1)$ 均相交才可能发生这种现象, 则显然 $C \notin \text{DOM}_B[P]$; 而在步骤 1 中已排除了 C 与 B 交叉的可能, 则有 $R[C] \supseteq B \cup \{R(q+1)\} \supset B$, 从而执行完步骤 2 后还有 $R[C] \supset B \cup B'$ 。

C 与 B 相交, 即存在与 C 对应的矩形 C' 。对任何 $E \in C'$, 设

$E=E(i+k,j)$, 其中 $E(i,j) \in C$ 且 $p \leq i \leq q$, 由于此时 $R[C] \supset B'$, 则 C 与 $R(i+k)$ 相交, 从而 $E=E(i+k,j) \in C$, 由此可知 $C' \subset C$ 。

设有 $E=E(i+k,j) \in U' \cap C$, 其中 $E(i,j) \in U$ 且 $p \leq i \leq q$, 由于同时 $E \in C$, 则 $C(j) \in C[C]$, 而 $R(i) \in B \subset R[C]$, 可知 $E(i,j) \in C$, 即 $U \cap C \neq \emptyset$, 则必有 $U=C$ 。

综上所述可知, 与 U' 相交的单元格只能是 U 本身, 且此时还有 $U \notin \text{DOM}_B[P]$ 和 $U' \subset U$ 即 $U \cup U' = U$ 。

这样, 在步骤 3-a 和 3-b 中, 当 $U \cup U' \neq U$ 或 $U \in \text{DOM}_B[P]$ 时, U 必然与 UG 中其他任何单元格均不相交, 则用 $U \cup U'$ 替换 U 或将 U' 加入 UG 后仍能保证 UG 中矩形互不相交;

步骤 4、5 中不再涉及单元格中元素格变换。因此可知算法完成时 T 是表格。

现在可对 T 应用导出表和继承格的概念及符号。

考查算法过程可知, T 上单元格只可能有两种情况, 要么是 T 上某单元格的继承格, 要么是继承格的复制格。由于算法过程中仅对复制格的 (左) 主格做了设置, 则继承格的主格仍是继承格, 因此 T 上是继承格的扩展格的主格线将保持不变 (由继承格构成)。

设 U' 是 T 上的纵主格 U (继承格) 的复制格, 则必有 $U \in \text{DOM}_B[P]$ 。若 $U \neq P$, 记 $V=U.xp$, 必有 $V \in \text{DOM}_B[P]$, 则 V 也有复制格, 记作 V' ; 据算法过程有 $U'.xp=V'$, 此时如果 V' 有主格线则 U' 也会有。而显然对 U 迭代取主格必会停在 P 上, 由 $P'.xp=P.xp$ 可知 P' 有主格线, 递归下去可知 U' 也有主格线。

综合上面论证可得: T 是报表。】

3.1.2 单元格变迁

将针对扩展格 P 执行扩展带复制算法的过程称为复制 P 。我们将研究复制扩展格后报表上单元格及扩展带的变化规律。

下面的结论均以纵主格为例叙述,对于横主格将有对称的结论。

仍沿用算法和定理 3.1 证明中的符号,记 $B'=R(p+k,q+k)$,将单元格 U 的继承格记作 $\underline{U}$, $U \in \text{DOM}_B[P]$ 时, $\underline{U}$ 的复制格记作 $\underline{U}'$ 。

定理 3.2 采用上述符号,设单元格 $U=M(a,b,c,d), a \leq c, b \leq d$, 复制纵主格 P 后有:

1. 若 $c < q$ 且 $U \notin \text{DOM}_B[P]$, 则 $\underline{U}=M(a,b,c,d)$, $\underline{U}'$ 不存在;
2. 若 $c \geq q$ 且 $a \leq p$ 且 $U \notin \text{DOM}_B[P]$, 则 $\underline{U}=M(a,b,c+k,d)$, $\underline{U}'$ 不存在;
3. 若 $c \geq q$ 且 $p < a \leq q$ 且 $U \notin \text{DOM}_B[P]$, 则 $\underline{U}=M(a,b,q,d)$, $\underline{U}'$ 不存在;
4. 若 $a > q$, 此时 $U \notin \text{DOM}_B[P]$, 则 $\underline{U}=M(a,b,c,d)+k$, $\underline{U}'$ 不存在;
5. 若 $U \in \text{DOM}_B[P]$, 此时 $p \leq a \leq c \leq q$, 则 $\underline{U}=M(a,b,c,d)$ 和 $\underline{U}'=M(a,b,c,d)+k$ 。

【对每种情况逐步执行算法即可得到相应结论。】

定理 3.2 中的 5 种情况覆盖了单元格的所有可能性。

推论 1 复制纵主格后,继承格及其复制格的列号、列数与原格相同。行号、行数相同的单元格的继承格及其导出格的行号、行数也对应相同。

【考查定理中 U 和 $\underline{U}$ 与 $\underline{U}'$ 的行号、行数、列号、列数的表

达式即知。】

推论 2 复制纵主格 P 后, 若有 $\underline{U}$ 与 B' 相交, 则 $R[\underline{U}] \supseteq B$, 从而 $R[\underline{U}] \supseteq B \cup B'$ 。

【考查定理中的 5 种情况, 当且仅当情况 2 中可能有 $\underline{U}$ 与 B' 相交, 而此时的条件表明 $R[\underline{U}] \supseteq B$ 。】

我们再来分析扩展带的变化规律。

定理 3.3 采用上述符号, 设 Q 是 T 上扩展格, 在复制纵主格 P 后有:

1. Q 是横主格时, $B[Q] = B[Q]$ 。

Q 是纵主格时, 设 $B[Q] = R(u, v)$, $u \leq v$ 。

2. 若 $v < q$ 且 $Q \notin \text{DOM}_B[P]$, 则 $B[Q] = R(u, v)$ 。

3. 若 $v \geq q$ 且 $u \leq q$ 且 $Q \notin \text{DOM}_B[P]$, 则

$$B[Q] \subseteq R(u, v+k) \text{ 且 } R(u) \in B[Q]$$

其中若 $v = q$ 且 $u > p$, 则 $B[Q] = R(u, v)$ 。

4. 若 $u > q$, 此时 $Q \notin \text{DOM}_B[P]$, 则 $B[Q] = R(u, v) + k$ 。

以上情况中 Q' 均不存在。

5. $Q \in \text{DOM}_B[P]$, 此时 $p \leq u \leq v \leq q$, 则

$$B[Q] = R(u, v) \text{ 和 } B[Q'] = R(u, v) + k$$

【情况 1~5 中均有 $Q \notin \text{DOM}_B[P]$, 则关于 Q' 不存在的结论显然成立。

分 5 种同样的情况讨论。

1. 分析算法过程可知

$$\underline{U}' \in \text{DOM}_B[Q] \Leftrightarrow \underline{U} \in \text{DOM}_B[Q] \Leftrightarrow U \in \text{DOM}_B[Q]$$

且由定理 3.2 的推论 1 有

$$C[\underline{U}'] = C[\underline{U}] = C[U]$$

故有

$$B[Q] = B[Q]$$

以下情况中设 $U = M(a, b, c, d) \in \text{DOM}_B[Q]$, 则 $u \leq a \leq c \leq v$ 。

2. 此时 $U \notin \text{DOM}_B[P]$ 和 $c \leq v < q$, 则

$$\underline{U} = M(a, b, c, d) \Rightarrow B[Q] = B[Q] = R(u, v)$$

3. 此时 $U \notin \text{DOM}_B[P]$, 而

$$R[\underline{U}] \subseteq R(a, c+k) \subseteq R(u, v+k) \Rightarrow B[Q] \subseteq R(u, v+k)$$

同时

$$B[Q] = R(u, v) \Rightarrow \text{存在 } U \text{ 使 } a = u$$

$$\Rightarrow \underline{U} = M(u, b, c, d) \text{ 或 } M(u, b, c+k, d) \Rightarrow R(u) \in B[Q]$$

若 $v = q$ 且 $u > p$, 则 $\underline{U} = M(a, b, c, d) \Rightarrow B[Q] = B[Q] = R(u, v)$ 。

4. 此时 $U \notin \text{DOM}_B[P]$ 和 $a > q$, 则

$$U = M(a, b, c, d) + k \Rightarrow B[Q] = B[Q] + k = R(u, v) + k$$

5. 分析算法过程可知

$$\underline{U}' \in \text{DOM}_B[Q'] \Leftrightarrow \underline{U} \in \text{DOM}_B[Q] \Leftrightarrow U \in \text{DOM}_B[Q]$$

而此时还有

$$\underline{U} = M(a, b, c, d) \text{ 和 } \underline{U}' = M(a, b, c, d) + k$$

从而可得

$$B[Q] = B[Q] = R(u, v) \text{ 和 } B[Q'] = B[Q] + k = R(u, v) + k$$

综上所述可知命题成立。】

定理 3.3 中的 6 种情况覆盖了扩展格的所有可能性。

推论 1 扩展格的继承格与其复制格的扩展带相离。

【考查定理中情况 5 即知。】

推论 2 扩展带相离的扩展格在复制纵主格 P 后的继承格的扩展带仍相离；除了其一是 P 的后主格且另一的扩展带与 $B[P]$ 交叉的情况外，可能出现的复制格的扩展带相离，复制格与继承格的扩展带也相离。

【仍沿用上述符号，设有两个扩展格 Q_1, Q_2 ，若其中有任何一个是横主格，则命题显然成立，因此我们仅考查 Q_1, Q_2 都是纵主格的情况。

设 $B[Q_1]=R(u_1, v_1)$, $B[Q_2]=R(u_2, v_2)$ ，穷举 Q_1, Q_2 在定理中列出的可能情况，为书写简单，用 $Q_1:i, Q_2:j$ 表示 Q_1 属于情况 i , Q_2 属于情况 j 。

$Q_1:2, Q_2:2$ 时，

$$B[Q]=B[Q] \Rightarrow B[Q_1] \cap B[Q_2] = B[Q_1] \cap B[Q_2] = \phi$$

$Q_1:2, Q_2:3$ 时，有 $v_1 < v_2 \Rightarrow B[Q_1] \cap B[Q_2] = \phi \Rightarrow u_1 \leq v_1 < u_2 \leq v_2$ ，从而

$$B[Q_1]=R(u_1, v_1) \text{ 和 } B[Q_2] \subseteq R(u_2, v_2+k) \text{ 相离}$$

$Q_1:2, Q_2:4$ 时，有 $v_1 < u_2$ ，则 $B[Q_1] \cap B[Q_2] = \phi \Rightarrow u_1 \leq v_1 < u_2 \leq v_2$ ，从而

$$B[Q_1]=R(u_1, v_1) \text{ 和 } B[Q_2]=R(u_2, v_2)+k \text{ 相离}$$

$Q_1:3, Q_2:3$ 时，有 $R(q) \in B[Q_1] \cap B[Q_2]$ ，不可能出现这种情况；

$Q_1:3, Q_2:4$ 时，有 $u_1 < u_2$ ，则 $B[Q_1] \cap B[Q_2] = \phi \Rightarrow u_1 \leq v_1 < u_2 \leq v_2$ ，从而

$$B[Q_1] \subseteq R(u_1, v_1+k) \text{ 和 } B[Q_2]=R(u_2, v_2)+k \text{ 相离}$$

$Q_1:4, Q_2:4$ 时，

$$B[Q]=B[Q]+k \Rightarrow B[Q_1] \cap B[Q_2] = (B[Q_1] \cap B[Q_2]) + k = \phi;$$

$Q_1:5, Q_2:2$ 时，

$$B[Q]=B[Q] \Rightarrow B[Q_1] \cap B[Q_2] = B[Q_1] \cap B[Q_2] = \phi$$

而 $v_2 < q < u_1 + k$ 则

$B[Q_1'] = R(u_1, v_1) + k$ 与 $B[Q_2] = R(u_2, v_2)$ 相离

$Q_1:5, Q_2:3$ 时, 有 $v_1 < v_2$, 则有

$$B[Q_1] \cap B[Q_2] = \emptyset \Rightarrow u_1 \leq v_1 < u_2 \leq v_2$$

从而

$B[Q_1] = R(u_1, v_1)$ 和 $B[Q_2] \subseteq R(u_2, v_2 + k)$ 相离

若 $Q_1 = P$, 则有 $R(q) \in B[Q_1] \cap B[Q_2]$, 不可能出现这种情况;

由于 $u_1 + k > q$, 若 $B[Q_2]$ 与 $B[P]$ 不交叉, 则有

$$v_2 = q \text{ 和 } u_2 < v_1 \leq p \Rightarrow$$

$B[Q_2] = R(u_2, v_2) \subseteq R(p, q)$ 与 $B[Q_1'] = R(u_1, v_1) + k$ 相离

此时若 $B[Q_2]$ 与 $B[P]$ 交叉, 则 $B[Q_1']$ 将可能与 $B[Q_2]$ 相交;

$Q_1:5, Q_2:4$ 时, 有 $v_1 < u_2$, 则 $B[Q_1] \cap B[Q_2] = \emptyset \Rightarrow u_1 \leq v_1 < u_2 \leq v_2$,

从而

$B[Q_1] = R(u_1, v_1)$ 和 $B[Q_2] = R(u_2, v_2) + k$ 相离

而 $v_1 + k \leq q + k < u_2 + k$ 则

$B[Q_1'] = R(u_1, v_1) + k$ 与 $B[Q_2] = R(u_2, v_2) + k$ 相离

$Q_1:5, Q_2:5$ 时,

$$B[Q] = B[Q] \Rightarrow B[Q_1] \cap B[Q_2] = B[Q_1] \cap B[Q_2] = \emptyset$$

$$B[Q'] = B[Q] + k \Rightarrow B[Q_1'] \cap B[Q_2'] = (B[Q_1] \cap B[Q_2]) + k = \emptyset$$

而 $v_2 < q < u_1 + k$ 则

$B[Q_1'] = R(u_1, v_1) + k$ 与 $B[Q_2] = R(u_2, v_2)$ 相离

同理 $B[Q_1]$ 与 $B[Q_2']$ 也相离。】

推论 3 实报表上扩展带相离的扩展格在复制纵主格后其继承格及可能出现的复制格的扩展带两两相离。

【由于实报表上不存在扩展带交叉的现象, 由推论 2 可知命题成立。】

3.1.3 封闭性

称某算法对某集合封闭是指对该集合任何元素执行该算法后得到的结果仍属于该集合。

定理 3.4 扩展带复制算法对实报表、(满)正实报表和整报表封闭。

【考虑到对称性,我们仅分析复制纵扩展带的情况,并沿用算法和定理 3.1 中的符号,记 $B'=R(p+k,q+k)$,逐步考查算法步骤。

设 T 是实报表。

先仅考查步骤 1,将该步骤看成对 T 的一次变换。由于 T 是实报表和 U 与 B 交叉,则 $U \in \text{DOM}_A[P]$ 。设 Q 是 P 的先主格,由于 $R[U]=R[V] \subset R[U]$,则做 $U \rightarrow V$ 后必有 $B[Q] \subseteq B[Q]$ 。对任何与 $B[Q]$ 相交的单元格 C ,有

$$\begin{aligned} C \text{ 与 } B[Q] \text{ 相交} &\Rightarrow C \text{ 与 } B[Q] \text{ 相交} \\ &\Rightarrow C \in \text{DOM}_C[Q] \Rightarrow C \in \text{DOM}_C[Q] \end{aligned}$$

即执行完步骤 1 后 T 仍是实报表。同理可证若 T 是整报表,执行完步骤 1 后的 T 也是整报表。而正报表上不存在单元格与扩展带交叉(定理 2.9),即步骤 1 中的替换动作不会执行。

综合可知,步骤 1 对实报表、(满)正实报表和整报表封闭。

这样,我们在下面的讨论中可以认为算法将从步骤 2 开始执行,所针对的报表 T 上不存在与 B 交叉且与 $R(b)$ 相交的单元格。

显然 T 上的扩展格只能是 T 上扩展格的继承格或其复制格,设 Q 是 T 上的扩展格,将单元格 U 的继承格记作 $\underline{U}$, $U \in \text{DOM}_B[P]$ 时, $\underline{U}$ 的复制格记作 $\underline{U}'$ 。

分 4 种情况讨论:

1. $Q \in US_Y[T]$

由定理 3.3 有 $B[Q]=B[Q]$, 对单元格 $\underline{U}$ 和 $\underline{U}'$, 有

$$\begin{aligned} \underline{U} \text{ 与 } B[Q] \text{ 相交} &\Rightarrow \underline{U} \text{ 与 } B[Q] \text{ 相交} \\ &\Rightarrow \underline{U} \in \text{DOM}_C[Q] \Rightarrow \underline{U} \in \text{DOM}_C[Q] \end{aligned} \quad (1)$$

和

$$\begin{aligned} \underline{U}' \text{ 与 } B[Q] \text{ 相交} &\Rightarrow \underline{U}' \text{ 与 } B[Q] \text{ 相交} \\ &\Rightarrow \underline{U}' \in \text{DOM}_C[Q] \Rightarrow \underline{U}' \in \text{DOM}_C[Q] \end{aligned} \quad (2)$$

$Q \notin \text{DOM}_B[P]$, 则 Q 没有复制格。

2. $Q \in US_X[T]$ 且 $Q \notin \text{DOM}_C[P]$

$Q \notin \text{DOM}_C[P]$, 则 $B[Q]$ 与 $B[P]$ 相离; 设 $B[Q]=R(u,v)$, 则有 $v < p$ 或 $q < u$ 。

$v < p$ 时, 由定理 3.3 有 $B[Q]=R(u,v)$ 。

显然, 不可能有 $\underline{U}'$ 与 $B[Q]$ 相交 ($R[U'] \subseteq B'$)。设 $\underline{U}$ 与 $B[Q]$ 相交, 则有 $E(i,j) \in \underline{U}$ 使 $u \leq i \leq v < p \leq q$, 根据定理 3.2 可知有 $E(i,j) \in U$, 即 U 与 $B[Q]$ 相交。

$q < u$ 时, 由定理 3.3 有 $B[Q]=R(u,v)+k$ 。

此时同样不可能有 $\underline{U}'$ 与 $B[Q]$ 相交 ($\underline{U}'$ 都在 $R(c+k)$ 上边), 设有 $\underline{U}$ 与 $B[Q]$ 相交, 则存在 $E(i+k,j) \in \underline{U}$ 使 $q+k < u+k \leq i+k \leq v+k$, 根据定理 3.2 可知有 $E(i,j) \in U$, 即 U 与 $B[Q]$ 相交。

两种情况均有

$$\begin{aligned} \underline{U} \text{ 与 } B[Q] \text{ 相交} &\Rightarrow \underline{U} \text{ 与 } B[Q] \text{ 相交} \\ &\Rightarrow \underline{U} \in \text{DOM}_C[Q] \Rightarrow \underline{U} \in \text{DOM}_C[Q] \end{aligned} \quad (3)$$

$Q \notin \text{DOM}_B[P]$, 则 Q 没有复制格。

3. $Q \neq P$ 且 $Q \in \text{DOM}_A[P]$

$Q \in \text{DOM}_A[P]$, 则 $B[Q] \supseteq B$, 设 $B[Q] = R(u, v)$, 则有 $u \leq p \leq q \leq v$ 。

记 $D = R[T](u, v+k)$ 。由定理 3.3 有 $D \supseteq B[Q]$ 。

设有继承格 $\underline{U}$ 与 D 相交, 则有 $E(i, j) \in \underline{U}$ 使 $u \leq i \leq v+k$ 。

若 $u \leq i \leq q$, 根据定理 3.2 可知有 $E(i, j) \in U$, 即 U 与 $B[Q]$ 相交。

若 $q+k < i \leq v+k$, 根据定理 3.2 可知有 $E(i-k, j) \in U$, 而 $u \leq q < i-k \leq v$, 则同样有 U 与 $B[Q]$ 相交。

若 $q+1=p+k \leq i \leq q+k$, 即 $\underline{U}$ 与 B' 相交, 根据定理 3.2 的推论 2 可知有 $R[U] \supseteq B$, 即仍有 U 与 $B[Q]$ 相交。

综合上述三种情况并结合 $D \supseteq B[Q]$ 有

U 与 $B[Q]$ 相交 $\Rightarrow U$ 与 D 相交 $\Rightarrow U$ 与 $B[Q]$ 相交

即有

$\underline{U}$ 与 $B[Q]$ 相交 $\Rightarrow U$ 与 $B[Q]$ 相交

$\Rightarrow U \in \text{DOM}_c[Q] \Rightarrow \underline{U} \in \text{DOM}_c[Q]$ (4)

考查算法过程可知, 对任何复制格 $\underline{U}'$ 均有 $\underline{U} \in \text{DOM}_B[P]$ 和 $\underline{U}' \in \text{DOM}_B[P']$, 而从 $\underline{P}.xp = \underline{P}'.xp$ 又可知, $\underline{P}$ 和 $\underline{P}'$ 都是 $\underline{Q}$ 的后主格, 即有 $\underline{U}' \in \text{DOM}_B[Q]$ 。

$Q \notin \text{DOM}_B[P]$, 则 $\underline{Q}$ 没有复制格。

4. $Q \in \text{DOM}_B[P]$

设 $B[Q] = R(u, v)$, 因 $Q \in \text{DOM}_B[P]$ 则 $p \leq u \leq v \leq q$ 。

由定理 3.3 有 $B[Q] = B[Q] = R(c, d)$ 。

设继承格 $\underline{U}$ 与 $B[Q]$ 相交, 则存在 $E(i, j) \in \underline{U}$ 使 $u \leq i \leq v \leq q$, 继而有 $E(i, j) \in U$, 即 U 与 $B[Q]$ 相交。于是

$\underline{U}$ 与 $B[Q]$ 相交 $\Rightarrow U$ 与 $B[Q]$ 相交

$\Rightarrow U \in \text{DOM}_c[Q] \Rightarrow \underline{U} \in \text{DOM}_c[Q]$ (5)

而复制格的相交行集包含于 B , 显然与 $B[Q] \subseteq B$ 相离, 不可

能与 $B[Q]$ 相交。

$Q \in \text{DOM}_B[P]$, 则此时 Q 有复制格 Q' 。根据定理 3.3 可知 $B[Q'] = R(u, v) + k$ 。

设有复制格 U' 与 $B[Q']$ 相交, 则必与 D 相交, 则存在 $E(i, j) \in U'$ 使 $u+k \leq i \leq v+k$, 从而有 $E(i-k, j) \in U$ 和 $u \leq i-k \leq v$, 即 U 与 $B[Q]$ 相交, 则

$$\begin{aligned} U' \text{ 与 } B[Q'] \text{ 相交} &\Rightarrow U \text{ 与 } B[Q] \text{ 相交} \\ &\Rightarrow U \in \text{DOM}_C[Q] \Rightarrow U' \in \text{DOM}_C[Q'] \end{aligned} \quad (6)$$

设有继承格 U 与 $B[Q']$ 相交, 则必与 $B' \supseteq D$ 相交。根据定理 3.2 的推论 2 可知有 $R[U] \supseteq B$, 即 U 与 B 相交, 从而 $U \in \text{DOM}_C[P]$, 但 $U \notin \text{DOM}_B[P]$ (否则 U 不可能与 B' 相交), 则必有

$$\begin{aligned} U &\in \text{DOM}_A[P] \setminus \text{DOM}[P] \\ &\Rightarrow U \in \text{DOM}_A[P] \setminus \text{DOM}[P] = \text{DOM}_A[P'] \setminus \text{DOM}[P'] \end{aligned}$$

而显然有 $\text{DOM}_A[P'] \setminus \text{DOM}[P'] \subset \text{DOM}_A[P']$, 因而

$$\begin{aligned} U \text{ 与 } B[Q'] \text{ 相交} &\Rightarrow U \text{ 与 } B \text{ 相交} \\ &\Rightarrow U \in \text{DOM}_A[P'] \subseteq \text{DOM}_A[Q'] \end{aligned} \quad (7)$$

综合上述 4 种情况可得结论: T 是实报表。

如果 T 同时还是正报表, 则可将(1) ~ (6)后两个 $\Rightarrow$ 结论修改为:

$$\begin{aligned} (1) &\Rightarrow U \in \text{DOM}_B[Q] \text{ 或 } C[U] \supseteq B[Q] \\ &\Rightarrow U \in \text{DOM}_B[Q] \text{ 或 } C[U] \supseteq B[Q] \\ &\quad (C[U] = C[U] \text{ 和 } B[Q] \supseteq B[Q]) \end{aligned} \quad (1')$$

$$\begin{aligned} (2) &\Rightarrow U \in \text{DOM}_B[Q] \text{ 或 } C[U] \supseteq B[Q] \\ &\Rightarrow U' \in \text{DOM}_B[Q'] \text{ 或 } C[U'] \supseteq B[Q'] \\ &\quad (C[U] = C[U'] \text{ 和 } B[Q] \supseteq B[Q']) \end{aligned} \quad (2')$$

$$\begin{aligned} (3) \Rightarrow U \in \text{DOM}_B[Q] \text{ 或 } C[U] \supseteq B[Q] \\ \Rightarrow \underline{U} \in \text{DOM}_B[Q] \text{ 或 } C[\underline{U}] \supseteq B[Q] \end{aligned} \quad (3')$$

$$(C[U] \supseteq B[Q] \Rightarrow C[\underline{U}] \supseteq B[Q])$$

$$\begin{aligned} (4) \Rightarrow U \in \text{DOM}_B[Q] \text{ 或 } C[U] \supseteq B[Q] \\ \Rightarrow \underline{U} \in \text{DOM}_B[Q] \text{ 或 } C[\underline{U}] \supseteq B[Q] \end{aligned} \quad (4')$$

$$(C[U] \supseteq B[Q] \Rightarrow C[\underline{U}] \supseteq D \supseteq B[Q])$$

$$\begin{aligned} (5) \Rightarrow U \in \text{DOM}_B[Q] \text{ 或 } C[U] \supseteq B[Q] \\ \Rightarrow \underline{U} \in \text{DOM}_B[Q] \text{ 或 } C[\underline{U}] \supseteq B[Q] \end{aligned} \quad (5')$$

$$(C[U] \supseteq B[Q] \Rightarrow C[\underline{U}] \supseteq C[U] \supseteq B[Q] = B[Q])$$

$$\begin{aligned} (6) \Rightarrow U \in \text{DOM}_B[Q] \text{ 或 } C[U] \supseteq B[Q] \\ \Rightarrow \underline{U}' \in \text{DOM}_B[Q'] \text{ 或 } C[\underline{U}'] \supseteq B[Q'] \end{aligned} \quad (6')$$

$$(C[U] \supseteq B[Q] \Rightarrow C[\underline{U}'] \supseteq D \supseteq B[Q'])$$

而(7)的应用条件下有, $R[U] \supseteq B \Rightarrow R[\underline{U}] \supseteq B \cup B' \supseteq B[Q']$ 。

综合这些结论可知, $\underline{T}$ 必是正报表。

如果 T 是整报表 (也是实报表), 则将(1)~(6)中的 DOM_C 换成 DOM_B 后仍成立; 而(7)的应用条件是 $\underline{U}$ 与 B' 相交 $\Rightarrow R[U] \supseteq B$ 和 $U \notin \text{DOM}_B[P]$, 在整报表上不可能有这种现象。因此 $\underline{T}$ 必是整报表。

如果 T 是满的正实报表, 仍假定步骤 1 被略去, 设有 $E(i, j) \in \underline{T}$ 。

$i \leq q$ 时, 有 $E(i, j) \in T$, 则有 $U \in \text{UG}[T]$ 使 $E(i, j) \in U$, 则 $E(i, j) \in \underline{U}$;

$i > q+k$ 时, 有 $E(i-k, j) \in T$, 则有 $U \in \text{UG}[T]$ 使 $E(i-k, j) \in U$, 则 $E(i, j) \in \underline{U}$;

$p+k \leq i \leq q+k$ 时, 设有 $U \in \text{UG}[T]$ 使 $E(i-k, j) \in U$ 。若 $U \in \text{DOM}_B[P]$, 则必有 $E(i, j) \in \underline{U}'$; 否则由于 T 是正报表, 则有 $R[U] \supseteq B$, 从而必有 $E(i, j) \in \underline{U}$ 。

综合可知， T 是满的。】

定理 3.4 是关于报表扩展的重要基础定理，它确保了以扩展带复制为基本环节的报表扩展变换能够连续合理进行。

我们还可以证明定理 3.4 的逆命题也成立，即如果对某报表执行扩展带复制算法后得到的导出表是实报表、(满)正实报表、整报表，则其原表一定也是实报表、(满)正实报表、整报表(只要考虑在算法过程中未改变的扩展带 B 即可)。这种性质称某算法对某集合反封闭，即仅对该集合中元素执行该算法后得到的结果才属于该集合。

需要指出的是，对于不是实报表的正报表(无论是否满)和不是正报表的满实报表，扩展带复制算法并不封闭。

3.2 主格扩展

3.2.1 主格扩展

定义单元格逻辑属性扩展状态，定义集为所有扩展格。属性值为假者称为未扩展的或未扩展格，反之称为已扩展的或已扩展格。在报表扩展变换前，所有扩展格的扩展状态设置为假。

为讨论方便，我们规定@是已扩展格。

为保证扩展过程合理进行，我们作一个并不过分的约定：

规则 3.1 报表中任何未扩展格的格值是非空数组。

如果格值不是数组，可理解成是一个由该格值构成的 1 元组，若格值为空集，则理解成是一个由 NULL(空值)构成的 1 元组。

算法 3.2 主格扩展算法。

对于报表 T 中的某个未扩展格 P :

1. 计算 $P.val$, 设计算结果为 $(v_1, \dots, v_n)$, 由规则 3.1 约定知 $n \geq 1$;
2. 将 P 的扩展状态设置为真;
3. 连续针对 P 执行 $n-1$ 扩展带复制算法 (算法 3.1), 每执行一次将得到一个 P 的复制格, 按次序分别记作 $P_n, P_{n-1}, \dots, P_2$, 并记 $P_1 = P$;
4. 做 $P_i.val = v_i$ ($1 \leq i \leq n$)。

显然在主格扩展算法完成之后, 对任何单元格 $U \in \text{DOM}_B[P]$ 均有 $n-1$ 个复制格, 我们将 U 的继承格连同这些复制格通称为 U 的导出格, 仍称 U 是这些导出格的原格, 即每个 $\text{DOM}_B[P]$ 中的单元格均有 n 个导出格。称第 $n-i$ 次执行扩展带复制算法产生的导出格为 U 的第 $i+1$ 个导出格, $i+1$ 为该导出格的序号, 特别地, U 的继承格也称为第 1 个导出格。

显然, 算法完成时导出表上所有单元格都是继承格或导出格。

针对未扩展格 P 执行主格扩展算法的过程称为扩展 P 。未扩展格被扩展后, 其导出格都成为已扩展格, 而其未扩展子格则会被复制成多个未扩展子格。

根据定理 3.1 和定理 3.4 并结合主格扩展算法的内容, 显然有如下结论:

定理 3.1 推论 对任何报表执行主格扩展算法的结果是报表。

定理 3.4 推论 主格扩展算法对实报表、(满)正实报表、整报表封闭。

并且类似可知, 主格扩展算法对实报表、(满)正实报表、

整报表反封闭。

相比更为基础的扩展带复制算法,主格扩展算法在处理报表扩展过程中更经常被作为一个基本操作。和前面一样,我们需要研究在主格扩展后报表上单元格与扩展带的变换性质。

类似地,下面的结论都以纵主格为例叙述,对于横主格有对称的结论。

设报表 T 上纵扩展格 P 的扩展带 $B[P]=R(p,q), p \leq q$, 记 $k=|B[P]|=q-p+1$, $n=|P.vall|$, 将单元格 U 的继承格记作 $\underline{U}$, 第 i 个导出格记作 U_i , 设 $U=M(a,b,c,d), a \leq c, b \leq d$ 。

定理 3.5 采用上述符号, 扩展 P 后, 有:

1. 若 $c < q$ 且 $U \notin \text{DOM}_B[P]$, 则 $\underline{U}=M(a,b,c,d)$, U_i 不存在;
2. 若 $c \geq q$ 且 $a \leq p$ 且 $U \notin \text{DOM}_B[P]$, 则 $\underline{U}=M(a,b,c+k*(n-1),d)$, U_i 不存在;
3. 若 $c \geq q$ 且 $p < a \leq q$ 且 $U \notin \text{DOM}_B[P]$, 则 $\underline{U}=M(a,b,q,d)$, U_i 不存在;
4. 若 $a > q$, 此时 $U \notin \text{DOM}_B[P]$, 则 $\underline{U}=M(a,b,c,d)+k*(n-1)$, U_i 不存在;
5. 若 $U \in \text{DOM}_B[P]$, 此时 $p \leq a \leq c \leq q$, 则 $\underline{U}=M(a,b,c,d)$ 和 $U_i=M(a,b,c,d)+k*(i-1)$ 。

【应用定理 3.2 对每种情况逐步执行算法并即可得到相应结论。】

推论 1 扩展纵主格后, 继承格和导出格的列号、列数与原格相同。行号、行数相同的单元格的继承格及其对应序号的导出

格的行号、行数也对应相同。

【考查定理中 U 和 $\underline{U}$ 与 U_i 的行号、行数、列号、列数的表达式即知。】

推论 2 扩展整报表上的纵主格后, 任何继承格和导出格的行数和列数均与其原格相同。

【整报表上单元格与扩展带之间不存在定理中 2,3 这两种情况, 而考查其他情况可知命题成立。】

推论表明整报表扩展时其单元格的大小(行数、列数)不变化。利用这一特点可以设计出存储效率更高算法实现方案。

再来考查主格扩展格其他扩展带的变化规律。

定理 3.6 采用上述符号, 设 Q 是 T 上扩展格, 在扩展纵主格 P 后有:

1. Q 是横主格时, $B[Q]=B[Q]$;
 Q 是纵主格时, 设 $B[Q]=R(u,v), u \leq v$
2. 若 $v < q$ 且 $Q \notin \text{DOM}_B[P]$, 则 $B[Q]=R(u,v)$;
3. 若 $v \geq q$ 且 $u \leq q$ 且 $Q \notin \text{DOM}_B[P]$, 则 $B[Q] \subseteq R(u, v+k*(n-1))$ 且 $R(u) \in B[Q]$; 其中若 $v=q$ 且 $u > p$, 则 $B[Q]=R(u,v)$;
4. 若 $u > q$, 此时 $Q \notin \text{DOM}_B[P]$, 则 $B[Q]=R(u,v)+k*(n-1)$; 以上情况中 Q' 均不存在。
5. $Q \in \text{DOM}_B[P]$, 此时 $p \leq u \leq v \leq q$, 则 $B[Q]=R(u,v)$ 和 $B[Q_i]=R(u,v)+k*(i-1)$ 。

【应用定理 3.3 对每种情况逐步执行算法即可得到相应结论。】

推论 1 扩展格的导出格的扩展带两两相离。

【考查定理中情况 5 即知。】

推论 2 扩展带相离的扩展格在扩展纵主格 P 后的继承格的扩展带仍相离；除了其一是 P 的后主格且另一的扩展带与 $B[P]$ 交叉的情况外，则可能出现的导出格的扩展带两两相离，导出格与另一扩展格的继承格的扩展带也相离。

【采用与证明定理 3.3 推论 2 类似的方法穷举定理 3.6 中的所有情况即可。】

推论 3 实报表上扩展带相离的扩展格在扩展纵主格后其继承格及可能出现的导出格的扩展带两两相离。

【由于实报表上不存在扩展带交叉的现象，由推论 2 可知命题成立。】

另外在主格扩展后关于扩展格的辖区，我们还有：

定理 3.7 对报表 T 上纵主格 U 和横主格 V ，在扩展某纵主格 P 后，有

1. $DOM[U, V]$ 或 $DOM[U_b, V]$ 与 $DOM[U, V]$ 基于取原格映射一一对应；
2. $DOM[U, @]$ 或 $DOM[U_b, @]$ 与 $DOM[U, @]$ 基于取原格映射一一对应；
3. $DOM[@, V]$ 与 $DOM[@, V]$ 基于取原格映射一一对应。

【若 $U \notin DOM_B[P]$ ，则 $DOM[U] \cap DOM_B[P] = \emptyset$ ，此时 $DOM[U]$ 中单元格及其主格均没有导出格，从而 $DOM[U, V]$ 与 $DOM[U, V]$ 中的单元格基于继承格与原格的对应关系（相当于等同关系）一

一一对应。

若 $U \in \text{DOM}_B[P]$, 则 $\text{DOM}[U] \subseteq \text{DOM}_B[P]$, 此时 $\text{DOM}[U]$ 中每个单元格将与 P 一起产生出 $n(|P_{\text{val}}|)$ 个导出格, 所有第 i 个导出格将构成 U_i 的辖区, 而导出格的上主格是其原格的上主格的继承格, 因而仍可用第 i 个导出格与原格的对应关系在 $\text{DOM}[U, V]$ 与 $\text{DOM}[U_i, V]$ 中的单元格之间一一对应。

即命题 1 成立。

用 $@$ 取代 $V, \underline{V}$ 重复上述论证过程, 其结论仍正确, 即命题 2 成立。

同时, 显然有 $\text{DOM}[@_x] \cap \text{DOM}_B[P] = \emptyset$, 则 $\text{DOM}[@_x]$ 中单元格及其主格均没有导出格, 从而 $\text{DOM}[@, \underline{V}]$ 与 $\text{DOM}[@, V]$ 中单元格可以用继承格与原格的对应关系 (相当于等同关系) 一一对应, 即命题 3 成立。】

定理 3.7 表明纵主格与横主格的交辖区在扩展过程中在某种意义下保持不变 (一一对应), 这个结论将确保后文中提出的层次坐标的合理性。

显然还可以得到下面的推论:

推论 对报表 T 上纵主格 U 和横主格 V , 在扩展某纵主格 P 后, 有

1. $|\text{DOM}[U_i, \underline{V}]| = |\text{DOM}[\underline{U}, \underline{V}]| = |\text{DOM}[U, V]|$;
2. $|\text{DOM}[U_i, @]| = |\text{DOM}[\underline{U}, @]| = |\text{DOM}[U, @]|$;
3. $|\text{DOM}[@, \underline{V}]| = |\text{DOM}[@, V]|$ 。

3.2.2 交换性

我们对主格扩展算法再做如下约定:

规则 2.2 主格扩展算法只能针对主格是已扩展格的未扩展

格执行。

在规则 2.2 下主格扩展算法对实报表有个良好的性质, 即下面的重要结论:

定理 3.8 (扩展交换定理) 主格扩展算法对于实报表可以交换, 即若在实报表 T 上同时可找到两个主格是已扩展格的未扩展格 P 和 Q , 则有:

1. 扩展 P 后 Q 没有导出格, 且仍是主格是已扩展格的未扩展格, 反之亦然;
2. 先扩展 P 后扩展 Q 和先扩展 Q 后扩展 P 得到的结果相同。

【显然未扩展格不可能是已扩展格的主格。由于 Q 是未扩展格和 P 是已扩展格, 则 $Q \notin \text{DOM}_B[P]$, 因此扩展 P 后 Q 没有导出格且扩展状态不变, Q 的扩展状态也不会改变; 对称地反之亦然, 因而命题 1 成立。

约定一些记号。在 T 上扩展 P 后的导出表记作 T_P , 再扩展 Q 后的导出表记作 T_{PQ} ; 而在 T 上扩展 Q 后的导出表记作 T_Q , 再扩展 P 后的导出表记作 T_{QP} 。

将 T 上的单元格 U 在 T_P, T_Q, T_{PQ}, T_{QP} 上的继承格分别记作 U_P, U_Q, U_{PQ}, U_{QP} ; U 在 T_P, T_Q 上的第 i 个导出格 (如果有的话, 以下同) 分别记作 U_{Pi} 和 U_{Qi} ; U_{Pi} 和 U_{Qi} 在 T_{PQ} 和 T_{QP} 上的继承格分别记作 U_{PiQ} 和 U_{QiP} ; U_P 和 U_{Pi} 在 T_{PQ} 上的第 j 个导出格分别记作 U_{PQj} 和 U_{PiQj} ; U_Q 和 U_{Qi} 在 T_{QP} 上的第 j 个导出格分别记作 U_{QPj} 和 U_{QiPj} 。

我们先证明 U_{PQ} 与 U_{QP} 及相关导出格的成员 (元素格) 对应相同。

当 P 和 Q 的扩展方向不同时, 不妨设 P 是纵主格而 Q 是横主格。

根据定理 3.6 可知, $B[P_Q]=B[P]$ 和 $B[Q_P]=B[Q]$; 再根据定理 3.5 推论 1 可知 U 与 U_Q 的行号、行数相同, U 与 U_P 的列号、列数相同。这样, $U \rightarrow U_P$ 和 $U_Q \rightarrow U_{QP}$ 中列号、列数不变, 而行号、行数的变动相同; $U \rightarrow U_Q$ 和 $U_P \rightarrow U_{PQ}$ 中行号、行号不变, 而列号、列数变动相同。则 U_{PQ} 与 U_{QP} 的行号、行数、列号、列数必然都相同, 即 $U_{PQ}=U_{QP}$ 。

类似地, 在 $B[P]$ 确定时, U_{PiQ} 的左主格、行号、行数与 U_{Pi} 相同, 即仅与 i 和 U 的左主格、行号、行数及 i 相关; 而 U_{QiP} 的左主格、行号、行数也仅与 i 和 U_Q 的左主格、行号、行数, 即 U 的左主格、行号、行数相关, 则可知 U_{PiQ} 和 U_{QP_i} 的左主格、行号、行数相同。对称地, 在 Q 确定时, U_{PiQ} 和 U_{QP_i} 的上主格、列号、列数也相同。即 $U_{PiQ}=U_{QP_i}$ 且主格相同, 同理有 $U_{PQ_i}=U_{QiP}$ 且主格相同。

同样过程可以证明 $U_{PiQ}=U_{Q_jPi}$ 且主格相同。

如果 P 和 Q 的扩展方向相同, 不妨设 P, Q 都是纵主格。由 T 是实报表可知 $B[P]$ 与 $B[Q]$ 相离, 则 U 不可能同时在 $B[P]$ 和 $B[Q]$ 之内。

设 $B[P]=R(a_P, b_P)$, $B[Q]=R(a_Q, b_Q)$, 不妨设 $a_P \leq b_P < a_Q \leq b_Q$ 。
记

$$k_P = |B[P]| = (b_P - a_P + 1), \quad s_P = k_P * |P.val| > 0$$

和

$$k_Q = |B[Q]| = (b_Q - a_Q + 1), \quad s_Q = k_Q * |Q.val| > 0$$

由定理 3.6 有

$$B[Q_P] = R(a_Q, b_Q) + s_P, \quad B[P_Q] = R(a_P, b_P)$$

设 $U=M(a,b,c,d)$, 根据定理 3.5 分情况讨论 U_P, U_Q, U_{PQ}, U_{QP} 的变化。

1. U 与 $B[P]$ 和 $B[Q]$ 均不相交时, 显然 $U \notin \text{DOM}_B[P]$ 且

$$U \notin \text{DOM}_B[Q]$$

若 $c < a_P < a_Q$, 则 $U_P=M(a,b,c,d)=U_Q$, 继而

$$U_{PQ}=M(a,b,c,d)=U_{QP}$$

若 $b_P < a \leq c < a_Q$, 则 $U_P=M(a,b,c,d)+n_P$, $U_Q=M(a,b,c,d)$, 继而

$$U_{PQ}=M(a,b,c,d)+n_P=U_{QP}$$

若 $b_P < b_Q < a$, 则 $U_P=M(a,b,c,d)+n_P$, $U_Q=M(a,b,c,d)+n_Q$, 继而

$$U_{PQ}=M(a,b,c,d)+(n_P+n_Q)=U_{QP}$$

2. U 与 $B[P]$ 相交而与 $B[Q]$ 不相交时, 则必有 $c < a_Q$ 和 $a \geq b_P$ 先考查 $U \notin \text{DOM}_B[P]$ 的情况。

若 $c < b_P$, 则 $U_P=M(a,b,c,d)=U_Q$, 继而

$$U_{PQ}=M(a,b,c,d)=U_{QP}$$

若 $c \geq b_P$ 且 $a \leq a_P$, 则 $U_P=M(a,b,c+n_P,d)$, $U_Q=M(a,b,c,d)$, 继而

$$U_{PQ}=M(a,b,c+n_P,d)=U_{QP}$$

若 $c \geq b_P$ 且 $a > a_P$, 则 $U_P=M(a,b,b_P,d)$, $U_Q=M(a,b,c,d)$, 继而

$$U_{PQ}=M(a,b,b_P,d)=U_{QP}$$

若 $U \in \text{DOM}_B[P]$, 则 $U_P=M(a,b,c,d)=U_Q$, 继而

$$U_{PQ}=M(a,b,c,d)=U_{QP}$$

此时扩展 P 时 U 有导出格, 同样分析可知

$$U_{Pi}=M(a,b,c,d)+(i-1)*k_P,$$

则

$$U_{PiQ}=M(a,b,c,d)+(i-1)*k_P=U_{QP_i}$$

而且明显 U_{PiQ} 与 U_{QP_i} 的主格也相同。

3. U 与 $B[P]$ 不相交而与 $B[Q]$ 相交时, 则必有 $a > b_P$ 且 $c \geq a_Q$ 。

注意由于已经假定了 P 与 Q 的方位, 此种情况与情况 2 并不完全对称。

先考查 $U \notin \text{DOM}_B[Q]$ 的情况。

若 $c < b_Q$, 则 $U_P = M(a, b, c, d) + n_P$, $U_Q = M(a, b, c, d)$, 继而

$$U_{PQ} = M(a, b, c, d) + n_P = U_{QP}$$

若 $c \geq b_Q$ 且 $a \leq a_Q$, 则 $U_P = M(a, b, c, d) + n_P$, $U_Q = M(a, b, c + n_Q, d)$,

继而

$$U_{PQ} = M(a, b, c + n_Q, d) + n_P = U_{QP}$$

若 $c \geq b_Q$ 且 $a > a_Q$, 则 $U_P = M(a, b, c, d) + n_P$, $U_Q = M(a, b, b_Q, d)$,

继而

$$U_{PQ} = M(a, b, b_P, d) + n_P = U_{QP}$$

若 $U \in \text{DOM}_B[Q]$, 则 $U_P = M(a, b, c, d) + n_P$, $U_Q = M(a, b, c, d)$,

继而

$$U_{PQ} = M(a, b, c_P, d) + n_P = U_{QP}$$

此时扩展 Q 时 U 有导出格, 同样分析可知

$$U_{Qi} = M(a, b, c, d) + (n_P + (i-1) * k_Q)$$

则

$$U_{Qip} = M(a, b, c, d) + (n_P + (i-1) * k_Q) = U_{PQi}$$

而且明显 U_{Qip} 与 U_{PQi} 的主格也相同。

4. U 与 $B[P]$ 和 $B[Q]$ 均相交时, 此时也有 $U \notin \text{DOM}_B[P]$ 且

$U \notin \text{DOM}_B[Q]$, 并且 $a \leq b_P$ 和 $c \geq a_Q$

若 $a \leq a_P$ 且 $c < b_Q$, 则 $U_P = M(a, b, c + n_P, d)$, $U_Q = M(a, b, c, d)$,

继而

$$U_{PQ} = M(a, b, c + n_P, d) = U_{QP}$$

若 $a \leq a_P$ 且 $c \geq b_Q$, 则 $U_P = M(a, b, c + n_P, d)$, $U_Q = M(a, b, c + n_Q, d)$,

继而

$$U_{PQ}=M(a,b,c+n_P+n_Q,d)=U_{QP}$$

若 $a > a_P$ 且 $c < b_Q$, 则 $U_P=M(a,b,b_P,d)$, $U_Q=M(a,b,c,d)$, 继而

$$U_{PQ}=M(a,b,b_P,d)=U_{QP}$$

若 $a > a_P$ 且 $c \geq b_Q$, 则 $U_P=M(a,b,b_P,d)$, $U_Q=M(a,b,c+n_Q,d)$,

继而

$$U_{PQ}=M(a,b,b_P,d)=U_{QP}$$

综合上面的讨论可知, $U_{PQ}=U_{QP}$ 且相关的导出格及其主格也对应相同。

算法中还设置了 P, Q 的 val 和扩展状态属性, 这些设置仅与 P, Q 的导出格相关, 而且扩展 $P(Q)$ 时 $Q(P)$ 没有导出格, 则 $P_{PiQ}.val=P_{Pi}.val$ 是 $P.val$ 的第 i 个成员, 而 $P_{QP_i}.val$ 也是 $P_Q.val=P.val$ 的第 i 个成员, 即有 $P_{PiQ}.val=P_{QP_i}.val$, 对称地有 $Q_{PQ_i}.val=Q_{QiP}.val$, 同理还可证明它们的扩展状态也对应相同。】

需要指出的是, 对于非实报表, 主格扩展算法并不具有可交换性。

3.3 报表扩展

3.3.1 报表扩展

算法 2.3 报表扩展算法。

1. 在报表上寻找一个可执行主格扩展算法的未扩展格(其主格是已扩展格), 如果找不到则算法结束;
2. 扩展找到的未扩展格(算法 2.2);
3. 跳转 1 重复执行。

我们知道,在主格扩展时可能会产生更多的未扩展格,但这个动作并不会使报表上的未扩展格无限增多,对此我们有:

定理 3.9 (扩展停机定理) 对任何报表执行报表扩展算法 (算法 2.3) 必在有限步内结束,并且结束时报表上不再有任何未扩展格。

【根据定理 3.1 及其推论和算法过程,如果算法能在有限步内结束,其结果必定仍是报表,我们只要证明算法的停机性。

设报表 T 的深度为 n , T 上深度为 i 的未扩展格个数为 u_i , 显然 $u_i \geq 0$ 。

考查主格扩展算法过程可知,在扩展 T 上深度为 k 的未扩展格 P 后,非负整数构成的数组 $u=(u_1, \dots, u_n)$ 将会产生如下变化:

- $i < k$ 的 u_i 不变;
- $i = k$ 的 u_i (即 u_k) 减小 1;
- $i > k$ 的 u_i 可能变大。

这相当于对数组 u 做如下动作:

1. 选择自然数 $k \leq n$ 使 $u_k > 0$;
2. 做 $u_k \rightarrow u_k - 1$; $i < k$ 时的 u_i 不变; $i > k$ 时的 u_i 随意调整。

显然当所有 $u_i = 0$ 时,动作将不能再进行,而此时对应的报表上已没有未扩展格,报表扩展算法也必将停止。

而若有某个 $u_i \neq 0$ 时,即报表上仍有未扩展格,则可在其主格线上找到第一个未扩展格执行扩展 (其主格必是已扩展格),即此时动作必能再进行。

定义如下命题:

$Q(u)$ 对于非负整数构成的数组 u 反复执行上述动作,经过有限次后 u 的所有成员都将变成 0,即动作结束。

$P(n)$ 对任何由非负整数构成的长度为 n 的数组 u , $Q(u)$ 成立。

$P(n, m)$ 对任何由非负整数构成的长度为 n 的数组 $u=(u_1, \dots, u_n)$, 如果 $u_1 \leq m$, 则 $Q(u)$ 成立。

显然 $P(1)$ 成立, 假设 $P(n)$ 成立。

设 $u=(u_1, \dots, u_{n+1})$ 是长度为 $n+1$ 的非负整数数组, 其中 $u_1=0$ 。显然不可能对 u 执行 $k=1$ 的动作, 即 u_1 不会再变化; 而 $k>1$ 的动作相当于仅对长度为 n 的数组 $v=(u_1, \dots, u_{n+1})$ 执行, 由于 $P(n)$ 成立, 可知这些动作会在有限次内结束。

即 $P(n+1, 0)$ 成立, 假设 $P(n+1, m)$ 成立。

设 $u=(u_1, \dots, u_{n+1})$ 是长度为 $n+1$ 的非负整数数组, 其中 $u_1=m+1$ 。对 u 执行上述动作时, 由于 $P(n)$ 成立, 则不可能无限次地执行 $k>1$ 的动作, 在有限次内必会执行一次 $k=1$ 的动作, 这将使 $u_1=m$, 由 $P(n+1, m)$ 成立可知再执行有限次后必停止。

即 $P(n+1, m+1)$ 成立。由归纳法可知对任何自然数 n 和非负整数 m , $P(n, m)$ 和 $P(n)$ 都成立, 即对任何非负整数数组 u , $Q(u)$ 成立。】

扩展停机定理是报表扩展的重要依据, 它确保了扩展算法最终能够得到一个结果。我们将对报表 T 执行报表扩展算法的过程称为扩展 T 。

根据扩展停机定理和定理 3.4 并考查报表扩展算法的过程, 显然有:

定理 3.10 (扩展封闭定理) 报表扩展算法对实报表、(满) 正实报表、整报表封闭。

类似地, 我们还可知报表扩展算法对实报表、(满) 正报表、整报表反封闭。

另外, 扩展交换定理 (定理 3.8) 表明, 采用报表扩展算法

扩展实报表时,其结果不依赖于算法第1步中主格的选择。一般情况下,由于报表的列数远比行数要少,先进行列方向变换会获得更高的运算效率,因此应尽量先扩展横主格后扩展纵主格。

而扩展非实报表则与第1步对未扩展格的选择有关,在需要保证一致性时必须指明选择方案,如未指明一般会选择次序在前边的。

3.3.2 源格

设扩展报表 T 的过程中将执行 n 次主格扩展算法,第 i 次主格扩展后的导出表记为 T_i ,并记 $T_0=T$ 。当 $0 \leq i \leq n$ 时,称 T_0 是 T_i 的源表, T_i 是 T_0 或 T_n 的中间表, T_n 是 T_i 的实例表,数组 $(T_0, T_1, \dots, T_k)$ 称为中间表 T_k 的扩展序列。注意这里 i 可以为 0,即 T 是自己的源表和中间表。

归纳定义 T_k 上单元格 u 的源格如下:

1. $k=0$ 时, u 的源格是 u 本身;
2. $k>0$ 时, u 的源格是 u 在 T_{k-1} 上的原格的源格。

主格扩展的导出表上所有单元格都有原格,因而中间表上所有单元格都有且只有一个源格;同时由于在主格扩展时原格与继承格或导出格的扩展方向相同(或同时无定义),因而非(纵、横)主格的源格也必是非(纵、横)主格。

单元格 u 的源格记作 $\text{src}(u)$,并特别规定 $\text{src}(@)=@$ 。称中间表上两个源格相同的单元格同源或互为同源格。

定理 3.11 关于源格的一些结论。在中间表上,有:

1. 单元格的(左、上)主格的源格是其源格的(左、上)主格,从而同源格的(左、上)主格同源;

2. 扩展格的子(属)格的源格是其源格的子(属)格,从而扩展格辖区内的单元格的源格属于其源格的辖区;

3. 同源纵(横)主格的列(行)号、列(行)数均相同,而行(列)号互不相同;

4. 纵主格和横主格的交辖区与其源格的交辖区基于取源格映射(src)一一对应;扩展格和@的交辖区与其源格和@的交辖区基于取源格映射一一对应;从而这些对应的交辖区成员数量相同;

5. 同源扩展格辖区内单元格数量相同,且都不小于其源格辖区内单元格数量;

6. 同源扩展格的扩展带相离;

7. 整报表上单元格的行数、列数与其源格相同。

【对中间表的扩展序列长度归纳,设中间表 T_k 的扩展序列为 $(T_0, T_1, \dots, T_k)$ 。

$k=0$ 时,由于 T_0 上与单元格的同源格仅有其本身,此时显然所有命题成立。

$k>0$ 时,考查从 T_{k-1} 到 T_k 的主格扩展算法过程,设算法针对 T_{k-1} 上扩展格 P 进行。将 T_k 上单元格 u, v 在 T_{k-1} 上的原格分别记作 U, V 。

设 $u.xp$ 有定义。

P 是纵主格时,若 $U \notin \text{DOM}_B[P]$,显然 $u.xp$ 是 $U.xp$ 的继承格;若 $U=P$,则 $u.xp$ 是 $U.xp$ 的继承格;若 $U \in \text{DOM}_B[P]$ 且 $U \neq P$,则 $u.xp$ 是 $U.xp$ 的导出格。

P 是横主格时,仍有 $u.xp$ 是 $U.xp$ 的继承格。

综合两种情况可得, $u.xp$ 的原格将是 $U.xp$,则应用归纳假设有

$$\text{src}(u.\text{xp})=\text{src}(U.\text{xp})=\text{src}(U).\text{xp}=\text{src}(u).\text{xp}$$

同理, 若 $u.\text{yp}$ 有定义时也有

$$\text{src}(u.\text{yp})=\text{src}(U.\text{yp})=\text{src}(U).\text{yp}=\text{src}(u).\text{yp}$$

即命题 1 成立。

设 u 是扩展格, 其子格 c 在 T_{k-1} 上的原格为 C 。由命题 1 可知, $c.\text{pu}(=u)$ 的原格必是 $C.\text{pu}$, 即 $C.\text{pu}=U$, 应用归纳假设有

$$u=c.\text{pu} \Rightarrow U=C.\text{pu} \Rightarrow \text{src}(U)=\text{src}(C).\text{pu} \Rightarrow \text{src}(u)=\text{src}(c).\text{pu}$$

显然将上面分析中的 pu (主格) 换成 xp (左主格) 和 yp (上主格) 后等式仍将成立, 因而此结论对属格也正确, 即命题 2 成立。

设 u, v 是同源纵主格。

P 是纵主格时, 算法不会改变单元格的列号、列数, 则应用归纳法有

$$\text{col}(u)=\text{col}(U)=\text{col}(V)=\text{col}(v)$$

及

$$\text{cols}(u)=\text{cols}(U)=\text{cols}(V)=\text{cols}(v)$$

P 是横主格时, $U \notin \text{DOM}_B[P]$, 由定理 3.2 的推论可知, 对此类单元格列号、列数的改变仅依赖于算法执行前的列号、列数, 而 U 没有导出格, 应用归纳假设有

$$\text{col}(U)=\text{col}(V) \ \&\& \ \text{cols}(U)=\text{cols}(V)$$

$$\Rightarrow \text{col}(u)=\text{col}(v) \ \&\& \ \text{cols}(u)=\text{cols}(v)$$

即同源纵主格的列号、列数均相同, 由定理 2.1 可知其行号互不相同。

同理对横主格有对称的结论, 即命题 3 成立。

设 u 是纵主格, v 是横主格, 由定理 3.7 有 $\text{DOM}[u, v]$ 与 $\text{DOM}[U, V]$ 基于取原格映射一一对应, 而由归纳假设又有 $\text{DOM}[U, V]$ 与 $\text{DOM}[\text{src}(u), \text{src}(v)]$ 基于取源格映射一一对应, 而原

格的源格仍是源格, 因此有 $\text{DOM}[u, v]$ 与 $\text{DOM}[\text{src}(u), \text{src}(v)]$ 基于取源格映射一一对应。

同样过程可以证明 $\text{DOM}[u, @]$ 与 $\text{DOM}[\text{src}(u), @]$ 基于取源格映射一一对应以及 $\text{DOM}[@, v]$ 与 $\text{DOM}[@, \text{src}(v)]$ 基于取源格映射一一对应。即命题 4 成立。

设 u, v 是以 C 为源格的同源纵主格, 由命题 4 可知, 对任何横主格 q 或 $@$, 有

$$|\text{IDOM}[u, q]| = |\text{IDOM}[C, \text{src}(q)]| = |\text{IDOM}[v, q]|$$

和

$$|\text{IDOM}[u, @]| = |\text{IDOM}[C, @]| = |\text{IDOM}[v, @]|$$

单元格只有唯一上主格, 横主格只有唯一源格, 记 $Y_i = \text{US}_Y[T_i] \cup \{@\}$, 则有

$$\begin{aligned} |\text{IDOM}[u]| &= \sum(\{|\text{IDOM}[u, q]| \mid q \in Y_k\}) \\ &\geq \sum(\{|\text{IDOM}[C, q]| \mid q \in Y_0\}) \end{aligned}$$

从而可知

$$|\text{IDOM}[u]| = |\text{IDOM}[v]| \geq |\text{IDOM}[C]|$$

对称地, 当 u, v 是横主格时上式也正确, 即命题 5 成立。

设 u, v 是同源纵主格。

P 是纵主格时, 若 $U, V \notin \text{DOM}_B[P]$ 或 $U, V \in \text{DOM}_B[P]$ 或 $U=P$ 或 $V=P$, 则均可应用定理 3.6 推论 2 并结合归纳假设有

$$B[U] \cap B[V] = \emptyset \Rightarrow B[u] \cap B[v] = \emptyset$$

若 U, V 中仅有一个属于 $\text{DOM}_B[P]$ 均不是 P , 设为 U , 则 U 有先主格为 P ; 根据命题 1 可知 V 必有先主格与 P 同源, 设为 Q , 则 $V \notin \text{DOM}_B[P] \Rightarrow P \neq Q$; 而归纳假设有 $B[P]$ 与 $B[Q]$ 相离, 则 $B[P]$ 不可能与 $B[V] \subseteq B[Q]$ 交叉, 仍应用定理 3.6 推论 2 并结合归纳假设有

$$B[U] \cap B[V] = \phi \Rightarrow B[u] \cap B[v] = \phi$$

P 是横主格时, $U, V \notin \text{DOM}_B[P]$, 即 u, v 是继承格, 由定理 3.6 及归纳假设有

$$B[u] \cap B[v] = B[U] \cap B[V] = \phi$$

对称地, 当 u, v 是横主格时上述论证也正确, 即命题 6 成立。应用命题 6 可再次证明命题 3 的后半句论断。

设 T_i 是整报表, 根据定理 3.5 推论 2 及归纳假设有

$$\text{rows}(u) = \text{rows}(U) = \text{rows}(\text{src}(u))$$

和

$$\text{cols}(u) = \text{cols}(U) = \text{cols}(\text{src}(u))$$

即命题 7 成立。】

对中间表 T 上的纵(横)主格 p , 将其在 T 上的同源格按行(列)号从小到大排序后, p 所在的序号称为 p 的同源号, 记作 $\text{seq}(p)$, 有时写作 $\#p$, 由定理 3.11 可知, $\text{seq}(p)$ 能被唯一确定。另外特别规定, $\text{seq}(@) = 1$ 。

注意, 只有主格才有同源号, 非主格没有同源号。

设中间表 T 的扩展序列为 $(T_0, \dots, T_k)$, p, q 是 T 上的两个不是 $@$ 的已扩展格(此时必有 $k > 0$), 递归定义 p, q 同束的概念:

1. T_k 是扩展 T_{k-1} 上未扩展格 P 后的导出表且 p, q 是 P 的导出格时称 p, q 同束, $p(q)$ 作为 P 的导出格的序号称为 $p(q)$ 的同束号;
2. p, q 在 T_{k-1} 上的原格同束时称 p, q 同束, $p(q)$ 的同束号与其原格相同。

同束的已扩展格也互称为同束格, 同束格是同一个未扩展格的导出格或其继承格, 因而显然同束格同源。

任何已扩展格均在扩展其未扩展的原格时产生,并同时获得同束号;在后续扩展变换中将只有继承格而只能继承同束号,因此其同束号可唯一确定。

已扩展格 p 的同束号记作 $\text{ord}(p)$,有时写作 $\&p$;为叙述方便,规定未扩展格 p 的同束号 $\text{ord}(p)=1$ 和 $\text{ord}(@)=1$ 。

定理 3.12 同束格的一些结论,在中间表上,有

1. 同源扩展格同束的充要条件是其主格相同;
2. 同束格的同源号与其同束号之差是定值,故同束格的同源号连续。

【显然扩展未扩展格时产生的同束格的主格相同,而在后续的扩展中,同束格及其共同主格均是已扩展格而只有继承格,将继续保持同束格的主格相同。即命题 1 的必要性成立。

对中间表的扩展序列长度归纳来证明充分性,归纳基础(长度为 0)时中间表上没有同源格的主格相同,命题成立。

设 u, v 是扩展纵主格 P 后产生的中间表上的主格相同的同源扩展格,记 u, v 的在主格扩展中的原格为 U, V 。由于主格扩展中继承格或导出格的主格必是其原格的主格的继承格或导出格,即单元格的主格的原格是其原格的主格,则 U 与 V 的主格也相同。

$U \neq V$ 时,由归纳假设知 U, V 同束,从而由定义有 u, v 也同束。

$U = V$ 时, u, v 均以 U 为原格,则必有 $U \in \text{DOM}_B[P]$ 和 u, v 是 U 的导出格。如果 $U \neq P$, 则 $U.pu \in \text{DOM}_B[P]$ 也会有导出格,这样 U 的各个导出格的主格将互不相同,这与条件矛盾,故必然有 $U = P$, 从而 u, v 同束。

综合可知,命题 1 成立。

设 P 是中间表上未扩展纵主格,记 $|P.\text{vall}|=n$,由定理 3.6 可知,

扩展 P 后的导出格 $P_i (1 \leq i \leq n)$ 的行号大小与 i 次序相同。

设扩展前 P 有同源格 Q (不一定未扩展), 由定理 3.11 知 $B[P]$ 与 $B[Q]$ 相离, 则 Q 在扩展 P 后没有导出格, 再由定理 3.5 可知

$$\text{row}(Q) < \text{row}(P) \Rightarrow \text{row}(Q) < \text{row}(P_i) \Rightarrow \text{seq}(Q) < \text{seq}(P_i)$$

或

$$\text{row}(Q) > \text{row}(P) \Rightarrow \text{row}(Q) > \text{row}(P_i) \Rightarrow \text{seq}(Q) > \text{seq}(P_i)$$

即 $\text{seq}(Q)$ 不可能落某两个 $\text{seq}(P_i)$ 之内, 则对任何 $1 \leq i, j \leq n$ 有

$$\text{seq}(P_i) - \text{seq}(P_j) = i - j = \text{ord}(P_i) - \text{ord}(P_j)$$

$$\Rightarrow \text{seq}(P_i) - \text{ord}(P_i) = \text{seq}(P_j) - \text{ord}(P_j)$$

再由定理 3.6 可知, 列号相同的单元格在主格扩展后的继承格的行号大小关系仍将保持, 即在后续扩展中 Q 的继承格的行号仍会大于或小于所有 P_i 的行号, 则不可能有其他同源继承格的同源号落在两个 $\text{seq}(P_i)$ 之间。

若某次扩展中 Q 有导出格, 由上面分析还可知对作为同源格的 P_i (P_j) 有

$$\text{row}(P) < \text{row}(Q) \Rightarrow \text{row}(P_i) < \text{row}(Q)$$

$$\Rightarrow \text{row}(P_j) < \text{row}(Q_j) \Rightarrow \text{seq}(P_j) < \text{seq}(Q_j)$$

或

$$\text{row}(P) > \text{row}(Q) \Rightarrow \text{row}(P_i) > \text{row}(Q)$$

$$\Rightarrow \text{row}(P_j) > \text{row}(Q_j) \Rightarrow \text{seq}(P_j) > \text{seq}(Q_j)$$

即也没有导出格的同源号落在两个 $\text{seq}(P_i)$ 之间, 故 P_i 在后续扩展中其继承格的同源号仍将保持连续。

显然, 对于横主格有对称的结论, 即命题 2 成立。】

同束号和同源号是我们下面设计单元格引用方案的基础。

4 语 法

4.1 层次坐标

4.1.1 基格

我们开始研究报表上表达式的书写与计算规则。

与常规的表达式类似，报表表达式也是由一些基本项（代表数据的符号）通过运算符（其实也是一种函数）连接书写而成，并按通行的计算规则计算（先乘除后加减、括号优先等）。

与常规表达式不同的是，为了完成单元格的格值计算，报表表达式除了要引用一般数据外，更关键的是要在表达式中能引用到单元格的格值和外部的数据来源的数据，如何引用这些内容是我们研究的重点。

报表表达式通常会作为单元格的属性（或一部分）存在，其计算结果与该单元格有很大相关性（下面会谈许多引用方式是相对的），书写相同的表达式在不同单元格属性中计算结果可能完全不同。也就是说，计算报表表达式时经常需要指明针对某个单元格进行，该单元格称为该表达式的基格。

基格对于表达式计算在大多数情况下是必需的，因此需要明确指明。但是，通常情况下表达式的基格能从上下文中可明显看出，因而就不再明确指出，如单元格的格表达式的基格即是该单元格本身。

4.1.2 层次坐标

表达式一般是在设计阶段确定的，也就是在源表中确定，而其计算时却可能引用到中间表（实例表）上的单元格。设计源表时并不知道中间表上有哪些单元格及其位置，因而无法采用平面坐标来引用。

我们需要有一种能在设计源表时可采用的对中间表单元格的引用方案。

对中间表 T 上某单元格 c ，设 $C=\text{src}(c)$ ：

1. 若 c 是非主格，设 c 的左主格线和上主格线分别为 $(p_0, \dots, p_n)$ 和 $(q_0, \dots, q_m)$ ，记

$$P_i = \text{src}(p_i), x_i = \text{ord}(p_i) \quad (0 \leq i \leq n)$$

和

$$Q_j = \text{src}(q_j), y_j = \text{ord}(q_j) \quad (0 \leq j \leq m)$$

则 c 可记作

$$C[P_0:x_0, P_1:x_1, \dots, P_n:x_n; Q_0:y_0, Q_1:y_1, \dots, Q_m:y_m]$$

由于总有 $P_0=Q_0=@$ 和 $x_0=y_0=1$ ，故可省略 $P_0:x_0$ 和 $Q_0:y_0$ 部分，简写为

$$C[P_1:x_1, \dots, P_n:x_n; Q_1:y_1, \dots, Q_m:y_m]$$

$m=0$ 时可简写为

$$C[P_1:x_1, \dots, P_n:x_n]$$

$n=0$ 时可简写为

$$C[; Q_1:y_1, \dots, Q_m:y_m]$$

$n=m=0$ 时（当且仅当 $c.xp=c.y_p=@$ 时发生）可直接简写为 C （此时 c 是 T 上唯一以 C 为源格的单元格，不会有歧义）。

2. 若 c 是扩展格, 设 c 的主格线为 $(p_0, \dots, p_n)$ (必有 $n \geq 1$), 记

$$P_i = \text{src}(p_i), x_i = \text{ord}(p_i) \quad (0 \leq i \leq n)$$

则 c 可记为

$$C[P_0:x_0, P_1:x_1, \dots, P_n:x_n]$$

由于总有 $P_0 = @$ 和 $x_0 = 1$, 故书写时可省略 $P_0:x_0$ 部分, 简写为

$$C[P_1:x_1, \dots, P_n:x_n]$$

上述写法称为单元格 c 的层次坐标, $x_i y_j$ 分别称为 c 在主格 $P_i Q_j$ 上的层次坐标, 其值显然不会为 0。同时称 $P_i Q_j$ 在 c 的层次坐标上。

显然, 中间表上任何单元格都有一个确定的层次坐标。

定理 4.1 中间表上不同单元格的层次坐标不同。

【设 u, v 是中间表上两个不同的单元格, 显然 u, v 不同源时其层次坐标不同。

设 u, v 同源, 由定理 3.11 可知 u, v 的 (左、上) 主格线长度相同。

当 u, v 是扩展格时, u, v 的主格线内必有对应 (序号相同) 成员不相同 (至少有 $u \neq v$), 设其中序号最小的对应不同成员为 p, q , 显然 p, q 同源且 $p.pu = q.pu$, 由定理 3.12 可知 p, q 同束, 而由 $p \neq q$ 又有 $\text{ord}(p) \neq \text{ord}(q)$, 即 u, v 至少在 $\text{src}(p)$ ($=\text{src}(q)$) 上的层次坐标不同。

当 u, v 是非主格时, 若 $u.xp \neq v.xp$, 则仍可重复上面论述证明 u, v 在其源格的左主格线内某个扩展格上的层次坐标不同, 对称地, $u.y_p \neq v.y_p$ 时也有同样结论。

若 $u.xp = v.xp = p$ 且 $u.y_p = v.y_p = q$, 为叙述方便先假设 $p \neq @$ 且

$q \neq @$ 。由定理 3.11 可知 $\text{DOM}[p, q]$ 与 $\text{DOM}[\text{src}(p), \text{src}(q)]$ 基于取源格映射一一对应, 而 $\text{src}(u) = \text{src}(v)$, 则必有 $u = v$, 这与条件矛盾, 即不可能发生这种现象。

考查定理 3.11 的表述可知, 当 $p = @$ 而 $q \neq @$ 或 $p \neq @$ 而 $q = @$ 时仍可重复上述论证过程。而当 $p = q = @$ 时, $\text{src}(u)$ 不在任何扩展格的辖区内, 在扩展过程中不会有导出格, 而只有唯一的继承格, 则必有 $u = v$, 仍与条件矛盾。】

定理 4.1 表明单元格的层次坐标唯一, 可以用来标志单元格。

采用层次坐标似乎能使我们拥有一种用源格表示中间表上单元格的方案, 但实际上, 除个别固定位置的引用外, 我们一般不清楚要引用的单元格在扩展后报表上的确切层次坐标, 而且单元格引用经常发生在另一单元格的格表达式中, 即针对某个基格的, 我们还需要有一种与基格相关的层次坐标。

对中间表上单元格 a, b , 若主格 P 同时在 a 和 b 的层次坐标上, 设 a, b 在 P 上的层次坐标分别为 x_a, x_b , 令 $x = |x_a - x_b|$, 则在以 a 为基格的表达式中书写 b 的层次坐标时, b 在 P 上的层次坐标 x_b 可写成如下形式:

1. $x_a \leq x_b$ 时写作 $+x$;
2. $x_a \geq x_b$ 时写作 $-x$;
3. $x_a = x_b$ 时写作 0。

这时称层次坐标是单元格 b 相对基格 a 的层次坐标, 其中 $\pm x, 0$ 称为单元格 b 相对基格 a 在 P 上的位移坐标, 当上下文中已确定或不关心基格时, 也简单地称为 b 在 P 上的位移坐标。

若单元格 c 在某主格 P 上的位移坐标为 0, 则其层次坐标中 P 及其位移坐标可省略不写; 若 c 在其层次坐标上所有主格上的位移坐标都是 0, 则其中的 $[]$ 部分可全部省略而简化成只剩源格名, 这时的层次坐标称为零层次坐标。

左主格和上主格均是 @ 的单元格的层次坐标也可能被简化写成只剩源格名, 显然这类单元格相对于任何单元格的位移坐标均为 0, 采用位移坐标时仍将写作形式相同的零层次坐标, 不会产生歧义。

零层次坐标在表达式中极为常见, 因此需要了解其产生的条件。显然, 任何单元格相对于其本身的层次坐标是零层次坐标。除此外, 我们还有:

定理 4.2 单元格 b 相对于基格 a 的层次坐标为零层次坐标的充要条件是:

1. b 是扩展格时, b 在 a 的主格线内;
2. a 是纵(横)主格且 b 是非主格时, $b \in \text{DOM}_A[a]$ 且 b 的上(左)主格为 @;
3. a, b 都是非主格时, $b \in \text{DOM}_A[a.xp] \cap \text{DOM}_A[a.yp]$ 。

【分命题罗列的情况讨论。

1. b 是扩展格。

若 b 在 a 的主格线内, 则 b 的主格线内任何主格 p 也在 a 的主格线内, a, b 在 $\text{src}(p)$ 上的层次坐标相同 (都是 $\text{ord}(p)$), 故 b 相对于 a 是零层次坐标。

若 b 相对于 a 是零层次坐标, 则作为 b 的层次坐标上的主格, $\text{src}(b)$ 也在 a 的层次坐标上, 即 a 的主格线上有主格 p 与 b 同源, 由充分性又知 p 相对于 a 也是零层次坐标, 即 p 与 b 的层次坐标

相同, 由定理 4.1 必有 $p=b$ 。

2. a 是纵 (横) 主格且 b 是非主格。

设 a 是纵主格。

$b \in \text{DOM}_A[a]$ 时 b 的左主格线内任何主格 p 也在 a 的主格线内, a, b 在 $\text{src}(p)$ 上的层次坐标相同, 而 $b.yp=@$ 则 b 在任何横主格上没有层次坐标, a 作为纵主格也不会有, 故 b 相对于 a 是零层次坐标。

若 b 相对于 a 是零层次坐标, 与 1 类似论证过程可证 $b.xp$ 在 a 的主格线内, 即有 $b \in \text{DOM}_A[a]$; 而 a 作为纵主格在任何横主格上没有层次坐标, 则 b 也没有, 故有 $b.yp=@$ 。

a 是横主格时的情况对称。

3. a, b 都是非主格。

若 $b \in \text{DOM}_A[a.xp]$, 则 b 的左主格线内任何主格也在 a 的左主格线内, 从而 a, b 在 b 的层次坐标上的任何纵主格上层次坐标相同, 同理由 $b \in \text{DOM}_A[a.yp]$ 有 a, b 在 b 的层次坐标上任何横主格上的层次坐标也相同, 即 b 相对于 a 是零层次坐标。

若 b 相对于 a 是零层次坐标, 与 1 类似论证过程可证 $b.xp$ 和 $b.yp$ 分别在 a 的左主格线和上主格线内, 即

$$b \in \text{DOM}_A[a.xp] \cap \text{DOM}_A[a.yp]$$

综合可知命题成立。】

通俗地讲, 定理 4.2 的内容可陈述为采用零层次坐标只能引用到基格导格的先辖区内的单元格。

采用允许位移坐标的层次坐标可以引用中间表上与基格相关的单元格 (绝大多数实际报表都是这类引用), 在设计源表时即使不知道扩展后报表上单元格的具体位置, 也可以正确地书写

引用这些单元格的表达式。

未特别指明时，下文中所提及的层次坐标中均可能含有位移坐标。

在表达式中出现的单元格（如其层次坐标）被缺省地理解为该单元格的格值。用 $c(x)$ 表示以单元格 c 为基格的表达式 x 的计算结果， $c(x)$ 是一个数而不再是一个单元格，称为 c 的计算数。

4.1.3 格集

中间表上某些单元格构成的集合称为格集，成员互相同源的格集称为同源格集。同源格集在报表计算时很常见，我们用下面的方式来表示同源格集。

1. 设 C 是源表上某非主格，其左主格线为 $(P_0, \dots, P_n)$ ，上主格线为 $(Q_0, \dots, Q_m)$ ，对任何 $0 \leq k \leq n$ ， $0 \leq l \leq m$ ，记

$$C[P_0:x_0, P_1:x_1, \dots, P_k:x_k; Q_0:y_0, Q_1:y_1, \dots, Q_l:y_l]\{\}$$

为某中间表上所有层次坐标为

$$C[P_1:x_1, \dots, P_k:x_k, P_{k+1}:u_{k+1}, \dots, P_n:u_n; Q_1:y_1, \dots, Q_l:y_l, Q_{l+1}:v_{l+1}, \dots, Q_m:v_m]$$

的单元格构成的集合。其中 $u_i, v_j \in \mathbb{N}$ ($k < i \leq n, l < j \leq m$)，并能使上述层次坐标有意义（所表示的单元格在中间表上存在）。

2. C 是源表上某扩展格，其左主格线为 $(P_0, \dots, P_n)$ ，对任何 $0 \leq k \leq n$ ，记

$$C[P_0:x_0, P_1:x_1, \dots, P_k:x_k]\{\}$$

为某中间表上所有层次坐标为

$$C[P_1:x_1, \dots, P_k:x_k, P_{k+1}:u_{k+1}, \dots, P_n:u_n]$$

的单元格构成的集合。其中 $u_i \in \mathbb{N}$ ($k < i \leq n$)，并能使上述层次坐标有意义（所表示的单元格在中间表上存在）。

上述写法称为同源格集的层次坐标，其中 x_i, y_j ($0 \leq i \leq k, 0 \leq$

$j \leq l$) 称为格集在主格 $P_i Q_j$ 上的层次坐标, 并称 $P_i Q_j$ 在格集的层次坐标上。

由于在 P_0 或 $Q_0 (= @)$ 上的层次坐标 x_0 或 y_0 总是 1, 故可省略, 但与单元格的层次坐标不同, 仅当 k 或 l 大于 0 时, 相应的 P_0 或 $Q_0 (= @)$ 本身可以省略。另外, 符号 $\{\}$ 不能省略, 否则在格集只有一个元素 ($k=n$ 且 $l=m$) 时无法与单元格的层次坐标区分。

用与单元格类似的方法, 可定义相对某基格引用格集时的层次坐标及其中的位移坐标, 与单元格不同, 为避免歧义, 格集的位移坐标省略写法有如下约定:

1. 零层次坐标 (没有 $[\]$ 部分) 仅可用于该格集层次坐标上的主格及其上的层次坐标与基格层次坐标上的主格及其上的层次坐标完全一致的情况;

2. 其他情况下, 除最后的主格 ($P_k Q_l$) 外, 层次坐标上的主格上的位移坐标为 0 时主格及其上位移坐标可以省略, 而 P_k 或 Q_l 本身不能省略, 仅当其是 $@$ 时可省略其上的位移坐标, 否则即使位移坐标为 0 也不能省略。

设 C 是源表上某单元格, x 是个逻辑表达式, 我们用 $C[\dots]\{x\}$ 表示下面的格集:

$\{c | c \in C[\dots]\{\} \text{ 且以 } c \text{ 为基格计算 } x \text{ 的计算结果为真} \}$

x 称为格集 $C[\dots]\{x\}$ 的过滤条件。

带有过滤条件 x 的格集层次坐标所在的表达式一般也有基格, 设为 a , 而在 x 中很可能引用相对 a 的单元格位移坐标, 为了与格集内单元格 (计算 x 时的基格) 区分, 我们约定在 x 中冠

以\$符的单元格将认为是相对 a 的。

同源格集有时会作为数组处理,此时如无特别说明,则认为其元素次序按其内单元格的原序排列。

与单元格类似,格集在表达式中常常也被缺省地处理为其元素格的格值的集合。在格集上同样可应用计算数,即用 $cs(x)$ 表示以格集 cs 中每个元素为基格计算表达式 x 得到的计算结果构成的集合, $cs(x)$ 是一个数集而不再是格集,称为格集 cs 的计算数集。

由于格集及其计算数集是个集合,故可以应用聚合函数,如写 $\text{sum}(C[\dots]\{\dots\})$ 或 $\text{sum}(C[\dots]\{\dots\}(\dots))$ 等。

4.2 数据源

4.2.1 数据集

数据集是一个由数据数组构成的矩阵,其中同列元素的数据类型相同。我们习惯于称构成其行的成员数据为数据集的数据。考虑到关系数据库的习惯,有时也分别称数据集的行和列为记录和字段。

为引用方便,我们会给数据集的某些列起一个可唯一标志的名字,称为列名或字段名,在叙述时常将列与其列名等同处理。由于同列数据的类型一致,该信息可作为列的属性处理,称为列数据类型。

数据集 ds 的第 i 行记作 $ds.R(i)$, 第 j 列记作 $ds.\#j$, 列名为 col 的列记作 $ds.col$ 。显然对所有使 $ds.R(i), ds.\#j, ds.col$ 存在的 i, j, col , 有

$$|ds.R(i) \cap ds.\#j| = |ds.R(i) \cap ds.col| = 1$$

因此可以合理地规定

$$ds.\#j[i] := ds.R(i) \cap ds.\#j_l, \quad ds.col[i] := ds.R(i) \cap ds.col_l$$

数据集的行集定义为一些由数据集的行按某种顺序构成的数组，数据集本身也是一个行集。从组成元素上看，行集是数据集的子集，但排列次序未必与数据集的行序相同。

数据集相关的表达式计算一般必须依赖于某个行集才能进行。我们约定若未某数据集表达式指明其依赖的行集，且在上下文中不能明显看出时，则认为数据集本身是其计算时用的行集。

对于数据集 ds 及其上某指定行集 $R=(r_1, \dots, r_k)$ ，设 j 为某列列号， col 为其列名。约定如下语法：

1. $ds[R].\#j$ 表示数组 $(a_1, \dots, a_k)$ ，其中 $a_i = r_i \cap ds.\#j_l$ ，特别当 $k=1$ 时， $ds[R].\#j = a_1$ ；

若 R 已被某些规则或上下文明显确定，则 $[R]$ 部分可省略不写，以下类似；

2. $ds[R].col$ 与 $ds[R].\#j$ 意义相同；

3. $ds[R].\#0$ 表示数组 $(q_1, \dots, q_k)$ ，其中 q_i 是 r_i 的行号，特别当 $k=1$ 时， $ds[R].\#0 = q_1$ ；

4. 对任何聚合函数 f ， x 是个表达式， $ds[R].f(x)$ 定义为函数返回值 $f(v_1, \dots, v_k)$ ，其中 v_i 是以 r_i 为行集计算表达式 x 的计算结果；在 x 中再次引用 ds 的数据时， ds 可省略不写，如 $ds.sum(\#1+\#2)$ ；

5. $ds[R].f(x_1, \dots, x_n)$ 定义为 $n+2$ 元函数返回值 $f(ds, R, x_1, \dots, x_n)$ ($n \geq 0$)；若 x_i 是表达式且又引用了 ds 中数据时， ds 可省略不写。

由 4、5 两条规则定义的形如 $ds[R].f(\dots)$ 的函数称为数据集 ds 上的函数，简称数据集函数。

按上述规则，显然有 $ds.\#j = ds[ds].\#j$ ， $ds.col = ds[ds].col$ ，这与

前述的将未定义行集按数据集本身处理的缺省规则一致。

对数据集 ds 及其上行集 $R=(r_1, \dots, r_k)$, 定义常用数据集函数:

1. $ds[R].I(x)$, x 是个表达式, 返回值是以 r_1 为行集计算表达式 x 的计算结果;
2. $ds[R].filter(x)$, x 是个计算结果为真假值的逻辑表达式, 返回值是按原次序排列 (在 R 中)、使 $ds[r_i].I(x)$ 为真的 r_i 构成的行集;
3. $ds[R].sort(x)$, x 是个计算结果可排序的表达式, 返回值是将 r_i 按 $ds[(r_i)].I(x)$ 的值排序后形成的新行集。

若数据集 ds 上的函数 f 是集合函数, 对 ds 的任何行集 R , $ds[R].f(\dots)$ 返回值中每个元素均可以某种已定义的规则对应于 R 的一个子集 (次序可以不同), 则称 f 为数据集 ds 的划分函数; 若对任何 $R \subseteq ds$, $ds[R].f(\dots)$ 返回值元素对应的行集两两相离且并集为 R , 则称 f 是完全划分函数。

对数据集 ds 及其某行集 $R=(r_1, \dots, r_k)$, 设 x 是表达式, $v_i=ds[r_i].I(x)$, 定义两个重要的划分函数:

1. $ds[R].select(x):=(v_1, \dots, v_k)$, v_i 对应行集 (r_i) ;
2. $ds[R].group(x):=(u_1, \dots, u_m)$; 其中 $u_i=v_{q(i)}$, q 是归纳定义在 N 上的函数:

$$q(1)=1$$

$$q(i+1)=\min(\{j|j>q(i) \text{ 且 } v_j \neq v_{q(i)}\} \cup \{k+1\})$$

$$m=\min(\{j|q(j)=k+1\})-1$$

$$u_i \text{ 对应行集 } (r_{q(i)}, r_{q(i)+1}, \dots, r_{q(i+1)-1}).$$

事实上, $group$ 函数的作用是对 $\{v_i\}$ 分组, 将连续相同的值合并起来处理。

为讨论方便, 再定义几个常用的导出划分函数:

3. $ds[R].select(x,fx,sx)$
 $:=ds[ds[ds[R].filter(fx)].sort(sx)].select(x)$
4. $ds[R].group(x,fx,sx)$
 $:=ds[ds[ds[R].filter(fx)].sort(sx)].group(x)$
5. $ds.rselect(x,fx,sx):=ds[ds].select(x,fx,sx)$
6. $ds.rgroup(x,fx,sx):=ds[ds].group(x,fx,sx)$

在 1、2 中定义的 select 和 group 是完全划分函数, 而 3、4 中的则不是, 另外注意 rselect 和 rgroup 函数没有 $[R]$ 部分, 不能选择行集。

4.2.2 缺省行集

定义两个与数据源相关的单元格属性, 定义集均为扩展格。

●数据源过滤条件 dsf

属性值是形如 (ds,x) 的序偶构成的集合, 其中 ds 是某数据集, x 是个逻辑表达式, 集合可以是 ϕ , 但不得有两个元素的左元相同。

●数据源关联行集 dsr

属性值是形如 (ds,R) 的序偶构成的数组, 其中 ds 是某数据集, R 是 ds 的行集。报表扩展前所有扩展格的该属性值设置为 ϕ 。

我们修改主格扩展算法 (算法 3.2), 为其增加两步:

6. 若 $P.val$ 是某数据集 ds 的划分函数, 设计算结果数组中的 v_i 对应的 ds 行集为 R_i , 做 $P_i.dsr+=(ds,R_i)$;

7. 对所有 i 和任何 $(ds,x) \in P_i.dsf$, 做 $P_i.dsr+=(ds,ds.filter(x))$ 。

易知, 加入了这两步后, 扩展交换定理、停机定理和封闭定理等仍然成立。

我们约定,在下文中提到的报表扩展均是采用增加了步骤的主格扩展算法。特别地,谈论的中间表意味采用修改后主格扩展算法的导出表。

根据算法过程易知,对中间表上任何扩展格 p ,若 p 未扩展,则有 $p.dsr=\phi$;若 p 已扩展,则 $p.dsr$ 在后续的扩展中不再变化(属性值与继承格相同)。

对中间表 T 上的主格 p 和数据集 ds ,定义格集关联函数 $pdr(p,ds)$ 。函数返回值是 ds 的行集,计算方案递归定义如下:

1. $p=@$ 时, $pdr(p,ds)=ds$; 否则:
2. 若 p 是已扩展格且在 $p.dsr$ 中存在左元为 ds 的成员,设其中序号最小者为 (ds,R) , 则 $pdr(p,ds)=R$; 否则:
3. $pdr(p,ds)=pdr(p.pu,ds)$;

从 pdr 函数的计算规则可知,若源格 P 的格表达式已经是 ds 的划分函数,则不必再有任何 $(ds,x)\in P.dsf$ (即使有也不会对 pdr 计算产生影响)。

有了 pdr 函数,我们建立数据集运算中的缺省行集规则:

规则 4.1 缺省行集规则。

以中间表 T 上某单元格 c 为基格计算表达式 x 时,若 x 中含有引用某数据集 ds 的部分且未在 x 中写明采用的行集(如上述修改算法中的 $filter$),则约定计算时采用的行集如下:

1. 若 c 是扩展格,采用 $pdr(c,ds)$ 。

需要注意的是,在主格扩展算法中计算格表达式时, c 还是未扩展格,此时 $c.dsr=\phi$,因而

$$pdr(c.pu,ds)=pdr(c.pu,ds)$$

即实际采用的行集是 $\text{pdr}(c.\text{pu}, ds)$, 而扩展 c 之后, $c.\text{dsr}$ 可能不再是 ϕ , $\text{pdr}(c, ds)$ 也可能与 $\text{pdr}(c.\text{pu}, ds)$ 不同。

2. 若 c 是非主格, 采用 $\text{pdr}(c.\text{xp}, ds) \cap \text{pdr}(c.\text{yp}, ds)$, 并沿用成员在 $\text{pdr}(c.\text{xp}, ds)$ 中的次序。

缺省行集非常重要, 它使数据集引用书写简单并且智能 (根据表达式相关的单元格位置而自动正确计算), 特别是交叉报表中纵横主格的交辖区内的单元格相关表达式; 同时可复用的数据集分组能提高计算效率; 配合以事先定义的数据集关联规则还将使应用复杂语义层成为可能。

报表 T 上所有表达式中涉及的数据集构成的集合称为 T 的数据源, 记作 $\text{DS}[T]$ 。据数据源可将报表再分类如下:

1. $|\text{DS}|=0$ 时称 T 为无源报表;

2. $|\text{DS}|=1$ 时称 T 为单源报表;

3. $|\text{DS}|>1$ 时称 T 为多源报表, 若 T 上有某个表达式中同时涉及了两个或以上的数据集 (包括通过单元格引用的间接情况), 则称 T 为多源关联报表。

4.3 附加引用

4.3.1 线性坐标

对中间表上某单元格 c , 设 $C=\text{src}(c)$,

1. 若 c 是非主格, 设

$$P=\text{src}(c.\text{xp}), Q=\text{src}(c.\text{yp}), x=\text{seq}(c.\text{xp}), y=\text{seq}(c.\text{yp})$$

则 c 可记为 $C[x;y]$, $Q=@$ 简写为 $C[x]$, $P=@$ 简写为 $C[;y]$, $P=Q=@$ 则直接简写成 C , 这与层次坐标在类似情况的简写形式一致;

2. 若 c 是扩展格, 设 $x=\text{seq}(c)$, 则 c 可记为 $C[x]$ 。

上述写法称为单元格 c 的线性坐标, x, y 分别称为 c 在主格 P (或 C), Q 上的线性坐标, 其值显然不会为 0; 同时称 P (或 C), Q 在 c 的线性坐标上。

显然, 中间表上任何单元格有一个确定的线性坐标。

与层次坐标类似, 我们也要证明线性坐标的唯一性。

定理 4.3 中间表上不同单元格的线性坐标不同。

【设 u, v 是中间表上两个不同的单元格, 显然 u, v 不同源时其线性坐标不同。

设 u, v 同源, 由定理 3.11 可知 u, v 的(左、上)主格线长度相同。

当 u, v 是扩展格时, $u \neq v \Rightarrow \text{srq}(u) \neq \text{seq}(v)$, 即 u, v 的线性坐标不相同。

当 u, v 是非主格时, 若 $u.xp \neq v.xp$ 或 $u.yp \neq v.yp$, 则同上理由仍有 u, v 的线性坐标不同。而定理 4.1 中证明了不可能有 $u.xp = v.xp$ 且 $u.yp = v.yp$ 的现象。】

对中间表上单元格 a, b , 设主格 P 在 b 的线性坐标上, 且 b 在 P 上的线性坐标为 x_b , 若 a 的主格线内存在以 P 为源格的单元格 p , 设 $x_a = \text{seq}(p)$, 令 $x = |x_a - x_b|$, 则在以 a 为基格的表达式中书写 b 的线性坐标时, b 在 P 上的线性坐标 x_b 可写成如下形式:

1. $x_a \leq x_b$ 时写作 $+x$;
2. $x_a \geq x_b$ 时写作 $-x$;
3. $x_a = x_b$ 时写作 0。

此时的线性坐标是单元格 b 相对基格 a 的线性坐标, 其中 $\pm x, 0$

称为单元格 b 在 P 上相对基格 a 的相对坐标, 当上下文中已确定或不关心基格时, 也简单地称为 b 在 P 上的相对坐标。

若单元格 c 在某主格 P 上的相对坐标是 0, 则在其线性坐标上 P 的相对坐标可省略不写, 若 c 在其线性坐标上所有主格 (最多两个) 上的相对坐标都是 0, 则其中的 $[]$ 部分可全部省略而简化成只剩源格名, 称为零线性坐标。

定理 4.4 对源表上非主格 C 及其左、上主格 P, Q , 若中间表上相对于基格的层次坐标 $C[P:\pm x; Q:\pm y]$ 存在, 则必与 $C[\pm x; \pm y]$ 相同; 对源表上扩展格 C , 若中间表相对于某基格的层次坐标 $C[C:\pm x]$ 存在, 则必与 $C[\pm x]$ 相同。

【记基格为 a 。

对于非主格 C , 设 $c=C[P:\pm x; Q:\pm y]$ 在中间表上存在。

$P=@$ 时, 必有 $x=0$, 且 c 在 P 上的相对坐标也总是 $0=\pm x$ 。

$P\neq @$ 时, a 的主格线内必有以 P 为源格的纵主格 p , 记 $u=c.xp$, 显然 u 与 p 同源。由位移坐标定义知 a 在 $\text{src}(P.xp)$ 的主格线内任何主格上的层次坐标与 c 相同, 从而 $p.xp$ 与 $u.xp$ 的层次坐标相同, 则 $p.xp=u.xp$, 由定理 3.12 可知 p 和 u 同束, 再由定理 3.12 和位移坐标定义有

$$\begin{aligned}\text{ord}(u)-\text{seq}(u) &= \text{ord}(p)-\text{seq}(p) \\ &\Rightarrow \text{seq}(u)-\text{seq}(p) = \text{ord}(u)-\text{ord}(p) = \pm x\end{aligned}$$

即 c 在 P 上的相对坐标也是 $\pm x$ 。

同理, c 在 Q 上的相对坐标为 $\pm y$, 从而有

$$C[P:\pm x; Q:\pm y] = C[\pm x; \pm y]。$$

对于扩展格 C , 设 $c=C[C:\pm x]$ 在中间表上存在。则 a 的主格线内必有以 C 为源格的扩展格 p , 即 c 与 p 同源。由位移坐标定

义知 a 在 $\text{src}(C.xp)$ 的主格线内任何主格上的层次坐标与 c 相同, 从而 $p.xp$ 与 $c.xp$ 的层次坐标相同, 则 $p.xp = u.xp$, 由定理 3.12 可知 p 和 c 同束, 再由定理 3.12 和位移坐标定义有

$$\begin{aligned}\text{ord}(c) - \text{seq}(c) &= \text{ord}(p) - \text{seq}(p) \\ \Rightarrow \text{seq}(c) - \text{seq}(p) &= \text{ord}(c) - \text{ord}(p) = \pm x\end{aligned}$$

即 c 在 C 上的相对坐标也是 $\pm x$, 从而 $C[C:\pm x] = C[\pm x]$ 。】

只有一层主格的简单层次坐标 $C[P:\pm x; Q:\pm y]$ 和 $C[C:\pm x]$ 在表达式中较为常用, 定理 4.4 允许我们有时 (保证层次坐标存在) 可以将其写成等价的线性坐标形式 $C[\pm x; \pm y]$ 和 $C[\pm x]$ 。

定理 4.4 还表明零线性坐标和零层次坐标代表的单元格相同 ($x=y=0$ 时), 因此不会产生歧义。由于层次坐标更常用, 我们把零线性坐标都作为零层次坐标处理, 零线性坐标的概念仅存在于理论讨论中。

与线性坐标相关的格集采用下面的书写形式:

1. 设 C 是源表上某非主格, 记

$$C[x_1, x_2; y_1, y_2]\{\} := \{C[x, y] | x_1 \leq x \leq x_2, y_1 \leq y \leq y_2\}$$

2. 设 C 是源表上某扩展格, 记

$$C[x_1, x_2]\{\} := \{C[x] | x_1 \leq x \leq x_2\}$$

上述写法称为同源格集的线性坐标, 而其中的 x_1, x_2, y_1, y_2 称为格集在相应主格上的线性坐标。 $\{\}$ 不可省略保持格集写法的一致性。

用与单元格类似的方法, 可定义相对某基格引用格集时的层次坐标及其中的相对坐标, 相对坐标为 0 时可省略, 但不能全部省略成零线性坐标 (没有 $[\]$ 部分) 的情况 (此时格集中只有一个元素), 至少要写上一个坐标值, 否则无法与零层次坐标的格集

区分，可能产生歧义。

线性坐标式的格集不很常用，在算法实现时有时会被放弃。

与层次坐标类似，格集的线性坐标中也可以加入过滤条件，即用 $C[...][x]$ 表示格集：

$\{c | c \in C[...]\}$ 且以 c 为基格计算 x 的计算结果为真

同样规定，在 x 中冠以 \$ 符的单元格将认为是相对于整个表达式基格的。

作为单元格（集），其线性坐标上也可附加计算数（集）。

4.3.2 平面函数

在报表表达式中还可能采用方位引用单元格，我们回顾并新定义一些相关函数（其中 o 表示基格， a, b, c 是单元格）：

1. $rows() :=$ 报表的总行数

$cols() :=$ 报表的总列数

2. $row(c) := c$ 的行号

$col(c) := c$ 的列号

$row() := row(o)$

$col() := col(o)$

3. $cel(c, y, x) := U(row(c) + y, row(c) + x)$

特别地，规定 $cel(@, y, x) = U(y, x)$

$cel(y, x) := cel(o, y, x)$

$cel(C) := cel(row(C) - row(src(o)), col(C) - col(src(o)))$

注意这里 C 是源格， $cel(C)$ 有时也写作 $\sim C$

4. $range(a, b)$

$:= \{u | row(u) \in [row(a), row(b)], col(u) \in [col(a), col(b)]\}$

$\text{range}(a,b)$ 有时也写作 $a:b$, 其返回值是个格集 (不一定同源), 可应用聚合函数, 如 $\text{sum}(a:b)$;

$\text{range}(c,y_1,x_1,y_2,x_2):=\text{range}(\text{cel}(c,y_1,x_1),\text{cel}(c,y_2,x_2))$

$\text{range}(y_1,x_2,y_2,x_2):=\text{range}(o,y_1,x_1,y_2,x_2)$

以上函数通称为平面函数。

5 计 算

5.1 计算前提

5.1.1 报表计算

原则上，单元格的属性表达式（包括格表达式）中可能引用表格上任何单元格（包括其自己）的属性值，但实际应用中一般都仅引用单元格的格值，过于随意引用会导致计算过程太复杂，而且几乎所有有业务意义报表都可以将随意引用替换成只引用单元格格值的情况。因此我们作一个不算过分的约定：

规则 5.1 单元格的属性表达式中只能引用单元格的格值。

这样，我们可以先仅考虑单元格的格值计算。

格表达式中可能引用其他单元格格值，显然只有表达式中引用的格值均已计算后，该表达式才能计算，即单元格之间存在计算依赖关系，表达式书写不当时还会导致循环引用而使计算无法进行。

定义名为计算状态的单元格逻辑属性，定义集为所有单元格，属性值为真者称为已计算的或已计算格，反之称为未计算的或未计算格。报表扩展变换前，所有单元格的计算状态均设置为假。但为叙述方便，特别规定@是已计算格。

如果所有主格已计算的未计算格的格表达式中引用的单元格均已计算或不存在，则称为可计算的或可计算格，否则称为不可计算的或不可计算格。注意，可计算格引用的单元格可以不存在，此时将把引用格的格值作为空值处理。另外，可计算格和不

可计算格都是未计算格。

再次调整主格扩展算法，将其中的第 2 步改为：

2. 将 P 的扩展状态和计算状态设置为真；

修改主格扩展算法后，再考查单元格计算状态的变化，显然有：

1. 未扩展格是未计算格，未计算扩展格是未扩展格；
2. 已扩展格是已计算格，已计算扩展格是已扩展格；
3. 可计算未扩展格的先主格是已计算格，即已扩展格；
4. 可计算非主格的主格线上的扩展格是已计算格，即已扩展格；
5. 已计算格的主格线上的扩展格是已计算格，即已扩展格。

将针对报表上可计算格 c 的如下动作称为计算 c ：如果 c 是扩展格，则扩展 c （执行调整过的主格扩展算法），否则计算 $c.val$ ，设结果为 v ，做 $c.val=v$ ，并将 c 的计算状态设置为真。

显然计算某可计算格不会对报表布局（单元格的_{数量、位置、主子关系}）产生有别于主格扩展算法的变动，这样如果将该动作视为对报表的一种变换，则此变换对报表、实报表、（横）正实报表、整报表都封闭和反封闭。

仿照报表扩展算法（算法 3.3）设计如下算法，使其能够将格值计算出来：

算法 5.1 报表计算算法

1. 在报表中寻找一个可计算格 c ，如果找不到则算法结束；
2. 计算找到的可计算格 c ；
3. 跳转至 1 重复执行。

报表扩展算法可以被理解为所有非主格的格表达式无须计算（如都是常数）时报表计算算法的特例。

与报表扩展算法类似，我们有

定理 5.1（计算停机定理）对任何报表执行报表计算算法必将在有限步内结束。

【与扩展停机定理的证明类似。设报表深度为 n ，其上深度为 i 的未计算格和可计算格的个数分别为 u_i ，显然 $u_i \geq 0$ 。报表计算算法过程中每计算一个深度为 k 的未计算格后，数组 $u=(u_1, \dots, u_n)$ 将产生如下变化：

$i < k$ 的 u_i 不变；

$i = k$ 的 u_i （即 u_k ）减小 1；

$i > k$ 的 u_i 可能变大；

扩展停机定理已证明，不可能对数组 u 做无限次上述变化而仍保持其每个成员均不小于 0，即经过有限次后报表计算必然停止。】

计算停机定理保证了报表计算算法的合理性，但与扩展算法不同，它不能保证结束时报表上没有未计算格，而只是没有可计算格。

显然报表计算算法对实报表、（满）正实报表和整报表封闭和反封闭。

对报表 T 执行报表计算算法的过程称为计算 T 。由于计算一定会停止，因而我们仍可像前面扩展报表时一样定义源表、中间表、实例表等相关概念。需要指出的是，中间表将定义为算法每次执行到步骤 1 时的报表，即某次主格扩展后或某次非主格计算

后（将对某个单元格 val 属性的修改也看成是一次变换）的导出表，实例表仍定义为最后一个中间表，同时定义中间表的计算序列为计算出指定中间表的过程中产生的中间表依次构成的数组（与扩展序列对应）。

考查前述内容可知，相关概念（源格、同源、同束等）及定理均仍成立。

将单元格的扩展状态、计算状态、值 val、数据源过滤条件 dsf、数据源关联行集 dsr 通称为单元格的计算属性。

设 T 和 T' 是同一源表的中间表，若对任何层次坐标 c 均有：

c 在 T 上存在 $\Leftrightarrow c$ 在 T' 上存在

且存在时 c （代表的单元格）在 T 和 T' 上的各项计算属性对应相同，则称 T 和 T' 计算同构，并称 T 和 T' 上层次坐标相同的单元格互为同位。

显然中间表的计算同构关系具有传递性，即若中间表 T, T', T'' 使 T 与 T' 计算同构且 T' 与 T'' 计算同构，则 T 与 T'' 也计算同构。

两个计算同构的报表上单元格数量显然相同。由于层次坐标中已隐含了单元格的扩展方向及主子关系，因此计算同构报表上（纵、横）扩展格及非主格数量也对应相同，同位单元格的（左、上）主格、子（属）格均对应同位且数量相同。

基于上述认识，我们经常将计算同构的报表在计算过程中理解为相同的。但需要指出的是，计算同构的报表并不完全相同，其对应单元格的方位（行号、列号、行数、列数）就可能不一样（这些信息一般不影响计算过程）。

定理 5.2 （计算交换定理）对于中间表上两个可计算格 a, b ：

1. 计算 a 后, b 仍是可计算格, 反之亦然;
2. 先 a 后 b 和先 b 后 a 执行计算得到的结果计算同构。

【采用类似扩展交换定理的证明方法, 记中间表为 T 。

若 a 是扩展格, a 可计算意味着 a 未计算即 a 未扩展, 而 b 可计算, 则必有 $b \notin \text{DOM}_B[a]$, 则计算 a 后 b 没有导出格而只有继承格; 反之亦然。

显然计算 a 后不会使任何已计算格的继承格变成未计算格, 则 b (继承格) 仍将是可计算格; 反之亦然, 故命题 1 成立。

将在 T 上计算 a 后的导出表记作 T_a , 再计算 b 后的导出表记作 T_{ab} ; 在 T 上计算 b 后的导出表记作 T_b , 再计算 a 后的导出表记作 T_{ba} 。

将 T 上的单元格 c 在 T_a, T_b, T_{ab}, T_{ba} 上的继承格分别记作 c_a, c_b, c_{ab}, c_{ba} ; c 在 T_a, T_b 上的第 i 个导出格 (如果有的话, 以下同) 分别记作 c_{ai} 和 c_{bi} ; c_{ai} 和 c_{bi} 在 T_{ab} 和 T_{ba} 上的继承格分别记作 c_{aib} 和 c_{bia} ; c_a 和 c_{ai} 在 T_{ab} 上的第 j 个导出格分别记作 c_{abj} 和 c_{aibj} ; c_b 和 c_{bi} 在 T_{ba} 上的第 j 个导出格分别记作 c_{baj} 和 c_{biaj} 。

我们这里用符号 $u == v$ 表示单元格 u 与 v 的层次坐标和计算属性相同, 注意 u, v 并不在同一个报表上。

若 a, b 都是非主格, 计算 a, b 时不会影响报表布局和其他单元格的计算属性, 此时显然有 $c_{ab} == c_{ba}$ 。

若 a, b 中有且仅有一个是扩展格, 不妨设为 a 。

当 $c \notin \text{DOM}_B[a]$ 时, c 主格线上任何主格均不属于 $\text{DOM}_B[a]$, 那么扩展 a 不会改变这些单元格的层次坐标和计算属性, 即有 $c_a == c$; 同样分析 c_b 和 a_b 的关系可知 $c_b == c_{ba}$, 而显然有 $c_{ab} == c_a$ 和 $c_b == c$, 从而 $c_{ab} == c_{ba}$ 。

当 $c \in \text{DOM}_B[a]$ 时, c 在扩展 a 后将有导出格, 分析主格扩展

算法可知, c_{ai} 在 $\text{src}(a)$ 上的层次坐标为 i , 而在其他主格上的层次坐标与 c 保持相同, 仅当 $c=a$ 时, $c_{ai}.\text{val}$ 会变成 $a.\text{val}$ 的第 i 个成员, 而其他情况的计算属性及此情况的其他计算属性均与 c 保持一致; 同样的分析可适用于 c_b 和 a_b , 而显然有 $c_b==c$ 及 $a_b==a$, 综合可知 $c_{bai}==c_{ai}==c_{aib}$ 。

若 a, b 都是扩展格。

$c \notin \text{DOM}_B[a]$ 且 $c \notin \text{DOM}_B[b]$ 时, 类似分析可知有 $c_{ab}==c_{ba}$;

$c \in \text{DOM}_B[a]$ 且 $c \notin \text{DOM}_B[b]$ 时, 类似分析可知有 $c_{aib}==c_{bui}$;

$c \notin \text{DOM}_B[a]$ 且 $c \in \text{DOM}_B[b]$ 时, 类似分析可知有 $c_{abi}==c_{bia}$ 。

当 $c \in \text{DOM}_B[a]$ 且 $c \in \text{DOM}_B[b]$ 时, 由于 a, b 均是可计算格 (未计算), 则 a, b 不在同一主格线内, 而扩展方向相同的主格的后辖区有交集意味着它们必在同一主格线内, 故此时 a, b 的扩展方向不同, 即 a, b 分别落在 c 的左、上主格线内。则必有 $c_{ai} \in \text{DOM}_B[b]$ 和 $c_{bi} \in \text{DOM}_B[a]$ 。

这样, 同上类似分析可知, 扩展 a 后 c_{ai} 在 $\text{src}(a)$ 上的层次坐标为 i , 其他主格上的层次坐标与 c 相同, 计算属性也与 c 相同 ($c \neq a$), 再扩展 b_a 后 c_{aibj} 在 $\text{src}(b)$ 上的层次坐标为 j , 其他主格上的层次坐标与 c_{ai} 相同, 计算属性也与 c_{ai} 相同 ($c \neq b$), 即 c_{aibj} 在 $\text{src}(a)$ 和 $\text{src}(b)$ 上的层次坐标分别为 i, j , 而在其他主格上的层次坐标与 c 相同, 计算属性也与 c 相同; 同样分析适用于 c_{bjai} , 故有 $c_{aibj}==c_{bjai}$ 。

T_{ab} (T_{ba}) 上单元格显然只可能是上述几种继承格或导出格之一, 故 T_{ab} 与 T_{ba} 计算同构, 即命题 2 成立。】

显然, 对两个计算同构的报表上同位的可计算格执行计算后得到的导出表仍然计算同构。由此可得

推论 1 设 a, b 是报表 T 上的可计算单元格, a', b' 是某个与 T 计算同构的报表 T' 上与 a, b 同位的单元格, 则对 T 先 a 后 b 计算和对 T' 上先 b' 后 a' 计算后得到的导出表也计算同构。

继而还可得:

推论 2 计算某个确定的源表, 无论在步骤 1 中如何选择可计算格, 得到的实例表均计算同构。

这一重要结论在后文中讨论报表可计算性及相关算法的复杂度时将有应用。

结合扩展交换定理, 还将有:

推论 3 交换实报表上两个可计算格的计算次序, 得到的结果报表相同。

即计算实报表时其结果不依赖于步骤 1 中对可计算格的选择。但计算非实报表的实例表却与步骤 1 对可计算格的选择相关。在需要保证一致性时必须指明选择方案, 未指明时一般会缺省选择前边的。

5.1.2 隐含引用

判断单元格是否可计算时需要检查其格表达式中引用的单元格, 一般情况下, 我们通过考查表达式中是否用到某元素以判断其是否被表达式引用, 这种办法在报表不再扩展时没有问题。但当格表达式中采用复杂方案(层次坐标、线性坐标、平面函数)引用单元格时, 对于仍在继续扩展的中间表, 表达式引用的单元格可能还未扩展出来(未计算), 这时难以与该格不存在的情况区分(不存在时将作为已计算处理); 或者, 引用的单元格及某些函数返回值可能会随着进一步的扩展而变化, 从而导致计算结果的不一致。

为避免计算时的二义性和结果的不一致,除了常规的引用认定方案外,我们还需要增加约定一些隐含的引用规则。

平面函数的返回值可能随着进一步扩展而变化。如对于某基格,若中间表上存在行号小于 $\text{row}()$ 的未扩展纵主格,那么对于该格在实例表上的继承格, $\text{row}()$ 返回值将不同,只有在这些未扩展纵主格扩展之后才能获得正确的 $\text{row}()$ 。几乎所有平面函数都有类似的问题,逐一分析每个函数的具体情况而设计不同规则很繁琐且实际意义不大,因而我们简单地约定:

规则 5.2 含有平面函数的表达式引用所有未扩展格。

规则 5.2 保证表达式中引用的平面函数均有确定的返回值。

已计算格的线性坐标也可能随着进一步扩展而变化。如对于某扩展格 $C[x]$,若有某个 $C[i]$ ($i < x$) 未扩展,则 $C[x]$ 在实例表上的继承格的同源号可能不再是 x ,也就不能写成 $C[x]$,而实例表上的 $C[x]$ 将可能是另一个单元格。对采用相对坐标的线性坐标也有类似问题,因而我们约定:

规则 5.3 对以 C 为源格的单元格或格集的线性坐标,设 P 是在该线性坐标上的主格, C 在 P 上的线性(相对)坐标为 x ($\pm x$ 或 0),则该线性坐标隐含引用 $P[i]$ ($P[\pm i]$ 或 $P[0]$), $1 \leq i \leq x$ 。

特别地,当线性坐标为 ± 1 和 0 (最常见的情况)时,规则 5.3 并未为其增加更多的引用单元格。

结合定理 3.11 和扩展过程中单元格变迁性质可知,规则 5.3 将保证表达式中引用的线性坐标在后续扩展中不再变化,从而将唯一代表某确定的单元格。

与平面函数和线性坐标不同, 层次坐标有更好的性质。

定理 5.3 报表计算过程中, 已计算格和可计算格的层次坐标保持不变 (与其继承格相同)。

【设有已计算格或可计算格 a 。

a 是非主格时, a 已计算或可计算, 则 a 的主格线上任何扩展格 p 都已扩展格, 故 p 不会再扩展从而后续计算中 p 不会有导出格且 $\text{ord}(p)$ 不再改变。因此, a 的层次坐标也将保持不变。

a 是扩展格时, 同理 a 的先主格 p 都已扩展且 $\text{ord}(p)$ 不再改变; 若 a 已扩展, 则 $\text{ord}(a)$ 也不再改变; 若 a 未扩展, 则 $\text{ord}(a)=1$, 而其继承格 (无论是否由于扩展 a 导致) 的同束号也是 1。从而 a 的层次坐标仍保持不变。】

层次坐标这一良好性质是由于主格必须先于子格和属格计算导致的。

根据定理 5.3 可知, 已计算格相对于可计算格的层次坐标在报表计算过程中也将保持不变, 这确保了在未计算格的表达式中使用位移坐标的可行性。

定理 5.3 表明在报表计算过程中 (中间表上) 使用层次坐标用于计算相对安全, 但仍可能发生引用尚未扩展出来的单元格的现象, 为避免判断是否可计算时的二义性, 我们约定:

规则 5.4 扩展格层次坐标 (C 是扩展格)

$$C[P_0:x_0, P_1:x_1, \dots, P_n:x_n]$$

隐含引用单元格

$$P_k[P_0:x_0, P_1:x_1, \dots, P_k:1]$$

非主格层次坐标 (C 是非主格)

$$C[P_0:x_0, P_1:x_1, \dots, P_n:x_n; Q_0:y_0, Q_1:y_1, \dots, Q_m:y_m]$$

隐含引用单元格

$$P_k[P_0:x_0, P_1:x_1, \dots, P_k:1]$$

和

$$Q_l[Q_0:y_0, Q_1:y_1, \dots, Q_l:1]$$

其中 $0 \leq k \leq n, 0 \leq l \leq m$, x_i, y_j 可以是位移坐标的形式。

定理 5.4 对于层次坐标 c , 设 cs 是 c 在规则 5.4 下隐含引用的层次坐标构成的集合, 如果 cs 在中间表上存在的成员均已计算, c 在中间表上存在的充要条件是在实例表上存在。

【由定理 5.3 可知, c 在中间表上存在则必在实例表上存在。若 c 在中间表上不存在, 沿用规则中的符号, 记

$$P_k[P_0:x_0, P_1:x_1, \dots, P_k:1] = p_k$$

和

$$Q_l[Q_0:y_0, Q_1:y_1, \dots, Q_l:1] = q_l$$

其中 $0 \leq k \leq n, 0 \leq l \leq m$ 。

考查 p_k 和 q_l 在中间表上的存在性。

显然 $p_0 = @$ 存在且已计算; 若 p_k 存在且已计算, 则 p_k 的同束格也均存在且已计算, 若 p_k 有同束号为 x_k 的同束格, 则 p_{k+1} 作为该同束格的子格也必存在故已计算, 这样如果假设成立则可迭代下去直到 c 存在, 产生矛盾。故必存在某个 k 使 p_k 存在(已计算), 但 p_k 没有同束号是 x_k 的同束格(或对称地发生在 q_l 上), 显然这时 $p_{k+1}, \dots, p_n$ 以及 c 在中间表和实例表上都不存在。】

定理 5.4 表明, 在中间表上表达式中引用到不存在的层次坐标时, 要么该层次坐标隐含引用的单元格未计算(此时该格可能尚未扩展出来, 暂作为未计算处理), 要么该单元格确实不存在(可作为已计算处理), 从而消除了二义性。

除上述单元格表达式导致的隐含引用外，数据集表达式也有隐含引用问题。

数据集表达式的计算依赖于采用的行集，缺省行集由 `pdr` 函数返回值确定，而 `pdr` 函数又依赖于 `dsr` 属性，继而与主格扩展的步骤 6 中的 `filter` 函数相关，只有 `dsf` 属性中的过滤条件表达式（`filter` 的参数）可计算（其引用的单元格都已计算），才能使 `dsr`、`pdr`、缺省行集、数据集表达式可计算。为此，我们约定：

规则 5.5 扩展格的 `dsf` 属性中的过滤条件表达式（`dsf` 属性值成员的右元）中引用的单元格必须包含于其格表达式引用的单元格（包括间接引用的）。

规则 5.5 保证可计算扩展格的 `dsf` 属性中的过滤条件表达式也可计算（即主格扩展算法的步骤 6 可执行）。这相当于数据集相关的表达式隐含地引用了 `dsf` 属性，看起来与规则 5.1 有所冲突，但并不会影响到相关结论的正确性。

作为规则 5.5 的替代方案，我们也可以约定在判断未扩展格是否可计算时，将 `dsf` 属性中的过滤条件表达式也视为格表达式，即仅当 `dsf` 属性中的过滤条件表达式和格表达式引用的单元格都已计算（或不存在），才认为该未扩展格可计算。

但算法实现时格表达式和过滤条件表达式的计算在不同步骤中进行，而表达式可计算的判断与其计算是同时进行的，事先判定会造成效率降低，且扩展动作又难以逆转。相反，可以采取变通手段在设计报表时将 `dsf` 属性中的过滤条件表达式中引用的单元格都被格表达式引用，因此我们采用了更简单的规则 5.5。

5.2 计算方法

5.2.1 反复遍历

为计算未计算格 c 而必须先计算的未计算格称为 c 的计算依赖格,即 c 的主格或其格表达式中引用的单元格(包括隐含引用),显然未计算格可计算的充要条件是其没有计算依赖格。

若报表 T 上存在 $n(>0)$ 个未计算格 $c_1, \dots, c_n$, 记 $c_0 = c_n$, 使得对所有 $0 \leq i \leq n$ 均有 c_{i+1} 是 c_i 的计算依赖格, 则称 T 有循环引用现象; 由这些未计算格构成的数组 $(c_1, \dots, c_n)$ 称为 T 的循环引用队列, 同一个报表上可能有多个循环引用队列, 两个循环引用队列可能有共同的元素。

定理 5.5 实例表有循环引用现象的充要条件是其上有未计算格。

【必要性显然。

设实例表 T 上有未计算格 c_0 , 由于 T 是实例表, 则 c_0 只能是不可计算格, 则有 c_0 的计算依赖格 c_1 , 而 c_1 未计算, 则也不可计算……以此类推, 必然存在单元格序列 $c_2, c_3, \dots$ 均不可计算(未计算), 其中 c_{i+1} 是 c_i 的计算依赖格, 而 T 上的单元格数量有限, 则必然存在有 $i < j$, 使 $c_i = c_j$, 则数组 $(c_i, \dots, c_j)$ 即是 T 上的一个循环引用队列。】

由定理 5.5 可知, 与任何有循环引用现象的实例表计算同构的实例表上也有循环引用现象(未计算格的同位格必然也未计算), 而计算交换定理推论 2 表明同一源表的实例表均计算同构; 这样, 实例表上是否存在循环引用现象将是源表(中间表)的某种固有性质。我们将实例表有循环引用现象的中间表(源表)称为不可计算的, 反之称为可计算的。不可计算报表(源表)通常

是设计错误导致的，需要在计算过程中识别出来。

另外，在计算报表过程中需要判断出未计算单元格是否可计算，但事实上并没有一种简单迅速的方法可以做到这一点，一般情况下只能试图去计算格表达式，计算失败后才知道其不可计算，而计算成功后则直接继续完成单元格的计算动作。即判断单元格的可计算性及其计算是一体的，整个动作称为试算单元格。

试算单元格是算法过程中复杂度较高的基本动作，其执行次数决定了整个算法的复杂度。其中成功的试算会跟随其他完成单元格计算的动作（如扩展），复杂度要大于失败的试算。由于所有实例表均计算同构，且任何未计算格都只需要也只可能被成功计算一次，因此对于确定的源表，在整个计算过程中成功的试算次数也是确定的，这与算法过程中可计算格的选择无关。那么，报表计算的复杂度差异仅取决于试算的失败次数。

我们在描述算法 5.1 步骤 1 中没有指出如何选择可计算格，在具体实现算法时必须要有个明确的方案。

一个很简单的方法是按原序遍历中间表上所有单元格，找到其中的可计算格计算。由于计算未计算格后可能会产生出新的单元格（扩展后的导出格），并且某被引用的未计算格经过计算变成已计算格后会使已遍历过的不可计算格变成可计算格。这样就需要反复地遍历中间表，直到某次完整遍历过程中未发现任何可计算格为止。

算法 5.2 报表计算的反复遍历法。

1. 记 c 是当前中间表上最前格：

2. 检查 c 是否已计算, 如未计算则试算 c ;
3. 如果 c 有跟随格, 则以其跟随格替代 c 跳转至 2 重复, 否则:
4. 如果最近完成的步骤 1-3 循环中有成功的试算, 则跳转至 1 重复; 否则:
5. 如果此时报表上没有未计算格, 算法正常结束, 否则:
6. 算法异常结束。

其中步骤 4、5 中的判断可通过在步骤 2 中记下相关信息处理, 不会增加算法的复杂度。另外需要注意的是, 由于在计算过程中可能会产生新的单元格, 这样每次执行到步骤 2 时, 中间表的单元格数组成员及其次序都可能会不同。

定理 5.6 对任何报表执行反复遍历法, 有

1. 算法将在有限步内停止;
2. 算法正常结束的充要条件是报表可计算;
3. 算法异常结束的充要条件是报表不可计算。

【考查反复遍历法的过程, 其步骤 1~4 的循环可被视作一种选择可计算格的方案, 由计算停机定理可知, 对任何源表执行该算法必定会在有限步后使得到的中间表上不再有可计算格, 此时步骤 1~3 的循环中不会再有成功的试算, 到步骤 4 时算法将不再重复而继续到步骤 5 并最终得到实例表而停止。即命题 1 成立。

再考察步骤 5、6 可知算法正常结束时报表上没有未计算格, 即报表可计算; 相反可知算法异常结束时有报表不可计算。

如果报表可计算, 则对报表执行算法不可能异常结束, 否则将推出报表不可计算而矛盾, 从而命题 2 成立, 同理命题 3 成立。】

将实例表上单元格个数记为 n ，在最坏情况，反复遍历法的试算的失败次数可能会达到 $n*(n-1)/2$ (每个单元格均引用其跟随格)。沿用可计算性理论的术语，可以说反复遍历法的试算失败复杂度为 $O(n^2)$ 。

5.2.2 单次遍历

关于可计算格在单元格数组中的位置，我们有如下的结论。

定理 5.7 如果可计算格 c 前边的单元格都是已计算格，那么计算 c 后 c (继承格) 前边的单元格仍都是已计算格。

【 c 是非主格时命题显然成立。

设 c 是扩展格，根据定理 3.5 可知，计算 c 后， c 前边的任何继承格的原格必然在 c 前边，因而是已计算格。对任何单元格 $u \in \text{DOM}_B[c]$ ，显然 u 是未计算格，由条件知 u 在 c 后边，则仍根据定理 3.5， u 的任何导出格必然也在 c 后边，即不可能有未计算的导出格在 c 前边。】

在试算单元格时我们不仅能够知道该单元格是否可计算，如果试算失败 (该单元格不可计算)，我们还能获得至少一个该格的计算依赖格，但由于表达式的多样性，仅通过试算我们很难知道该格所有的计算依赖格。

利用这个特点和定理 5.7，我们可以设计一个效率更高的报表计算算法。

算法 5.3 报表计算的单次遍历法。

1. 记报表的最前格为 c ;
2. 检查 c 是否已计算，如已计算跳转到 8，否则:

3. 令数组 $q=(c)$;
4. 如果 q 是空集, 则跳转至 8;
5. 设 u 是 q 中最后一个成员, 试算 u , 若成功 (计算 u 时 q 作为上下文中的集合将会同时依照元素变换规则而变换) 则做 $q+=u$ (u 已是继承格) 并跳转至 4 重复, 否则:
6. 设试算失败后得到 u 的某个计算依赖格 v , 若 $v \in q$, 则算法异常结束; 否则:
7. $q+=v$, 跳转至 5 重复;
8. 如果 c 在报表上有跟随格则以其跟随格取代 c 跳转至 2 重复, 否则:
9. 算法正常结束。

定理 5.8 采用单次遍历法计算报表时, 有

1. 算法将在有限步内结束;
2. 算法正常结束时报表上没有未计算格, 即源表可计算;
3. 算法异常结束时 (步骤 6), 实例表将有循环引用队列, 即源表不可计算。

【考查算法过程并沿用其中的符号。

算法由两重循环构成, 外层循环 (步骤 1~8) 以中间表上单元格数量作为循环次数, 显然是有限值。内层循环 (步骤 3~7) 以数组 q 的长度作为循环次数, 任何未计算格一旦被加入到 q 中, 若在当次循环中再次被加入 q 时, 算法将异常结束; 若没有被二次加入 q , 则当次循环的次数最多是报表上未计算格的数量, 亦为有限值, 而两重有限的循环必然会在有限步内结束, 即算法若不在步骤 6 异常结束, 则必在步骤 9 正常结束。

考查外层循环, 执行到步骤 8 的条件是步骤 2 时 c 已计算或

步骤 4 时 q 为空集, 由于 $c \in q$, 而只有试算成功后才能将 c 从 q 中删除, 即此时 c 已成功试算而成为已计算格, 再由定理 5.6 可知不会有任何未计算格在 c 前边, 这样当 c 不再有跟随格而导致算法正常结束时, 报表上所有单元格均已计算, 即没有未计算格, 从而可知源表可计算。

如果算法在步骤 6 异常结束, 由 q 的形成规则可知其子集 $\text{to}[q](u,v)$ 是当时报表上的一个循环引用队列, 显然其成员 (继承格) 在后续的任何可能的计算 (扩展) 中均不可能被计算, 则在任何实例表上将构成一个循环引用队列, 从而可知源表不可计算。】

采用单次遍历法不仅可以判断出报表是否可计算, 同时在报表不可计算时还能给出一个循环引用队列。但与反复遍历法不同, 异常结束的单次遍历法不能保证当时的中间表上没有可计算格, 事实上报表计算还未结束, 当时的中间表并非实例表, 但由于已知其不可计算, 再计算下去没有意义。算法正常结束时则可以保证计算完成。

定理 5.9 设实例表上单元格数量为 n , 执行单次遍历法最多进行 n 次失败的试算即会结束, 即单次遍历法的试算失败复杂度为 $O(n)$ 。

【延用算法的符号, 考查算法过程。任何一次失败的试算都将使某个未计算格加入到数组 q 中, 同一个未计算格第 2 次加入到数组 q 时会导致算法异常结束, 即只有 1 个或者根本没有未计算格被 2 次加入数组 q 。而我们知道, 报表上最多有 n 个未计算格, 且最前格无须试算将被自动加入到 q 中, 这样最多可

能有 $n-1+1=n$ 次未计算格加入数组 q 的动作, 即试算失败次数不超过 n 。】

定理 5.9 表明单次遍历法比反复遍历法优越。

5.2.3 源格遍历

考查源表上单元格的格表达式 (其上所有单元格都未计算), 若单元格 A 的格表达式中显式地出现了单元格 B (如层次坐标、线性坐标) 或 B 是 A 的主格, 则称 A 符号引用了 B ; 另外特别规定, 如果 $A.val$ 中采用了平面函数或某些不能确定其引用单元格的函数 (如 $eval$ 及其他类似自定义函数), 则强制规定 A 符号引用了源表上其他所有单元格。

单元格间是否符号引用可以简单地通过考查表达式内容和单元格之间的主子关系判断, 无须试算。

如果在源表 S 上存在 $n(>0)$ 个单元格 $C_1, \dots, C_n$, 记 $C_0=C_n$, 使得对所有 $0 \leq i < n$ 均有 C_i 符号引用了 C_{i+1} , 则称 S 有 符号循环引用现象, 数组 $(C_1, \dots, C_n)$ 称为 S 的 符号循环引用队列。

如果由源表 S 上所有单元格排列成的数组 Q_S 满足: Q_S 中任何成员 C 符号引用的单元格在 Q_S 中的序号均小于 C 的序号, 则称 Q_S 是 S 上的一个 可计算队列。一个源表可能有多个可计算队列。

算法 4.4 源表上可计算队列寻找算法。

1. 记源表的最前格为 C , 并令数组 $Q_S = \phi$;
2. 如果 C 符号引用的单元格全部属于 Q_S , 则 $Q_S += C$, 跳转到 8, 否则:

3. 令数组 $Q=(C)$;
4. 若 Q 是空集, 则跳转至 8;
5. 设 U 是 Q 中最后一个元素, 检查 U 符号引用的单元格, 若全部属于 Q_S , 则做 $Q_S+=U$ 和 $Q-=U$, 跳转至 4, 否则:
6. 若有 U 符号引用的某单元格 $V \in Q$, 则算法异常结束; 否则:
7. $Q+=V$, 跳转至 5;
8. 若 C 有跟随格, 则以 C 的跟随格取代 C , 跳转至 2; 否则:
9. 算法正常结束, 此时 Q_S 是源表的一个可计算队列。

注意步骤 2 和步骤 5 中的动作 $Q_S+=x$, 仅当 $x \notin Q_S$ 时才会在 Q_S 最后上加入成员 x , 若动作执行前已有 $x \in Q_S$, 则不作任何加入操作。

定理 5.10 对任何源表执行可计算队列寻找算法, 有

1. 算法必会在有限步内停止;
2. 算法正常结束时, 源表有可计算队列;
3. 算法异常结束时, 源表有符号循环引用现象。

【考查算法过程并沿用其中的符号。

算法由两重循环构成, 外层循环(步骤 1~8)以源表上单元格数量作为循环次数, 显然是有限值。内层循环(步骤 3~7)以数组 Q 的长度作为循环次数, 任何单元格一旦被加入到 Q 中, 若在当次循环中再次被加入 Q 时, 算法将异常结束; 若没有被二次加入 Q , 则当次循环的次数最多是源表上单元格的数目, 亦为有限值, 而两重有限的循环必然会在有限步内结束, 即算法若不在步骤 6 异常结束, 则必在步骤 9 正常结束。

考查外层循环, 执行到步骤 8 的前提是在步骤 2 中将 C 加入

Q_s 或步骤 4 时 Q 为空集, 由于 $C \in Q$, 而只有将 C 加入 Q_s 后才能将其从 Q 中删除 (步骤 5), 即此时必有 $C \in Q_s$, 当 C 不再有跟随格而导致算法正常结束时, 源表上所有单元格均已加入到 Q_s 中, Q_s 是由源表上所有单元格构成的数组。而由 Q_s 的加入规则可知, 任何新加入 Q_s 的单元格符号引用的单元格均已在 Q_s 中, 从而使 Q_s 满足可计算队列的定义。

如果算法在步骤 6 异常结束, 由 Q 的形成规则可知其子集 $\text{to}[Q](U, V)$ 即是源表上的一个符号循环引用队列, 即源表有符号循环引用现象。】

考查算法过程可知, 可计算队列寻找算法与单次遍历法流程完全一样。事实上, 如果将 Q_s 中成员视为“已计算格”, 而为 Q_s 添加成员视为“计算”单元格, 则算法可视为一个简化版 (不在计算过程中产生新单元格) 单次遍历法。

定理 5.11 源表上存在可计算队列的充要条件是其上没有符号循环引用现象。

【设源表有可计算队列 Q_s 和符号循环引用队列 $(C_1, \dots, C_n)$ ($n > 0$), 记 $C_0 = C_n$, 设 C_i 在 Q_s 中的序号为 q_i ($0 \leq i \leq n$), 则由可计算队列和符号循环引用定义可知必有 $q_n = q_0 > q_1 > \dots > q_n$, 产生矛盾。因此可计算队列和符号循环引用队列不可共存, 从而必要性成立。

如果源表上没有符号循环引用现象, 则对其执行寻找可计算队列算法必将正常结束, 因而必然存在可计算队列, 即充分性也成立。】

对于有可计算队列的源表, 我们有更高效的报表计算方法。

算法 4.5 报表计算的源格遍历法。

1. 调用寻找可计算队列算法, 如果失败(出现符号循环引用)则算法异常结束, 否则:
2. 将得到的可计算队列记为 Q_s , 按成员在 Q_s 中的次序依次对其成员 C 执行: 试算当前的中间表上所有以 C 为源格的单元格;
3. 算法正常结束。

下面的引理将保证算法的可行性。

引理 对任何报表执行源格遍历法时, 步骤 2 中的试算一定会成功。

【延用算法中的符号。设 c 是中间表上以 C 为源格的未计算格, 如果此时存在 c 的计算依赖格 u , 记 $U=\text{src}(u)$ 。当 u 是 c 的主格时显然 C 符号引用 U ; 否则, 由 $c.\text{val}=C.\text{val}$ (未计算时) 可知 U 必出现在 $C.\text{val}$ 中, 即仍有 C 符号引用 U 。则 U 在 Q_s 中的序号小于 C 的序号, 从而 u 将先于 c 在步骤 2 中被试算。考虑到归纳法原理即可知, u 不存在, 即 c 可计算, 从而试算必然成功。】

考查源格遍历法过程和上面引理, 很容易得到:

定理 5.12 有可计算队列的源表可计算。

【对有可计算队列的源表执行源格遍历法。定理 5.11 表明该源表上没有符号循环引用现象, 则执行步骤 1 后必将成功找到一个可计算队列, 而由引理可知继续执行算法将成功试算所有未计算格, 从而获得成没未计算格的实例表。】

推论 没有符号循环引用现象的源表可计算。

需要注意的是, 定理 5.12 及其推论的否(逆)命题并不成立, 可计算的源表未必一定有可计算队列, 有符号循环引用现象的源

表也未必不可计算。

一般情况下由于源格的数量很少,源格遍历法实际执行的复杂度很低。由于算法过程中没有失败的试算,可以说其试算失败复杂度为 $O(0)$,因此其总体复杂度比单次遍历法更低,但算法仅对没有符号循环引用的报表有效。

由符号引用的定义易知,没符号循环引用的源表上至多只能有一个单元格的格表达式中会出现平面坐标或类似 eval 的函数。这样我们索性约定仅能对单元格格表达式中没有平面坐标(或 eval 函数等)的源表执行源格遍历法。

事实上,现实中许多规模很大(实例表单元格数量多)的报表反而计算关系简单,常常可以满足上述条件(没有符号循环引用及平面坐标等),很适合采用源格遍历法。一般来讲,可以对报表先用源格遍历法试验(仅对数量很少的源格操作故而代价很低),如果算法异常结束,则再采用单次遍历法重新计算。

5.3 报表填写

5.3.1 数据库

先准备一些与报表填写相关的关系数据库知识,这里仅叙述需要用到的内容,而非完整的关系数据库模型。

数据集 ds 的主码是某个由其列构成的一个数组 $(p_1, \dots, p_k)$, 记作 $ds.PK$, 满足: 对 ds 的所有行 $R(i)$, 数组 $(ds.p_1[i], \dots, ds.p_k[i])$, 称为 $R(i)$ (第 i 行) 的主码, 记作 $ds.PK(i)$ 或 $ds.PK(R(i))$, 因 i 的不同而互不相同。其中 p_i 称为 ds 的主码列。

有主码的数据集称为数据表, 由有限个数据表构成的集合称为数据库。

设数据表 dt 有 m 个字段 $f_1, \dots, f_m$, 记 $p=dt.PK=(p_1, \dots, p_k)$, 我们为 dt 定义如下的函数和算法:

1. 选出函数

对于给定的数组 $v=(v_1, \dots, v_k)$, 其中 v_i 是数据:

如果存在行号 r 使 $dt.PK(r)=v$, 则返回 r ; 否则返回 0;

查找函数记作 $dt.SELECT(v)$ 。

2. 插入算法

对于给定的数组 $c=(c_1, \dots, c_n)$ 和 $u=(u_1, \dots, u_n)$, 其中 c_i 是 ds 的列, u_i 是数据, 满足 $p \subseteq c$ (显然有 $k \leq n$):

令 v 是个 k 元组, 其第 i 个成员是 u_j , 满足 $c_j=p_i$;

若 $dt.SELECT(v)>0$, 则算法失败; 否则:

令 w 是个 m 元组, 若存在 $c_j=f_i$, 则其第 i 个成员为 u_j , 否则为空值 (NULL);

做 $dt+=w$; 此时 dt 的行数加 1, 算法成功; 易知此时 dt 仍是数据表。

插入算法记作 $dt.INSERT(c, u)$ 。

3. 删除算法

对于给定数组 $v=(v_1, \dots, v_k)$, 其中 v_i 是数据:

记 $r=dt.SELECT(v)$, 若 $r=0$, 则算法失败; 否则:

做 $dt-=dt.R(r)$, 此时 dt 的行数减 1, 算法成功; 显然 dt 仍是数据表。

删除算法记作 $dt.DELETE(v)$ 。

4. 更新算法

对于给定数组 $v=(v_1, \dots, v_k)$, $c=(c_1, \dots, c_n)$ 和 $u=(u_1, \dots, u_n)$, 其中 v_i, u_j 是数据, c_i 是 dt 的列:

设 $r=dt.SELECT(v)$, 若 $r=0$ 则算法失败; 否则:

令数组 pw 是个 k 元组, 若存在 $c_j=p_i$, 则其第 i 个成员为 u_j , 否则为 $dt.p_i[r]$;

如果 $dt.SELECT(pw)$ 大于 0 且不等于 r , 则算法失败, 否则:

令数组 w 是个 m 元组, 若存在 $c_j=f_i$, 则其第 i 个元素为 u_j , 否则为 $dt.f_i[r]$;

做 $dt.R(r) \rightarrow w$, 算法成功; 可以证明此时 dt 仍是数据表。

更新算法记作 $dt.UPDATE(v,c,u)$ 。

这 4 个函数和算法是关系数据库的基本操作, 我们在这里仅对其作一个描述, 而不证明其合理性。

5.3.2 平面填写

通俗地讲, 报表填写是指根据源表上设计的规则改变实例表上单元格的格值, 并再将格值按规则保存到数据库中的工作。

我们定义一个结构较为复杂的单元格属性用于处理数据的保存, 称为更新属性 upd, 该属性可能对任何单元格有定义。

对单元格 C , $C.upd$ 是一个序偶构成的集合 $\{(dt_1, u_1), \dots, (dt_n, u_n)\}$, 其中 dt_i 是个数据表, 集合 $\{dt_1, \dots, dt_n\}$ 称为 C 的更新数据表, 记作 $C.dt$, 而 u_i 又是一个序偶构成的集合 $\{(f_1, x_1), \dots, (f_m, x_m)\}$, 其中 f_j 是 dt_i 的字段, 集合 $\{f_1, \dots, f_m\}$ 称为 dt_i 的更新字段, 记作 $dt_i[C].fld$, 满足 $dt_i.PK \subseteq dt_i[C].fld$; x_j 是一个计算串, 称为 f_j 的更新表达式, 记作 $f_j[C][dt_i].exp$ 。

与一般单元格属性不同, 我们还特别规定允许为 @ 设置更新属性。

上述记法中[]及其内的部分能在上下文中确定时常常省略不写。

更新属性一般在设计源表时设置(不一定所有单元格都有该属性),作为单元格属性,更新属性在报表计算时会一起跟随扩展过程被复制到导出格,从而使实例表上某些同源格也有更新属性。

基于更新属性,我们设计如下的数据更新算法。

算法 4.6 实例表数据更新算法。

设有实例表 T , 在 T 上修改某些单元格格值后的报表记作 T' :

1. 对所有有更新属性的单元格 c (包括@), 做:
2. 对所有数据表 $dt \in c.dt$, 做:
3. 设 $dt.PK=(p_1, \dots, p_k)$, 令 $v=(v_1, \dots, v_k)$, 其中 $v_i = \text{eval}(f_i.exp)$ 满足 $f_i = p_i$, 计算在 T 上进行并以 c (原格) 为基格;
4. 设 $f=dt.fld=(f_1, \dots, f_m)$, 令 $u=(u_1, \dots, u_m)$, 其中 $u_i = \text{eval}(f_i.exp)$, 计算在 T' 上进行并以 c 为基格;
5. 如果 $dt.SELECT(v) > 0$, 则执行 $dt.UPDATE(v, f, u)$, 否则执行 $dt.INSERT(f, u)$ 。

数据更新算法并不关心实例表上单元格的格值是否被修改,可以在任何时刻执行以在数据表中保存数据,并且在过程并不考查格值的计算方案(即数据来源),而且同时可将同一批格值保存到多个数据表中,从而做到数据来源与去向无关,并可有多处去向,通俗地说,就是报表上数据的有来有去、来去无关、一来多去。这是许多实际应用中填写报表的要求。

另外,更新属性可以定义在包括@在内的任何单元格上,这

样可能设计出非常丰富的填写方案。如定义在@（或其属格）上时可形成整表对应单条记录的自由式填写；而定义在某扩展格（一般是纵主格）辖区内单元格上时可形成每行（列）对应一条记录的行列式填写（这是最常见的情况）；定义在横、纵主格辖区交集内单元格上时可形成每格对应一条记录的交叉式填写；而且这几种填写方式可以在同一个实例表内混合出现。

报表 T 上所有单元格的更新数据表的并集称为报表的更新数据源，记作 $DT[T]$ 。 $|DT|>0$ 的报表称为填报表， $|DT|=1$ 的报表称为单源填报表， $|DT|>1$ 的报表称为多源填报表。

在现实应用中进行数据更新时，除上述算法中处理过的直接向数据表保存数据的情况外，有时还可能通过存储过程（或某些别的程序过程）来保存数据，但方法基本一致，这里不再赘述。

除数据保存外，修改实例表上格值的过程中还有一些功能性的要求，如某个单元格的格值将在其他单元格的格值改变时根据某个事先约定的表达式自动计算出来，某批单元格的格值必须满足某种事先规定好的逻辑关系而不能随意改变，填写有误时应及时提醒，……；这些功能要求通称为前端响应（这只是通俗说法，而非严格定义）。

无论处理前端响应还是数据更新都需要面临在报表扩展后再计算表达式（计算串）的问题，原则上计算串可以是任何符合前述语法规则的表达式。但是在实际应用中，报表的填写经常发生在处理能力相对较弱的环境下（如 WEB 页面），此时如果完整地记录单元格间的主子关系并实现层次坐标等计算将导致实现成本过高和效率过低，为此我们需要一种更简单而易于实现的表

达式规则。

将计算串中出现的单元格和格集均替换成其在实例表中的平面坐标,得到的新的计算串称为平面计算串,将计算串转化成平面计算串的过程称为平面化。平面化过程中单元格的替换比较简单,直接换成平面坐标即可,而格集则比较麻烦一些,将连续一片的矩形可写成 $a:b$ 的形式,不连续的格集则只能将其中所有单元格罗列出来。

平面计算串中的单元格名称上已具有方位信息,无须知道单元格的主子关系即可快速定位单元格,也无须计算复杂的层次坐标,因而可以方便地在处理能力较弱的 WEB 环境下应用。

平面计算串支持下的报表填写过程称为平面填写。

平面化还有一个额外的用途,将实例表转换成 EXCEL 格式(这是很常规的现实需求)时可以带出部分设定好的计算公式,使得生成的 EXCEL 文件仍具有连动计算的能力。

5.3.3 线性填写

上面讨论的填写假定了在过程中报表的布局不再变化,而仅是修改其中的数据(格值),填写过程可以看成是静态的。但有时现实应用要求在填写的同时能够改变报表的布局(增删记录),也就是动态填写。我们将面临在填写过程中继续扩展及带来的计算串自动调整问题和填写完毕后数据更新的删除问题。

具体来讲,填写过程中的布局变化是指增加删除报表行列;而考虑到记录的完整性,我们约定:

规则 4.6 报表填写过程中的增加和删除操作只能针对叶子主格进行,且一次增加和删除的行列必须是该主格的完整

扩展带。

理论上我们仍可以重新执行前面讲述的算法处理有增删操作的填写。但是, 基于采用平面计算串的同样理由, 我们需要设计一种可在处理能力较弱的环境下工作的简化模型。

仅主格的更新属性有定义的整报表称为整填报表。由于更新属性只能定义在主格上, 因此不可能用整填报表实现交叉式填写。

我们下面只考虑整填报表在填写过程中的增删问题, 在实际应用时经常甚至还会限制为只能单向报表。

报表增加操作与扩展带复制算法类似, 这里不再重复; 而删除操作也只要简单地将相应扩展带中的行列删除(算法 2.2)即可。但在填写过程中的删除动作却仅是在叶子主格上作个标记(相当于为叶子主格设计一个名为删除状态的单元格逻辑属性), 并非真的删除(在实际应用中一般还会采取某些处理令该主格的扩展带不再显示), 要在数据更新时才实际执行删除动作。

算法 4.7 有增删动作的数据动态更新算法

设有实例表 T , 对 T 做了增删动作及单元格格值修改后得到的报表记作 T' :

1. 对 T' 上所有有删除标记的叶子主格 c , 做 (2、3 步):
2. 如果 c 有更新属性且 c (原格) 在 T 上存在 (即 c 不是在填写过程中复制出来的复制格), 对所有数据表 $dt \in c.dt$, 做:
 - a) 设 $dt.PK=(p_1, \dots, p_k)$, $v=(v_1, \dots, v_k)$, 其中 $v_i = \text{eval}(f_i, \text{exp})$ 满

足 $f_j = p_i$, 计算在 T 上进行并以 c (原格) 为基格;

b) 执行 $dt.DELETE(v)$;

3. 在 T' 上针对 c (原格) 执行删除操作 (删除 c 的扩展带内的行列);
4. (这一步起相当于调用执行数据更新算法, 但有少许差异) 对所有有更新属性的单元格 c (包括 @), 做:
5. 对所有数据表 $dt \in c.dt$, 做:
6. 如果 c 在 T 上存在, 则设 $dt.PK = (p_1, \dots, p_k)$, 令 $v = (v_1, \dots, v_k)$, 其中 $v_i = eval(f_j.exp)$ 满足 $f_j = p_i$, 计算在 T 上进行并以 c (原格) 为基格;
7. 设 $f = dt.fld = (f_1, \dots, f_m)$, 令 $u = (u_1, \dots, u_m)$, 其中 $u_i = eval(f_i.exp)$, 计算在 T' 上进行并以 c 为基格;
8. 如果 c 在 T 上存在并且 $dt.SELECT(v) > 0$, 则执行 $dt.UPDATE(v, f, u)$, 否则执行 $dt.INSERT(f, u)$ 。

在动态填写过程中仍然会面临计算串的计算问题, 但此时平面计算串已无能为力, 我们无法在报表扩展带增删后重新计算出一个平面计算串 (平面化工作经常会依赖于层次坐标, 而采用平面计算串的主要目的恰恰是为了在填写过程中避免掉层次坐标)。同样理由, 我们还需要设计一种能在简单环境下应用的计算串规则。

满足如下条件的计算串 (表达式) 则称为线性计算串:

1. 所有的层次坐标都是零层次坐标;
2. 格集中没有过滤条件;
3. 没有线性坐标和平面函数。

由于 1 的限制, 根据定理 4.2, 采用线性计算串只能引用单

元格导格的先辖区内的单元格。

线性计算串的获得无法通过某种类似平面计算串的平面化工作自动完成，只能在源表设计时就加以限制，即约定：

规则 4.7 填写时可增删的整填报表上所有计算串必须都是线性计算串。

规则 4.7 只限制报表填写时的计算串规则（如更新表达式和用于前端响应的某些属性），但并不限制其中格表达式的规则。

线性计算串支持下的整填报表动态填写过程称为线性填写。

线性计算串和线性填写的实现较为简单，在实际应用中，其保留下的能力对于绝大多数需要动态填写的报表已足够。与平面计算串相比，线性计算串实现时占有的空间要少得多，无须将每个单元格的计算串事先计算出来保存，可以实时计算，但处理能力要弱。因此，平面计算串适合处理格式运算复杂但容量较小（单元格数量少）的报表，而线性计算串则适合处理格式运算简单但容量大的报表。

利用线性计算串的简单性，完全采用线性计算串的整报表不仅能应用于可增删填写，用于巨大报表的计算中也可以起到提高运算性能和减少存储量的作用。

6 总 结

我们将本文中讨论的报表数学模型总称为非线性报表模型，与之对应的还有线性报表模型，也就是目前业内许多传统报表工具软件所采用的数学模型。

无论线性报表还是非线性报表，严格来讲并不是指某种类型的报表，而是一种设计和制作报表的理论体系。但有时我们也习惯于将能基于线性报表模型设计的报表称为线性报表，而需要基于非线性报表模型设计的报表称为非线性报表。所有线性报表均可用非线性报表模型设计，故也是非线性报表，但反之不然。

线性报表模型是基于数据库理论发展出来的，其核心是对数据集做分组计算后将数据填入表格中，单元格等表格布局方面的概念则相当弱化。运算几乎全部集中在有极强规律性的数据集上，导致表格的变换和计算也被迫变得规律性很强，难以设计出复杂多样的报表布局和灵活随意的计算引用关系。

非线性报表模型则完全不同，我们深入研究并充分利用了表格本身的固有特征，使单元格成为整个理论体系的核心，数据集则仅作为某种提供数据来源的从属属性存在。在非线性报表模型下，几乎所有变换和计算都以表格为主体进行，从而可以方便地设计出具有复杂样式和灵活计算的报表。

非线性报表模型并不是从线性报表模型基础上发展出来的，两种模型的基础不同，也无法精确地用某种理论的术语解释另一

种，这里只能作些通俗的描述。

延用非线性报表理论的术语，线性报表大体上可以被理解成为如下几类报表的交集（其实还是这个交集的真子集）：

1. 整报表；
2. 单片报表；
3. 单源报表；
4. 其上所有的表达式都是线性计算串的报表；
5. 没有符号循环引用现象的报表；
6. 单向报表；

而且，即使满足上述所有限制的报表也不一定能基于线性报表模型设计出来（其中的计算仍不具有我们前面叙述的那种灵活性），另外，线性报表模型在填写方面也几乎是空白。虽然目前有部分传统产品发展了线性报表模型，在 2、3、6 方面有所空破，即能够处理一部分多源报表、分片报表和交叉报表，也有极个别产品能够处理非常简单的单源填报表（行式填写）。但无论如何，线性报表仍是比非线性报表简单得多的理论体系。

当然，简单也有效率高的优势。对于能用线性报表模型描述的报表（虽然远比非线性报表模型可描述的要少），其运算时间和占用存储都比非线性报表模型要少得多，在报表巨大但形式简单时仍有一定的应用价值。

本文中的讨论非线性报表理论仅是初步的内容，仍有许多问题可以进行深入研究和思考，例如：

1. 布局方面
 - a) 有限扩展：扩展带可能仅是个矩形而不是整个行/列；
 - b) 交替扩展：如纵主格可能有横子格；

c) 分页处理;

2. 计算方面

a) 格间关联模型与统计图: 有些格集之间可建立对应关系;

b) 允许属性引用时如何解决循环引用问题;

3. 填写方面

a) 非叶子主格的动态填写

b) 交叉动态填写: 动态填写时更新属性不一定只对主格有定义;

4. 算法的高效实现方案

这些问题都有很实际的应用背景, 解决它们对于拓广报表应用领域和改善应用体验有重大的现实意义, 我们将在后续的工作中作进一步研究讨论。

参 考 文 献

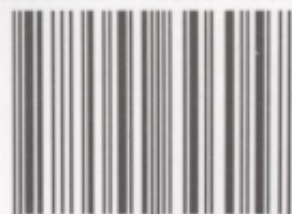
- [1] Martin D. Davis, Elaine J. Weyuker 著. 张立昂, 陈进元, 耿素云 译. 可计算性 复杂性 语言. 北京: 清华大学出版社, 1989.
- [2] 陆钟万著. 面向计算机科学的数理逻辑. 北京: 北京大学出版社, 1989.
- [3] В. А. Зорич 著. 蒋铎, 王昆扬, 周美珂, 邝荣雨译. 数学分析. 北京: 高等教育出版社, 1987.
- [4] 莫宗坚, 蓝以中, 赵春来著. 代数学. 北京: 北京大学出版社, 1986.

责任编辑：赵 晖

责任印制：安利平

封面设计：金柏阳光广告公司

ISBN 978-7-5046-4605-7



9 787504 646057 >

定价：18.00元